Cathedrals Opening Bazaars: Firms' Open-Source Projects and Community Participation*

Xiaofeng Gong

This version: November 6, 2026

Abstract

This paper studies how participation in firms' open-source projects relates to the technologies the firms patent. I link Compustat firms to their USPTO patents and GitHub accounts and use a large language model to place each project in the Cooperative Patent Classification, the scheme patent examiners use. I document that most of these projects lie in technologies in which the firm was recently granted patents. I then compare projects first active in 2015–2022, conditional on firm-by-year and technology-by-year effects and on controls for each project's early activity, text and language. I find that projects in the firm's patented technologies have more new participants and more contributions from them. The difference in new participants holds for people with no earlier activity on the firm's GitHub accounts and without the five largest firms. Contributions from everyone are also higher, and among projects with a contribution there is no clear difference in new participants' share. According to my estimates, projects in the firm's patented technologies have 29.5 percent more new participants in the 24 months after their first active quarter, 45.5 percent more contributions from them, and 37.6 percent more contributions from everyone.

1 Introduction

A firm can patent inventions and publish open-source code in similar technologies, and the open-source code sometimes attracts people to contribute. When firms publish code in the technologies they patent, who participates, and how much do new participants contribute to the firms' projects? Firm-level totals cannot show whether participation is higher in the projects that overlap a firm's patented technologies, so I make the comparison repository by repository, with code and patents in one classification. Repositories in the technologies where their firm was recently granted patents have 29.5 percent more new participants and 45.5 percent more contributions from them in the 24 months after their first active quarter. These are associations, conditional on effects for each firm and year and for each technology and year and on controls, and the estimates leave open why.

*The image of the cathedral and the bazaar is from Raymond (2001), where the two are styles of building software. Casting a firm that patents as the cathedral is this paper's adaptation.

Raymond (2001) contrasts two ways of building software. In the cathedral, a small group works in isolation and releases rarely. In the bazaar, code is released early and often, and many people take part. A large firm that files patents plays the cathedral, and the open-source code it publishes, which anyone can read, use and build on, enters the bazaar. Projects run by companies account for a large share of public open-source code (Spinellis et al., 2020).

Firms choose what to reveal and what to keep secret. Patenting can reveal information that raises a competitor's profits (Horstmann et al., 1985), and firms also reveal some of the code they develop and keep the rest proprietary (Henkel, 2006). Firms in every industry protect knowledge by means other than patents (Cohen et al., 2000), and what a firm shares and what it protects is a question in the economics of appropriability (Teece, 1986).

As in the bazaar, people new to the firms' projects take part in them. The software-engineering literature studies how newcomers join projects (Pinto et al., 2018; Casalnuovo et al., 2015). I describe participants by their earlier activity on GitHub, and no actor is matched to an employer. The repositories studied are the 102,987 repositories of the firms defined below, first active in 2015–2022 (section 3.4). Of these repositories, 53.5 percent have at least one *new participant*, a person who first contributes to the repository in its first two years, and 41.1 percent have one with no earlier activity at the firm on GitHub (section 3.3). GitHub's public record shows the work people leave on a repository, while its users stay unseen, and the new participants are the users whose first work on it falls in those two years (section 3.3). The perspectives the paper draws on point both ways. One line of work expects less participation in projects tied to patented inventions, and others expect more under conditions they state (section 2).

Two gaps in the data kept a firm's patents and its open-source projects apart. Open-source code carries no patent class, so setting a firm's projects beside the technologies of its patents needs a classification of the code. Compustat and the patent records list no GitHub accounts, so each study builds its own link (Cao et al., 2026). Studies of corporate open-source code therefore relate how much code a firm opens to its patents, across firms (Fosfuri et al., 2008; Cao et al., 2026), or follow contributions around a dated event (Chen et al., 2022; Haese and Peukert, 2026). This paper adds a comparison of each repository with its own firm's patents. The data built here close both gaps. Crosswalks link firms to their patents and to their GitHub accounts, and a large language model (LLM) labels the repositories with the main groups, the detailed patent classes, that examiners assign to patents. Each labeled repository can then be set against its firm's own patents in one classification, repository by repository.

I link Compustat records to their USPTO patents and their GitHub repositories, and I label each repository with main groups of the Cooperative Patent Classification (CPC), the detailed classes that patent examiners assign (section 3). A person participates in a repository with a contribution, which is an issue, a pull request, a review or a comment, in GH Archive's record of GitHub's public events (section 3.3). I call a repository *inside* when at least half of the weight of its CPC labels lies in main groups where its firm was granted patents in the five grant years up to and including the year of its first activity, so that grants later in that year count, and I call those main groups

the firm's *patented main groups*. Inside records shared technology classes, not whether a patent covers the repository's code. Of the 102,987 repositories studied, 82,046 (79.7%) are inside, so the comparison is with the minority of the firm's repositories that lie outside its patented main groups. The main estimates use the labeled repositories of the firms with at least five patents and five labeled repositories in a five-year window ending in any year of 2019–2024, and one column takes every firm linked to a repository and a patent (section 3.4).

The paper makes three contributions. The first is a method that places open-source code in the patent classification, repository by repository. An LLM reads each repository's text and assigns it main groups in two stages, first subclasses, the level of the classification above the main group, and then the main groups under them (section 3.2). I check the method against two references built independently of it. The labels share at least one main group with those of the European Patent Office's (EPO) text categorizer, a tool that assigns patent classes to text, for 76.4% of the repositories both classify, against 38.5% when the label sets are dealt out at random among repositories of the same CPC section, the classification's broadest level. Run on patents, the same two stages with another LLM, DeepSeek-V4-Flash, give labels that share a main group with the examiners' classification for 89.1% of them, against 20.8% when the model's label sets are dealt out at random among patents of the same CPC section (Appendix B.1). Ganglmair and Kann (2025a) place research disclosures in patent classes, and the May 2026 draft of Won (2026) infers a CPC profile for a released project. Here each repository of the firms in the data carries labels of its own (Appendix B.5).

The second contribution is the data. They link Compustat records to their patents and to their open-source code, and unlike the links that earlier studies built for their own use, each link records its supporting evidence and the year in which the firm owns the account. I group each firm with its subsidiaries under the top firm in its ownership tree that has a Compustat record, which holds both the patents and the GitHub organizations, and a repository belongs to the firm whose GitHub organizations hold it in the year of its first activity (section 3.1). The patent crosswalk covers 99.7% of the 2015–2021 patents that DISCERN (Arora et al., 2021) links to firms headquartered in the United States, and 96.3% of the 2015–2024 patents that the extended data of Kogan et al. (2017) link to CRSP firms. Both are among the crosswalk's inputs, so that agreement is partly built in. The Global Corporate Patent Dataset (GCPD) of Bena et al. (2017) and Barkemeyer et al. (2026) never entered the crosswalk, and where both link a 2015–2022 patent to a Compustat firm, GCPD names the same firm for 92.8% of the patents. The account crosswalk names the same firm as the commit-email data of Spinellis et al. (2020) for 93.9% of the accounts both hold. Among patenting firms with no accepted account, a search of GitHub by name finds an account with a matching name for 39.4% of them, which bounds from above the share of those firms whose accounts the crosswalk misses, provided the name search finds every such account (Appendix A.4).

The links also follow the firms' projects into US patent documents, whose references cite 0.65% of these firms' projects, against 0.179% of the projects of other firms in Compustat with the same mix of creation year, stars and language (Appendix D.1). This fact shows that the links reach

patent documents, and it enters no estimate of participation.

The third contribution is the result. Conditional on firm-by-year and main-group-by-year effects and controls for the repository's early activity, text and language, inside repositories have 29.5 percent more new participants in their first two years than other repositories (95 percent interval 14.6 to 46.4), and I call this difference the *inside difference*. Unlike the studies across firms cited above, the data here compare repositories by whether they lie in their own firm's patented main groups. Inside repositories also have 45.5 percent more contributions from new participants, counting their issues, pull requests, reviews and comments, and 37.6 percent more contributions from everyone, which is more activity overall, and among repositories with a contribution there is no clear difference in new participants' share of contributions (95 percent interval −0.52 to 2.55 points). For new participants with no earlier activity at the firm on GitHub, the inside difference is 27.6 percent. The paper's economic contribution is to document how participation aligns with firms' patented technologies, and project importance, audience and corporate support remain possible explanations of this conditional association. With firm-by-year and main-group-by-year effects, the main estimate compares a firm's repositories inside its patented main groups with its other repositories, and within a main group and year, one firm patents the main group and another does not, so the comparison is also partly between repositories of different firms (section 4.2).

Section 2 states what published perspectives predict for the inside difference. Section 3 builds the comparison from the linked data, and section 4 reports it, its main estimate first.

2 Literature review

The question is whether firms' open-source code in the technologies they patent draws more or fewer new participants, and the published perspectives point both ways. None of the perspectives below names patent classes or repositories, so each prediction for code in the firm's patented main groups states how the perspective maps to repositories.

2.1 Perspectives that predict less participation

Lin and Rai (2024) and the work they summarize predict less participation in code tied to patents. They suggest that patenting software could deter open communities from contributing, and they summarize earlier work as finding that projects tied to patented inventions attract less interest from users (Bar-Gill and Parchomovsky, 2003; Wen et al., 2013). They expect less participation at firms that put more weight on patenting. Wen et al. (2013) predict that user interest and developer activity fall in projects related to the technology that an enforcement action concerns, although this paper observes no enforcement. Under its mapping to repositories (Appendix Table 22), this line predicts fewer new participants in inside repositories.

2.2 Appropriability, products and visible contributions

Theory gives several reasons for firms to release open-source code. Firms may take their profit in a proprietary segment complementary to the released program (Lerner and Tirole, 2005), and some firms profit from products and services complementary to open-source products (Athey and Ellison, 2014). A sponsor may open a platform to drive its adoption and to harness outside developers (Economides and Katsamakas, 2006; Parker and Van Alstyne, 2018). Users in open-source communities typically reveal their innovations freely (von Hippel and von Krogh, 2003), and firms that contribute gain more from their use of open-source code than peers that do not (Nagle, 2018). Developers make open-source code available to everyone on equal terms, which Harhoff et al. (2003) call free revealing.

The first perspective rests on appropriability. Teece (1986) ties an innovator's ability to capture the profits from an innovation to its appropriability regime, and in his view commercializing an innovation needs its know-how to be used together with other capabilities or assets. Later work carries this to the release of code. Alexy et al. (2013) define selective revealing as the release of knowledge that the firm could have kept proprietary, and Henkel (2006), citing Teece, names complementary assets as a means of appropriation that works whatever the license of the released code. Fosfuri et al. (2008) argue that a firm's stock of software patents can be such a complementary asset to its open-source products. Under its mapping to repositories (Appendix Table 22), the firm's patents mark where its code is better supported, and this perspective predicts more participation in inside repositories. These sources explain what firms release, not who takes part in it, and Fosfuri et al. (2008) note that a patenting firm's tight control of a project might discourage independent developers. On SourceForge, Belenzon and Schankerman (2015) study who takes part in open-source projects from observed membership and contributions, and they find that developers sort on a project's license, size, intended audience and corporate sponsorship.

The second perspective concerns code tied to products. Upstream knowledge can be public while the downstream products built on it are protected (Gambardella and Hall, 2006), and commercial firms support open-source projects (Athey and Ellison, 2014). Appendix Table 22 gives this perspective's mapping to repositories.

The third perspective concerns visible contributions. Theories of participation explain why developers contribute, for delayed rewards that signal their talent (Lerner and Tirole, 2002) or out of reciprocal altruism (Athey and Ellison, 2014). In Lerner and Tirole (2002), the signal is stronger the more visible a performance is to the relevant audience and the more attributable it is to one person, and in open source, outsiders can see what each person contributed and whether it worked. On GitHub, an issue or a pull request carries work that others can see and judge, since a pull request makes a change visible for review (Kalliamvakou et al., 2015), actions on the platform are public, and others read a person's actions there as signals of competence (Dabbish et al., 2012; Pinto et al., 2018). A star carries no such work. It is tied to an account, but developers give stars mainly to show appreciation or to bookmark a project (Borges and Valente, 2018), and studies count them as popularity, apart from contributions (Pinto et al., 2018). Under its mapping to repositories

(Appendix Table 22), this perspective predicts a larger inside difference on issues and pull requests than on stars.

2.3 Related studies

Empirical studies relate firms' open-source code to their patents across firms. Firms with larger stocks of software patents release more open-source products (Fosfuri et al., 2008). Lin and Rai (2024) find that software firms' open-source activity rose after the Supreme Court's 2014 *Alice* decision narrowed the scope of software patents, mainly among firms with a large stock of software patents beforehand. Other studies measure firms' use of and investment in open-source code (Gortmaker, 2025), place firms in a patent-technology space (Habibi, 2025), relate firms' patent profiles to the level of their GitHub activity (Cao et al., 2026), and value open-source projects from market reactions (Emery et al., 2025).

The closest papers on firms' code and patents relate the two in other ways. Appendix D.1 sets Petralia (2025) and Petralia and Boschma (2026), on patents that mention a project, beside this paper's counts of projects cited in patent documents. For the participation result, Lin and Rai (2024) relate a firm's open-source activity to the scope of software patents across firms and industries. Here repositories are compared by their overlap with their own firm's grants, and the comparison is partly across firms. Wen et al. (2016) relate start-ups' entry with open-source products to IBM's pledge of patents, market by market and year, through a concordance from patent classes to software markets. Here the tie is drawn repository by repository within the firm, with no dated event. Chen et al. (2022) and Haese and Peukert (2026) follow contributions by a firm's own developers and by outsiders around a dated event, an acquisition or Microsoft's move to Chromium, and assign developers to firms by membership lists or email domains. Here no event is used and no actor is matched to an employer.

3 The data and the Cooperative Patent Classification

I link Compustat firms to their open-source code on GitHub and to their USPTO patents, and an LLM assigns patent classes to the code. Which repositories are compared, and how is each placed against its firm's patents?

3.1 Firms and their repositories

I start from the firms in Compustat and combine four sources (Appendix A.1) at the level of the firm defined below. Compustat supplies the list of firms and each firm's industry. A *Compustat record* is one firm's entry in Compustat, which may be a subsidiary's. The *Compustat record firm* is the firm with a Compustat record at the top of the ownership tree, together with its subsidiaries (Figure 1). It holds both the patents and the GitHub organizations of the firm and its subsidiaries. A repository belongs to the Compustat record firm whose organizations hold it in the year of its first

recorded activity, and the repository's participants are the people who contribute to it (section 3.3). From here on, *the firm* means the Compustat record firm.

The *linked firms* are the firms that the account crosswalk, which links Compustat records to GitHub organizations, links to at least one organization (Appendix A.4). The firms studied are the linked firms with at least five patents and at least five labeled open-source projects in a five-year window ending in any year of 2019–2024 (section 3.4). I call them the open-source-software-active firms, or *OSS-active firms*.

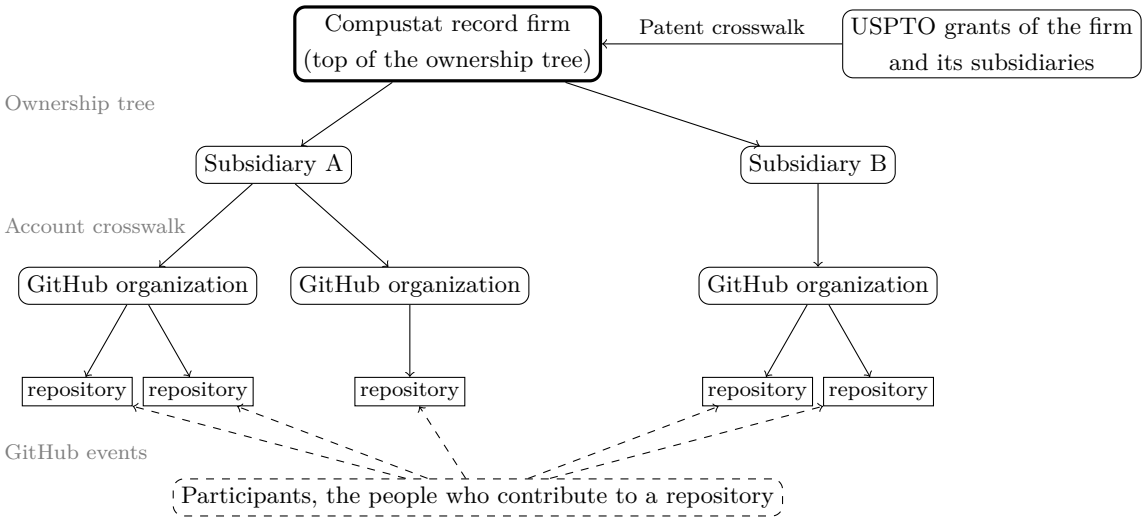


Figure 1: From a Compustat record firm to its repositories and their participants

Notes: The Compustat record firm is the firm with a Compustat record at the top of the ownership tree. The patents of the firm and its subsidiaries are assigned to it through the patent crosswalk, and the account crosswalk links each GitHub organization to the Compustat record that runs it (section 3.1). A repository belongs to the Compustat record firm whose organizations hold it in the year of its first recorded activity. Its participants are the people who contribute to it (section 3.3).

GitHub was the largest host of source code (Conti et al., 2025) and by far the most popular platform for open-source development (Lin and Rai, 2024). An account on GitHub belongs to a user or to an organization, and as in Conti et al. (2025), the accounts linked here are mostly organization accounts (Appendix A.4). A repository holds the code and history of a project under an account, though a repository is not necessarily a project (Kalliamvakou et al., 2016). I use repository and project interchangeably. The events come from GH Archive, a public record of GitHub's public timeline (Conti et al., 2025), and a project's activity is its dated events from 2015 onward.[1] Events are extracted only for the GitHub organizations that the crosswalk links to firms, and I call an event in that extract a *held* event. The account crosswalk links the firms to their organizations, as Cao et al. (2026) link Compustat firms to theirs. Appendix A.2 lists what the GitHub data leave out.

Neither Compustat nor the patent records list a firm's GitHub accounts, so I build the link

[1]The events include pushes, issues, pull requests, releases, stars and forks of the project. The archive's yearly files do not correspond to calendar years. They lack most of January 2018, 2019 and 2020 and include January 2021 twice, and I use corrected event dates throughout.

from firms to their open-source code. The account crosswalk also dates its links. *Dated ownership* is a record, for each year, of the firm that owns a project's account, and so it follows repositories through acquisitions and the ends of firms' records. A second crosswalk, built separately, links Compustat records to PatentsView assignee identifiers. An LLM also takes part in both crosswalks. Appendix A.4 describes the search for the accounts, checks both crosswalks against published ones and follows the counts along the chain.

3.2 Classifying repositories by patent class

The patents are the firms' utility grants of 2015–2024 from PatentsView, the United States Patent and Trademark Office's (USPTO) public data, each dated by its grant year. The data hold grants from 2015 on. Every timing rule below uses grant years, and no version on application years is estimated. The Cooperative Patent Classification (CPC) runs from sections through subclasses and main groups to subgroups. A patent's inventive CPC codes are those marked as inventive information, as distinct from additional information, and each patent's weight is split equally across the main groups of its inventive codes. A main group is one of the firm's patented main groups for a repository when any of the firm's grants in the five grant years up to the year of the repository's first activity carries weight in it, however small (section 4.1). Because the data hold grants from 2015 on, the patented main groups of the earliest repositories rest on fewer than five grant years.

Open-source code carries no patent classification, so I assign patent classes to compare it with a firm's patents. A prompted LLM can assign main groups to any project without word lists or labeled examples. The model is GLM-5.3-Flash, which I call *the labeler*, and it assigns the repository's text to classes of the classification used for patents. It reads the project's name, description, topics, programming language and README, with long READMEs truncated. It labels in two stages, first CPC subclasses and then the main groups under them (Figure 2), and Appendix B.4 gives each stage. Two other LLMs serve only as checks. DeepSeek-V4-Flash labels patents, so that its labels can be compared with the examiners' (Appendix B.6), and it also labels a sample of repositories (Appendix C.3). An anonymous model served under the name `space-bunny-alpha`, which I call *the second labeler*, labels a sample of the repositories (Appendices B.1 and C.3).

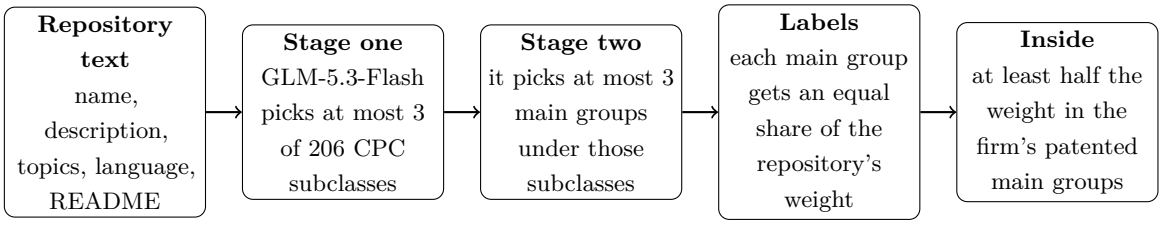


Figure 2: How a repository is labeled

Notes: The model reads the text of the 209,352 projects linked to firms, as it stood in 2026, at temperature zero. Either stage may return no applicable class, which leaves the repository unlabeled, and 188,899 of the projects receive at least one main group. The firm's patented main groups are those of its grants in the five years up to the repository's first-activity year (section 4.1).

A repository is inside when at least half of its label weight lies in its firm's patented main groups. Each main group the model returns carries an equal share of the project's weight, so the ranking of the labels leaves their weight unchanged. Of the 209,352 projects linked to firms, 188,899 receive at least one main group. The model reads the repositories' text as it stood in 2026, after every outcome window. I define inside at the level of the main group because the firms' open-source code is concentrated in a few classes (Appendix A.7). At the level of the CPC subclass, G06F alone holds 72.56% of the firms' open-source code, so most repositories of a firm that patents in G06F would share a class with its patents whatever they do. At the main group, the same code spreads over 676 main groups and takes 11 of them to reach 80 percent.

One repository, the running example, shows the steps (Appendices A.9 and B.8). The repository `facebookresearch/fastMRI` belongs to Meta Platforms, one of the OSS-active firms, and its first activity falls in the fourth quarter of 2018. The labeler read its name, description, topics, programming language and README, and returned three main groups from the subclasses G06N, G06T and G16H, each with a third of the repository's weight. They are G06N 20/00 (machine learning), G06T 5/00 (image enhancement or restoration) and G16H 30/00 (handling of medical images). Meta was granted patents in G06N 20/00 and G06T 5/00 in 2015–2018 and none in G16H 30/00, so two thirds of the repository's weight lies in Meta's patented main groups, and the repository is inside.

I check the labels against two references built independently of them (Appendix Table 13). The first reference is the European Patent Office's text categorizer, a tool that assigns patent classes to text, run on the same repositories. The second is the examiners' classification of patents. For it, DeepSeek-V4-Flash applies the same two stages to the titles and abstracts of patents, and its labels are set against the classes the examiners assigned, as Lee and Hsiang (2020) score a patent classifier. A set of labels agrees with a reference when the two share at least one main group. The checks measure agreement with other classifications rather than accuracy, and Appendix B.1 reports them in full and shows how far a label error can move a repository across the inside threshold.

3.3 New participants and the outcomes

Participation is measured from GH Archive's record of GitHub's public event stream, held for the organizations the crosswalk links to firms. The held record has one row per organization, repository, event type, calendar quarter and actor, with the number of such events. The record holds fifteen event types, which fall into three families (Appendix Table 8). Attention is starring and forking. Contribution is opening, closing and commenting on issues, opening, closing, reviewing and commenting on pull requests, and commenting on commits. Write and administration covers pushes and the other actions that need write access to the repository.

Participation in a repository is a contribution-family event on it by an actor who is not a bot, and the repository's *participants* are the people who contribute to it in this way. Wright et al. (2023) measure participation by new commits, but these records hold commits only inside push events, which fall in the write family. I do not count pushes and the other write events as participation,

because they need write access to the repository, which the firm grants, whereas contribution-family events are open to anyone with a GitHub account. Write events enter two of the outcomes below only, whether new participants keep working on a repository and whether the repository is still updated, and no estimate counts pushes as participation.

The held record shows the work people leave on a repository and some signs of attention, its stars and forks, while reading, downloading and running its code leave no trace in it. Studies of user innovation describe a project's contributors as the part of its users who go on to post or change code, beside users who take the code and contribute nothing (von Hippel, 2005), readers of a help forum who never post (Lakhani and von Hippel, 2003) and the silent subscribers of a project's mailing lists (von Krogh et al., 2003). In these terms, a repository's participants are the users who leave contribution-family work on it. A screen of actors' names and behavior flags bots, which are dropped from every count (Appendix A.1).

A *new participant* on a repository has no held event of any type through its first-activity quarter, and makes a first contribution-family event on it in the following eight quarters. A repository's clock starts at its *first-activity quarter* q_0, the first calendar quarter with a held event by an actor who is not a bot, which the text also calls the repository's first activity. The year of q_0 is the repository's *first-activity year* y_0. The *window* is the 24 months after q_0, quarters $q_0 + 1$ to $q_0 + 8$.

I fix the window's length so that every repository is counted over the same span of its life, whatever its year of first activity, and a repository first active early does not gather more participants only because it is observed longer. Two years is the longest span the records cover in full for every repository first active through 2022, since they end in 2024. On the repositories studied that were first active in 2015–2020, whose four years after q_0 are all held, the window holds 53.0 percent of the people who first contribute to the repository within those four years, just over half, and between 49.8 and 57.0 percent in each year of first activity. Medappa et al. (2025) likewise judge a GitHub project's survival over the two years from its start, a horizon they base on earlier evidence that many open-source projects stop within their first two years. Pinto et al. (2018) count the newcomers to company projects in the six months after the projects become public. No study among those cited here counts new participants over this window, so its start at first activity, its length and its quarterly grain are this paper's own choice.

The first-activity quarter is the first held event under a linked organization, not the repository's creation. The regressions carry indicators for a repository created in the four quarters up to q_0 and for one with no creation record (section 3.4), and Appendix A.4 counts each case. Actors with a held event of any type on the repository in q_0, stars and forks included, are its *founding-quarter actors*. I do not count them as new, because quarterly data cannot order events within q_0 and so cannot separate them from the people who published the repository. Inside repositories have more of them on average (Table 3), and the regressions control for their number.

A new participant's *arrival* is their first held event of any type on the repository, which can come before their first contribution, and I count arrivals as person–repository pairs. The software-engineering literature calls such people newcomers (Pinto et al., 2018; Casalnuovo et al., 2015).

New participants follow the bazaar of Raymond (1998), in which a project's users become its co-developers. Raymond describes the early Linux community as "a great babbling bazaar of differing agendas and approaches," and he writes of a project's users that, "properly cultivated, they can become co-developers." In the layers that Crowston and Howison (2005) describe around a project's core developers, co-developers submit patches and active users report bugs, while passive users "do not speak on the project's lists or forums." The contribution family records the work of co-developers and active users, and the passive users are those the held record leaves unseen.

Several studies of open-source projects, firms' projects among them, count new contributors as part of the community around a project and describe its continued activity as depending on them. Pinto et al. (2018) write that "going open source means creating, maintaining, and fostering a community around technology," for example by attracting new developers, and they count the new contributors to company projects after the code becomes public. Haese and Peukert (2026) study "the open source community behind" a firm-led project through its external contributors, among them newcomers with no history of contributing to it. Steinmacher et al. (2015) write that open collaboration communities "leverage and depend on new contributors," and Zhou and Mockus (2015) that projects "need ongoing contributions from new and existing participants." Valiev et al. (2018) count a project as sustainable from a community perspective if it attracts and retains newcomers, and in their survival models of Python packages, packages with more contributors are less likely to become dormant. Yamashita et al. (2016) likewise measure a GitHub project's attraction of developers by the share of its contributors in a period who did not contribute before, and Nagaraj and Piezunka (2024) a volunteer community's recruitment by its first-time contributors in each quarter. New participants count these arrivals, and long-term contribution follows whether they stay.

Two questions about a new participant's held events before arriving split the new participants further. The first is whether the person was *active at the firm*, with a held event of any type on the firm's organizations at any time before the first held event on the repository. *Participants new to the firm* are the new participants who were not active at the firm. The second is whether the person was *active at another linked firm*, with a held event on another linked firm's organizations in the four quarters before arriving, and one outcome counts the participants new to the firm who were active at another linked firm. The two questions give four groups, (a) to (d) in Table 1. A person active at both the firm and another linked firm, group (b), counts among the new participants but not among those new to the firm. On the reference set, the labeled repositories with a single firm first active in 2015–2022 that the main estimates use (section 3.4), group (b) holds 251,114 of the 1,385,308 new participants' arrivals. The groups are defined by held events alone. No actor is matched to an employer, so a participant new to the firm is new to its organizations on GitHub, not known to come from outside the firm. Appendix A.5 gives the rules for bots and for actors who may be the firm's staff.

Table 1: New participants by whether they were active at the firm (at any time before) and at another linked firm (in the four quarters before)

	Active at another linked firm in the four quarters before arriving		
Active at the firm before arriving	No	Yes	All
Yes	(a) 270,345	(b) 251,114	521,459
No, the participants new to the firm	(d) 621,168	(c) 242,681	863,849
All new participants, (a) to (d)	891,513	493,795	1,385,308
Founding-quarter actors, never new			2,712,361

Notes: The numbers are person–repository pairs on the reference set, over each repository's 24-month window. A person is active at the firm with a held event of any type on the firm's GitHub organizations at any time before the first held event on the repository, and active at another linked firm with a held event on another linked firm's organizations in the four quarters before it. The four cells do not overlap and together hold every new participant. Cell (a) is active at the firm only, (b) at both, (c) at another linked firm only, and (d) at neither. The outcomes count cells (a) to (d), the participants new to the firm in (c) and (d), and those of them active at another linked firm in (c). The founding-quarter actors, active in q_0, fall in no cell. No actor is matched to an employer, so the people in (a) and (b) are not known to be the firm's staff, and those in (c) and (d) are not known to come from outside the firm. In distinct people, the new participants are 853,648, and (a) to (d) hold 155,481, 130,829, 185,433 and 566,961, which sum to more than the total because a person new on several repositories can fall in different cells on each. The founding-quarter actors are 881,757 distinct people.

Outcome variables. The outcomes are counted in the window, except long-term contribution, which follows each arrival past it. *Contributions* are the contribution-family events of new participants. These are issues, issue comments, pull requests, pull-request reviews and review comments, and commit comments, the ways in which people without write access report problems and submit code for consideration (Medappa et al., 2025). *Stars* are watch events by actors whose first held event of any type on the repository falls in the window, whom I call *new users*. A watch event is the starring of a repository (Conti et al., 2025), and developers star repositories mainly to show appreciation and to bookmark them (Borges and Valente, 2018). Stars are a common measure of a project's popularity (Zhou et al., 2020). Those studies count all of a repository's stars, while this count keeps only the actors new to the repository in the window.

Long-term contribution is the share of new participants arriving within 12 months who have a contribution- or write-family event on the repository in quarters 4 to 7 after arriving, on repositories first active in 2015–2021. Unlike the contributions count, it also counts write-family events. The name follows the term "long-term contributor" in Casalnuovo et al. (2015) and Zhang et al. (2021), and Haese and Peukert (2026) likewise follow which contributors continue and how many newcomers arrive. Zhou and Mockus (2015) define a long-term contributor as one who stays with a project for at least three years and stays productive. The window of quarters 4 to 7 after arriving is shorter than three years, and it is this paper's own. An arrival in quarter $q_0 + 4$ is followed to quarter $q_0 + 11$, so long-term contribution runs past the 24-month window, and only repositories first active through 2021 enter, so that every arrival is followed in full. Its unit is the new participant's arrival,

651,778 arrivals on the reference set, with standard errors clustered by firm.

A repository is *maintained* if it has a push or a release in its second year, quarters $q_0 + 5$ to $q_0 + 8$. Medappa et al. (2025) count a project as inactive when no code changes are made within two years of its start, and the maintained indicator is close to the opposite of that rule. Coelho et al. (2018) likewise count a project with a recent release as under maintenance. The maintained indicator is estimated, like long-term contribution, by a linear probability model, and it has its own row in Table 4.

3.4 The regression dataset

The regression dataset holds repositories, each assigned to its Compustat record firm in the year of its first held activity (section 3.1). Among repositories with a held event that is not a bot's, 111,009 repositories first active in 2015–2022 each have a firm, among 695 firms. That range leaves at least eight quarters of activity before the data end in 2024. A repository has a *single firm* when only one OSS-active firm's organizations hold it in its first-activity year (Appendix A.3). The base sample, the *reference set*, is the 102,987 labeled repositories with a single firm (Table 2). Columns 1, 2, 4 and 6 of Table 4 draw their rows from the reference set, column 3 widens the firms, and Table 4 gives each column's count. Appendix A traces every step from the event records to each estimation sample.

The firms are the OSS-active firms of section 3.1, those with a five-year window ending in any year of 2019–2024 that holds at least five patents and at least five labeled open-source projects. On dated ownership, 713 firms meet that condition, and these are the OSS-active firms. Appendix A.3 reconciles this count with the firms that the crosswalk's current links give. With the crosswalk's links extended to grants of 2011–2014, a window can end in any year of 2016–2024, and 761 firms meet that condition. The estimates here keep the windows ending in 2019–2024.

For the grants of 2011–2014, the crosswalk flags the owning entity as unknown for 19.2% of the links, against 9.4% for 2015–2024. On the repositories linked to the firms in 2011–2014, the labels cover 84%, 89%, 84% and 90% of the events in 2011, 2012, 2013 and 2014. For the events of 2012–2014 that carry no account id, the bot screen reads the account's login. Event counts can be compared across 2014 and 2015 within two limits, a step in the events per grouped record, and events with no account id on logins that the later screen does not cover.

Column 3 of Table 4 adds every firm the crosswalks link to a repository and a patent, with 1,110 firms in its sample. Inside is defined from the firm's patents in the five grant years up to the repository's start, so each repository needs such patents. On the wider set, 32,451 repositories of 1,454 firms have no such patents and enter no comparison. This group is defined repository by repository, so its firms are not only firms without patents. Some of them are granted patents in other years, and some also own repositories that enter the comparison. The main estimates keep the OSS-active firms. The linked data were built and checked on them, and with at least five patents and five labeled repositories in the window, inside is well defined for each firm.

Participation in the regression dataset is large in total (Table 2, Panel B). Over the 24-month

windows of the 111,009 repositories first active in 2015–2022, an actor's first held event on a repository occurs in 9,826,995 actor–repository pairs. Of these, 1,402,368 are new participants' arrivals, by 861,804 distinct people, and the rest are actors with no contribution-family event on the repository in the window. On the reference set, new participants arrive 1,385,308 times, by 853,648 distinct people. Of the new participants' arrivals on the reference set, 863,849, by 710,484 people, are new to the firm, and 242,681, by 185,433 people, are new to the firm and active at another linked firm. The outcomes are described repository by repository, and Table 3 summarizes them.

Table 2: The regression dataset

	Repositories	Firms
Panel A. Repositories and firms		
With a held event, owned by an OSS-active firm at first activity	141,224	
first active in 2015–2022	111,009	695
labeled, with a single firm (the reference set)	102,987	693
of which inside	82,046	
Columns 1 and 2 of Table 4	102,980	686
Column 3, every linked firm with a patent	107,018	1,110
Column 4 of Table 4, its estimate for new participants	96,854	626
Panel B. Participation		
	Events or pairs	Distinct people
On the repositories first active in 2015–2022		
Actors' first held events on a repository in its window	9,826,995	
New participants' arrivals	1,402,368	861,804
On the reference set		
Held events of every type, 2015–2024	114,936,122	5,994,624
Contribution-family events, 2015–2024	62,496,484	2,002,193
Push events, 2015–2024	18,625,845	
New participants' arrivals	1,385,308	853,648

Notes: Panel A counts repositories with a held event that is not a bot's, each assigned to the OSS-active firm that holds it in the first-activity year. Columns 1 and 2 drop firms with a single repository, and column 3 draws its repositories from every firm linked to a repository and a patent. Column 4 also needs the main group of the repository's top-ranked label and drops the cells that its effects leave with one repository, which leaves 98,795 repositories of 667 firms for the two shares of Table 4. Each Poisson estimate further drops the firms or cells whose outcome is zero throughout, and column 4's row gives the count for new participants after that drop (Appendix A). Panel B counts held events by actors who are not bots, the founding-quarter actors included, over all held quarters (section 3.3). The window is a repository's 24 months after its first-activity quarter, and an arrival is one person's first held event on one repository. The records hold pushes, not the commits inside them.

Most repositories lie inside their firm's patented main groups. Of the 102,987 labeled repositories with a single firm, 82,046 (79.7%) lie inside their firm's patented main groups, as defined just before Table 3. Section 4 therefore compares the inside repositories with the minority that lie outside, conditional on firm-by-year and main-group-by-year effects and on controls for the repository's early

activity, text and language, defined below.

Many repositories have no new participant, and these zeros stay in the comparison. Within two years of first activity, 53.5 percent of these repositories have at least one new participant, and 41.1 percent have one with no earlier held event on the firm's GitHub organizations (section 3.3). The rest have none. The estimator of section 4.1, Poisson pseudo-maximum likelihood, keeps these repositories in the comparison rather than dropping them or adding a constant before taking logarithms. It still drops the firms, or the groups of repositories that share a firm and year or a main group and year, that hold a single repository or in which no repository has a new participant (Table 2).

Controls. The regressions control for the repository's activity at its start, its text and its language. The *founding controls* are measured at or before the first-activity quarter, except the length of the text, which comes from the text the labeler read, as it stood in 2026 (Appendix A.4). They are the logarithms of one plus the events in q_0 of each family (attention, contribution, and write and administration), of the number of founding-quarter actors, and of the length of the text the labeler read. The controls also include an indicator that the repository was created in the four quarters up to q_0, indicators for repositories with no creation or text record, and language effects for the 20 most common languages, the others grouped.

Table 3 summarizes the outcomes and the founding controls over all, inside and other repositories. A repository is inside when at least half of its label weight, split equally over its labels, lies in main groups in which its firm was granted patents in the five grant years up to its first-activity year. The running example, fastMRI, is inside, since two of its three labels lie in Meta's patented main groups (section 3.2). Inside repositories have more new participants on average than the others, before any comparison within the firm, 14.36 against 9.89.

Table 3: Summary statistics

		All		Inside		Other	
	N	Mean	SD	Mean	SD	Mean	SD
Panel A. Outcomes, in the 24 months after the first-activity quarter							
All new participants	102,987	13.45	147.94	14.36	160.59	9.89	81.13
new to the firm, no earlier activity there	102,987	8.39	114.35	8.69	122.96	7.21	71.16
of those, active at another linked firm in the prior year	102,987	2.36	30.4	2.24	30.44	2.81	30.25
Contributions by new participants	102,987	110.24	1098.11	119.82	1187.03	72.75	638.81
Stars by new users	102,987	68.76	716.36	71.95	760.6	56.28	506.95
Maintained in the second year (0/1)	102,987	0.422	0.494	0.43	0.495	0.394	0.489
Long-term contribution, per arrival (0/1)	651,778	0.115	0.319	0.116	0.32	0.11	0.313
Panel B. Inside and the founding controls							
Inside the patented main groups (0/1)	102,987	0.797	0.402	—	—	—	—
Attention events in q_0	102,987	25.19	286.02	26.6	304.82	19.67	195.56
Contribution events in q_0	102,987	19.41	126.63	20.68	135.17	14.41	85.12
Write events in q_0	102,987	18.98	75.61	19.27	80.37	17.82	52.95
Founding-quarter actors	102,987	26.34	275.55	27.75	293.85	20.8	187.29
Text length (characters)	102,987	2951.38	2218.62	3029.2	2219.18	2646.49	2,190
Repositories			102,987		82,046		20,941

Notes: The rows are the reference set's repositories (Table 2), except the long-term-contribution row, which has one row per new participant who arrives within 12 months on a repository first active in 2015–2021. Section 3.3 defines each outcome, and section 4.1 defines inside, so the table shows only inside's overall mean and standard deviation. The prior year is the four quarters before arriving. The founding controls enter the regressions as the logarithm of one plus each. Panel B does not show the indicator for a repository created near its first activity, the indicators for no creation or text record, or the language effects, which also enter as controls. Appendix Table 11 gives each variable's median, upper percentiles, maximum and total.

4 Participation where code and patents overlap

The firms are the OSS-active firms of section 3.1, and 693 of them hold the 102,987 repositories of the reference set (Table 2), Meta Platforms, which holds the running example, among them. By SIC division, 318 of these firms are in services and hold 67.4% of the repositories, and 258 are in manufacturing and hold 19.4%. The five largest by repositories, Microsoft, IBM, Amazon, Alphabet and Cisco, hold 33.7% of the repositories.

The literature of section 2.2 gives reasons a firm may release open-source code. A firm may take its profit in products, services or assets complementary to the released code (Lerner and Tirole, 2005; Athey and Ellison, 2014; Henkel, 2006), or open a platform to drive its adoption and draw in outside developers (Economides and Katsamakas, 2006; Parker and Van Alstyne, 2018). Outside developers may in turn contribute where others can see their work (Lerner and Tirole, 2002). These are the literature's reasons, and the estimates below do not show why any firm releases code.

This section uses the data to ask whether repositories inside their firm's patented main groups

have more new participants over their first two years than other repositories, conditional on firm-by-year and main-group-by-year effects and the founding controls. The firm-by-year effects hold fixed what differs between firms in a year, such as size, reputation and how the firm runs its open-source program, and the main-group-by-year effects what differs between main groups in a year.

In the main estimate, a firm's repositories in the technologies it patents (inside its patented main groups) have more new participants, and more contributions from them, than other repositories, and the comparison is partly across firms. These repositories also have 29.3 percent more contributions from others, and the proportional differences for new participants and others are not separated from each other (section 4.2). These are conditional associations.

4.1 Estimation strategy

The comparison sets a firm's repositories inside its patented main groups against its other repositories, and the outcomes count, over each repository's first two years, its new participants, their contributions and the stars of new users (section 3.3). I estimate the difference between inside and other repositories by Poisson pseudo-maximum likelihood (PPML). The reasons come from the outcomes, which are counts with a long tail and many zeros, and from the users the records leave unseen.

The counts are uneven across repositories. On the reference set, the median repository has 1 new participant and the 90th percentile 18, while the mean is 13.45 and the largest repository has 23,766. Among all the repositories first active in 2015–2022, the largest one percent hold 50.7 percent of the new participants, and the largest ten percent hold 85.4 percent.

Zeros are common in these counts (section 3.4), and a model in which inside multiplies the expected count fits both the long tail and the zeros. Under an exponential mean, the expected count of inside repositories is e^{β} times that of other repositories for a coefficient β on inside, conditional on the effects and controls. This ratio is the same for small and large repositories, and the exponential keeps the expected count positive. The main estimate keeps the largest repositories, and the exponential mean gives them the weight of their counts. Without the five largest firms, its 29.5 percent for new participants and 45.5 percent for contributions are 28.1 and 40.1 percent, and without the top one percent of repositories by founding-quarter activity they are 19.7 and 40.8 percent. The five firms therefore do not carry the estimates, while the estimate for new participants falls by about a third without the repositories with the most founding activity. The intervals overlap, and no test compares the runs. A linear model of the count would give a difference in levels, and a model of the logarithm cannot take the zeros without an added constant.

PPML estimates such a model with the zeros kept, and two of its properties matter here. PPML is consistent when the conditional mean is right, whether or not the counts are Poisson, and conditioning on each firm-and-year total removes the firm-by-year effect from the likelihood, as conditioning on each firm's total removes a firm effect (Santos Silva and Tenreyro, 2006; Wooldridge, 1999). Standard errors are clustered by firm, which allows any dependence among one firm's repositories, and Appendix C.1 gives the derivation.

A repository's users are unseen (section 3.3), and they enter its expected count as a factor

of a product. The expected number of new participants over the window is the number of the repository's users during the window times the share of them who leave contribution-family work on it. In logarithms the product is a sum, so if each factor is an exponential function of inside status, the effects and the controls, the product is one too, and β is the sum of the two factors' log ratios. The firm-by-year and main-group-by-year effects absorb the part of the users common to a firm and year or to a main group and year, since conditioning on each group's total removes a multiplicative effect (Wooldridge, 1999). Variation in the users within those cells, where it moves with inside status, stays in β. The users and the share enter the expected count as a product, so I keep the exponential mean for this reason as well as for the zeros and the uneven counts.

I therefore estimate the model by PPML, with firm-by-year and main-group-by-year effects and the founding controls. For each count outcome y_r of section 3.3 and a repository r of firm p, first active in year y_0, whose top-ranked label lies in main group $g(r)$, its *primary main group*, the model of the main estimate is

$$E\left[y_r \mid \text{Inside}(r), p, g(r), y_0, X_r\right] = \exp\left(\beta\,\text{Inside}(r) + \alpha_{py_0} + \lambda_{g(r)y_0} + X_r'\delta\right). \tag{1}$$

The left-hand side is the expected count of the outcome for repository r, given whether it lies inside, its firm p, the Compustat record at the top of its ownership tree (section 3.1), its primary main group $g(r)$, its first-activity year y_0 and its founding controls X_r. The model describes the mean of the count given these terms, and Inside(r) is among them because β compares the mean of an inside repository with that of a repository outside that shares its firm-by-year level, its main-group-by-year level and its controls. The firm's patented main groups are the main groups in which the firm was granted a patent in the five grant years up to y_0, and Inside(r) indicates that at least half of the repository's label weight, with equal weights on its labels, lies in those main groups. Here α_{py_0} is a firm-by-year effect, $\lambda_{g(r)y_0}$ a main-group-by-year effect, the year being that of first activity, and X_r the founding controls of section 3.4 with the language effects. PPML estimates β on the counts as they are, and only the estimate is transformed afterwards, to $100(e^{\beta}-1)$, the percent difference in the expected count between inside and other repositories, with its standard error on the same scale. The two shares, maintenance and long-term contribution, come from linear probability models with the same effects as the counts, and the differences they give are in percentage points.

4.2 Baseline results

If the appropriability perspective of section 2.2 holds, as Appendix Table 22 maps it to repositories, inside repositories are expected to have more new participants than the firm's other repositories. If the line of section 2.1 holds, so that code tied to patents draws less participation, inside repositories are expected to have fewer. The other perspectives of section 2.2 predict how the inside difference varies across repositories and participants, which section 4.3 reads. An estimate that matches a prediction is consistent with its perspective and does not establish it.

In the main estimate, column 4 of Table 4, inside repositories have 29.5 percent more new

participants than other repositories, conditional on firm-by-year and main-group-by-year effects and the founding controls (95 percent interval 14.6 to 46.4) and 45.5 percent more contributions from them. Averaged over inside repositories, the difference from the specified comparison is 3.34 new participants in the 24 months (95 percent interval 1.86 to 4.64). This average comes from a skewed count, since 43.7% of inside repositories have no new participant, and the difference is 1.68 without the top one percent of repositories by founding-quarter activity. For participants new to the firm the difference is 27.6 percent, and it is 21.9 percent for participants new to the firm active at another linked firm in the four quarters before arriving.

The six columns of Table 4 differ in what they compare, and I call a column's effects and controls its *form*. Columns 1 to 3 compare a firm's inside and other repositories with firm and year effects, column 2 with the founding controls and column 3 on the wider set of firms. Column 4 has firm-by-year, main-group-by-year and programming-language effects with the founding controls, so the repositories compared need not share both the firm and the main group, and its comparison is partly across firms (Appendix C.2). In its estimation sample, 930 of the 1,181 main-group-by-year cells hold both inside and other repositories, with a median of 9 firms a cell, and 921 of them have two or more firms. Column 5 keeps column 4's form and draws inside at the CPC subgroup, and column 6 drops the main-group-by-year effects, so it compares repositories of one firm first active in one year. I call an estimate *significant* when its p-value, adjusted by Holm's method for the outcomes tested together (Holm, 1979) (Table 4's notes), is below five percent, and two or three stars mark such an estimate.

Table 4: Participation in a firm's repositories inside and outside its patented main groups

	(1) Firm and year effects	(2) With founding controls	(3) Every linked firm	**(4) Main-group-by-year effects**	(5) Inside at the subgroup	(6) No main-group effects
				Main estimate		
Panel A. New participants within 24 months, inside against other repositories						
All new participants (%)	77.3***	34.3***	79.9***	**29.5***	27.8***	36.8***
	(25.7)	(11.1)	(22.4)	(8.1)	(7.0)	(9.7)
new to the firm, no earlier activity there (%)	76.6***	30.5**	79.8***	27.6***	35.8***	33.3***
	(27.9)	(12.4)	(26.2)	(9.9)	(9.0)	(11.7)
of those, active at another linked firm in the prior year (%)	69.4***	26.2**	73.0***	21.9***	30.8***	25.5**
	(28.0)	(12.7)	(27.0)	(8.0)	(9.0)	(10.9)
Panel B. Contributions within 24 months, inside against other repositories						
Contributions by new participants (%)	84.5***	28.1**	108.3***	**45.5***	21.6*	41.4***
	(27.6)	(12.8)	(26.6)	(12.9)	(11.0)	(13.8)
All contributions (%)	81.2***	19.5**	99.9***	**37.6***	15.4	34.7***
	(22.9)	(8.7)	(24.0)	(10.2)	(8.4)	(9.3)
Contributions by others (%)	77.5***	11.4	91.6***	29.3***	9.2	27.7***
	(21.9)	(8.9)	(24.6)	(11.5)	(8.0)	(9.3)
Panel C. Stars, maintenance and long-term contribution						
Stars by new users (%)	44.0*	19.5*	42.6*	7.1	11.4*	16.0
	(28.8)	(12.3)	(29.5)	(11.3)	(7.3)	(11.6)
Maintained in the second year (points)	6.4***	3.8***	5.6***	2.4***	—	3.8***
	(1.2)	(1.1)	(1.0)	(0.7)		(0.8)
Long-term contribution (points)	0.5	0.1	0.8	0.2	—	0.1
	(0.9)	(0.7)	(0.7)	(0.4)		(0.6)
Firm and year effects	Yes	Yes	Yes			
Firm-by-year effects				Yes	Yes	Yes
Main-group-by-year effects				Yes	Yes	
Founding controls		Yes		Yes	Yes	Yes
Language effects		Yes		Yes	Yes	Yes
CPC level of inside	main group	main group	main group	main group	subgroup	main group
Repositories	102,980	102,980	107,018	96,854	96,854	97,951
Firms	686	686	1,110	626	626	630

Notes: Column 4 is the main estimate, and its key entries are in bold. Each entry in Panels A and B and the first row of Panel C is $100(e^{\beta}-1)$, the percent difference in the expected count between inside and other repositories, where β is the Poisson pseudo-maximum-likelihood coefficient on the inside indicator in equation (1). The maintained and long-term-contribution rows are differences in percentage points from linear probability models with the same effects. Standard errors clustered by firm, on the scale of each entry, are in parentheses. The founding controls are those of section 3.4, and the prior year is the four quarters before arriving. All contributions are every actor's contribution-family events in the window, bots aside, and contributions by others are those less the new participants'. Repository and firm counts are those of the estimate for new participants, after Poisson drops the firms or cells whose outcome is zero on every repository in columns 4 to 6, and before that drop in columns 1 to 3. Column 4's two shares use 98,795 repositories of 667 firms, and column 6's maintained share uses 99,455 repositories of 671 firms. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method (Holm, 1979) within its column. The three families are the counts of Panel A with contributions by new participants and stars, all contributions and contributions by others with all events (not shown), and the two shares of Panel C with the shares of repositories that have a new participant, one new to the firm and one of those active at another linked firm (not shown).

The point estimate for new participants is larger with coarser technology effects, and it is positive in all three forms, though no test compares them. It is 36.8 percent with no technology effects (column 6), 34.8 percent with effects at the subclass, one level above the main group, by cohort, the year of first activity (Appendix Table 17), and 29.5 percent with main-group-by-year effects (column 4), and each passes Holm's adjustment. For contributions by new participants the same

three are 41.4, 49.6 and 45.5 percent, which are not ordered in the same way.

With firm and year effects alone the differences are larger, and the founding controls attenuate them. In column 1, with a firm effect and a year effect, inside repositories have 77.3 percent more new participants than the firm's other repositories (95 percent interval 33.5 to 135.6). The founding controls bring this to 34.3 percent, and on the scale of β the coefficient falls by 49 percent. On the wider set of firms, the difference is 79.9 percent. The founding controls can absorb part of the inside difference itself (Appendix C.2).

Inside repositories have more contributions overall, not only more from new participants. In column 4, all contributions are 37.6 percent higher, and contributions by others, everyone other than the new participants, are 29.3 percent higher. In one stacked model, built as in section 4.4, the inside coefficient for others less that for new participants is -0.1179 (standard error 0.0959, p-value 0.2192), and in no column are the two groups separated from each other.

New participants make 49.6 percent of all contributions on inside repositories and 49.7 percent on the others, shares of the totals, in which large repositories count for more. Repository by repository, among those with a contribution, the new participants' share of a repository's contributions is 1.02 points higher inside in column 4's form (95 percent interval -0.52 to 2.55, adjusted p-value 0.2496) and 1.66 points higher in column 6's (interval 0.13 to 3.19, adjusted p-value 0.1308), so neither is separated from zero after Holm's adjustment, though column 6's interval excludes zero before it. That share exists only for repositories with a contribution, and inside repositories are 2.53 points more likely to have one in column 4's form, so the two sides differ slightly in which repositories are compared, and I set no bound in advance for how close the two shares must be. Inside repositories have more contributions from both groups, and the estimates do not establish that new participants account for a larger share.

Three qualifications go with column 4. First, a linear regression with column 4's effects can put negative weights on some repositories, and in that regression for new participants they sum to -0.206, against -0.025 with column 6's, a diagnostic of the linear regression and not of the Poisson coefficient (Appendix C.2). Second, if a repository's new participants are the users who leave contribution-family work on it, inside repositories may have more users or a larger share of users who leave such work, and the data cannot separate the two (Appendix C.2). Third, a grant can follow a repository's first activity, and the labeler read the text as it stood in 2026 (Appendices A.7 and B.7).

In column 4, inside repositories are 2.4 points more likely to be maintained in their second year. Stars by new users differ by 7.1 percent and long-term contribution by 0.2 points, and neither is separated from zero after Holm's adjustment. Appendix C.5 discusses these outcomes and the test of the perspective of visible contributions. The positive inside difference itself runs against the mapped prediction of less participation in code tied to patents (section 2.1).

4.3 Contrasts by perspective

The line of work of section 2.1 predicts the sign of the inside difference, and the three perspectives of section 2.2, with a fourth, predict where it is larger. Each reaches the data through a mapping that I state, and Appendix Table 22 sets the mappings out.

The fourth perspective, of related experience, predicts where the difference is larger but not its sign. In Athey and Ellison (2014), programmers who have used open-source code publish their improvements, and the mapped prediction concerns new participants whose earlier activity lies in the same or a related main group. In their conclusion, Athey and Ellison (2014) remark that raising a project's quality can have perverse effects, so this perspective fixes no sign for the inside difference itself.

For less participation in code tied to patents (Lin and Rai, 2024; Wen et al., 2013), β, the coefficient on inside, is predicted negative. For appropriability (Teece, 1986), β and the interaction of inside with the firm's patent weight in the repository's primary main group are both predicted positive. For code tied to products (Gambardella and Hall, 2006; Athey and Ellison, 2014), the interaction of inside with membership in the two classes of a classifier of repositories that hold such code, described below, is predicted positive. For visible contributions (Lerner and Tirole, 2002), the inside difference on issues and on pull requests minus that on stars is predicted positive. For related experience (Athey and Ellison, 2014), the inside difference for new participants with related experience minus that for the others is predicted positive.

Each prediction of where the difference is larger is read from one contrast in column 4's form, and Appendix C.5 sets out the visible-contribution contrasts.

The estimates compare proportional differences, and a larger gain in participation can come with a smaller proportional difference (Wooldridge, 2023; Chen and Roth, 2024), so a prediction of where the difference is larger needs a condition on the proportional difference that none of the perspectives states. This qualification was written after the results were seen.

None of the five contrasts for the last four perspectives above passes its declared test, Holm's adjustment at the five-percent level, so these tests give no statistically clear support for their mapped predictions (Appendix Table 23). Clear support here means that a contrast passes Holm's adjustment at the five-percent level, the level at which I call an estimate significant (section 4.2). As in Galasso and Schankerman (2015), each prediction is read from one contrast, estimated separately in column 4's form.

I report every contrast with its interval for two reasons. First, the mappings, the contrasts and the outcomes adjusted together by Holm's method were specified before the results, and reporting all of them keeps the reading from resting on the subset of contrasts that happen to pass (Anderson, 2008). Second, each contrast tests one perspective's mapped prediction, and its interval shows which sizes of the predicted contrast these data leave open, so a contrast that misses the test still informs its perspective.

The repository-level mapping of the appropriability perspective (Teece, 1986) predicts a larger inside difference where the firm holds more patent weight in the repository's primary main group.

The interaction of inside with the weight is 6.5 percent for new participants (95 percent interval −7.3 to 22.4), and no outcome's interaction is significant. The interaction compares how the count changes with the weight among inside repositories with how it changes among outside ones. The weight is zero for 19,982 of the 20,250 outside repositories with a primary main group, so the outside side of the comparison rests on the 268 with positive weight, and the interval reaches 22.4 percent.

The mapping of the perspective of code tied to products (Athey and Ellison, 2014; Gambardella and Hall, 2006) predicts a larger inside difference in code tied to the firm's products. A rule-based classifier sorts each repository by its own text into six classes, and code tied to products joins two of them, (i) software development kits, interfaces and libraries, and (ii) drivers, integrations and plugins. The interaction of inside with membership in those two classes is −10.1 percent (interval −28.1 to 12.4), not significant, with a point estimate opposite the mapped prediction and an upper limit of 12.4 percent.

The mapping of the perspective of visible contributions (Lerner and Tirole, 2002) predicts a larger inside difference on issues and pull requests than on stars, which gives two contrasts. With inside as in Table 4, the inside difference on the new participants' issues exceeds that on stars by 31.6 percent (interval 1.5 to 70.7), and on their pull requests by 36.3 percent (interval −1.3 to 88.2). The pointwise interval for issues excludes zero and the one for pull requests includes it, and each contrast misses the declared test (Holm's adjustment over issues, pull requests and commit comments), with adjusted p-values of 0.1152 and 0.1193.

The mapping of the perspective of related experience (Athey and Ellison, 2014) predicts a larger inside difference for new participants whose earlier held events include a repository in the same or a related main group. The contrast between their inside difference and that of the other new participants is −7.5 percent (interval −17.0 to 3.1), not significant, again with a point estimate opposite the mapped prediction and an upper limit of 3.1 percent.

Of the five contrasts, only the issue contrast has a pointwise interval that excludes zero. Here the mapped predictions describe patterns in the data and say nothing of what a firm intends.

4.4 Heterogeneity across new participants

The splits by participants' histories and by what they do next divide the inside difference among groups of arrivals, so, taken together, they describe who arrives and what the arrivals then do, not what any group prefers.

By their whole held history before arriving, new participants fall into three groups. People with a history at the firm have a held event of any type on the firm's organizations before the quarter of their first held event on the repository. People with a history only at another linked firm have no such event and at least one on another linked firm's organizations. People with a history at no linked firm have no held event on any linked firm's organizations, the firm's included, though they may still have been active on personal accounts or on organizations outside the linked firms. The history runs from 2015 to the quarter of arrival. Table 1 uses the same history for the firm but only the four quarters before arriving for another linked firm, so its cells differ from these groups. For

each group, the outcome is the repository's count of new participants in that group, estimated in column 4's form. Chen et al. (2022) likewise estimate contributions to a firm's open-source projects separately for its own developers and for outsiders, a split approximated here by held history, with no actor matched to an employer, and Haese and Peukert (2026) set contributors with an earlier history on one project against the others.

Over the full history from 2015, the inside difference is 31.7 percent for people with a history at no linked firm, which is significant, 32.8 percent for people with a history at the firm, and 23.7 percent for people with a history only at another linked firm, both also significant. The history is shorter for repositories first active early, so a check looks back the same 16 quarters before every arrival, on the repositories first active in 2019–2022, as studies measure a history over a window of the same length for everyone (Deryugina and Molitor, 2020) or hold the first years of the data back as history (Cunningham and Kapacinskaite, 2026). There, the inside difference for people with a history at no linked firm is 5.2 percent, and not significant. The inside difference for this group, narrower than the participants new to the firm, is therefore fragile, significant on the whole history and not on the 16 quarters. The two estimates differ both in how far the history looks back and in the repositories they cover, so they do not show which of the two changes accounts for the difference between them. No group's inside difference is distinguishable from another's. The differences between groups come from one model in which the groups' counts are stacked, every effect and control is interacted with the group, and standard errors are clustered by firm, as in Albert and Caggese (2021) and Batistich and Bond (2023). Its group coefficients equal those of the separate models. A history counts held events on the linked firms' organizations only, not all of GitHub.

New participants also differ in what they do in the four quarters after arriving. This split covers the new participants who arrive in the repository's first year, quarters $q_0 + 1$ to $q_0 + 4$, so that every person is followed for four full quarters within the window. The arrival quarter itself is left out, since the order of events within a quarter is not held. Two questions, both about contribution-family events in those four quarters, place each person. The first is whether the person contributed to another repository of the firm's organizations, and the second whether the person contributed on another linked firm's organizations. People thus go on to neither, to another linked firm only, to the firm's other repositories only, or to both, and as with the history, each group's count is estimated in column 4's form.

The inside difference appears in every group. Among people who go on to neither, it is 23.4 percent, which is significant. For people who go on to the firm's other repositories only, it is 30.8 percent, also significant. In column 4, no contrast between two groups is significant, and the wide intervals leave open whether the groups differ. The groups are defined by observed events, and no actor is matched to an employer, so they describe who arrives. Studies of open-source projects likewise class newcomers by what they do later (Zhou and Mockus, 2015) and divide contributions by where they go (Chen et al., 2022).

4.5 Robustness

Inside rests on a share of label weight, a level of the classification and the firm's own patents, each of which could be drawn another way. A repository's label weight is split equally over the main groups the labeler returns for it (section 3.2), and the main estimate counts the repository as inside when at least half of that weight lies in the firm's patented main groups.

Table 5 repeats column 4 of Table 4 with inside drawn in other ways. Panel A moves the cut on the share of label weight, as Kerr (2010) moves the cut that defines a breakthrough patent and Bandiera et al. (2021) the cut on a share that defines their indicator. The inside difference in new participants and in contributions is positive under every definition of inside, and Appendix C.2 reports each in the forms of Table 4's columns 1 and 2. When every label must fall in the firm's patented main groups, the inside difference in new participants is 16.4 percent. This is about half the estimate of Table 4. It passes Holm's adjustment here, narrowly, with an adjusted p-value of 0.0413. In a check that drops GitHub's create and member events from the records, it does not pass, at 15.6 percent on new participants and 18.0 percent on contributions, with adjusted p-values of 0.0663 and 0.0763. In the running example, fastMRI has two of its three labels in Meta's patented main groups, so it falls outside when all of the weight must lie there (Appendix A.9).

When the share of the repository's label weight in those main groups replaces the inside indicator, the inside difference is 43.5 percent for a move of the share from zero to one (95 percent interval 21.0 to 70.1), which is significant. Bandiera et al. (2021) likewise report the estimate on a continuous share beside the indicator built by cutting it, and here the share's estimate is read for its sign.

The cuts of Panel A are not doses, because the share divides by the number of a repository's labels. Two labels in the firm's patented main groups give a share of one on a repository with two labels and of two thirds on a repository with three. Each cut also changes both sides of the comparison, since raising it moves repositories from inside to the other repositories. Raising the cut from two thirds to all of the weight moves 1,552 reference repositories from inside to the other repositories, 1,397 of them in column 4's sample. Each of them has 3 labels, of which all but one lie in the firm's patented main groups. Their raw mean of new participants is 19.67, against 14.94 for the repositories with all of their label weight in those main groups. No other range of the share has a higher raw mean. The fall in Panel A from 30.9 percent at two thirds to 16.4 percent at all of the weight comes from how the cut sorts repositories between the two sides, since nothing else changes between the two estimates.

In a direct test of the step from a share of two thirds to a share of one, no fall is shown, which is not the same as showing none. In column 4's form, repositories with a share of one have 32.5 percent more new participants than those with two thirds within three-label repositories (95 percent interval −15.1 to 106.8), and −1.2 percent across all repositories with label-count effects (interval −26.4 to 32.6). For contributions by new participants the two are 11.6 percent (interval −36.4 to 95.7) and −11.1 percent (interval −40.4 to 32.7). Holm's adjustment across these four estimates, a family of their own, leaves none separated from zero, and the smallest adjusted p-value is 0.8613. The intervals are wide, so they leave open a fall as well as a large rise. Panel A's fall is a different

comparison, since each cut moves repositories between the two sides.

Measured by the number of labels in the firm's patented main groups, no fall is shown, and no steady rise is shown either. In one model in column 4's form with label-count effects, one, two and three such labels go with 29.2, 34.3 and 43.2 percent more new participants than none, and 54.8, 88.8 and 74.6 percent more contributions. Of the four steps from one count to the next, two for each outcome, only the step from one label to two on contributions is separated from zero after Holm's adjustment (adjusted p-value 0.0336), and the point estimate for contributions is lower at three labels than at two.

The inside difference stays positive when inside is drawn without the share. With inside read from the first label alone, or from any label with one row per label, and with main-group-by-year and label-count effects, inside repositories have 22.0 and 26.6 percent more new participants and 49.1 and 45.2 percent more contributions, each with an adjusted p-value below one percent (Appendix C.2 and Table 16). With a control for repository size, measured after the outcome window, those two estimates for new participants are no longer separated from zero after Holm's adjustment, and the two for contributions still are (Appendix Table 16).

Panel B redraws inside one level coarser and one level finer in the classification, as Thompson and Fox-Kean (2005) redraw control patents at a finer class and Bloom et al. (2013) rebuild their measure with finer classes, which they caution are not necessarily better. At the CPC subclass, 92,365 of the 102,987 reference repositories are inside, so the comparison rests on the few left outside. There the inside difference misses Holm's adjustment in the forms of Table 4's columns 1, 2 and 4 (Appendix C.2). The subclass estimate for new participants is not smaller, 43.3 percent, but its standard error is 25.0, against 8.1 at the main group, where the estimate is 29.5 percent, and 7.0 at the subgroup, where it is 27.8 percent. Of the three levels, the standard error is small only at the main group and the subgroup, and drawing inside at those levels needs labels that name main groups (section 3.2).

GitHub's archived flag, which marks a repository its owner has made read-only, is read from 2026 records. It marks 34.9 percent of column 1's inside repositories and 22.6 percent of the others (Appendix A.4). The flag describes 2026 rather than the outcome window, and no estimate drops archived repositories.

By arrival quarter, the column-4 inside difference in new participants is positive in every quarter, with larger point estimates in later quarters (Appendix Table 19). It is 12.6 percent in the first quarter, not significant, and 41.1 percent in the eighth, which is significant. In column 1, the first quarter is significant, at 44.1 percent. The format follows Abebe et al. (2021), who estimate an effect for each period since a start date. No test across quarters was declared, and pointwise intervals support only pointwise readings (Freyaldenhoven et al., 2019), so the larger later estimates are a description. The clock starts at the first observed activity rather than a date on which outsiders could first see the repository, so the profile describes participation by the repository's age.

Forks, repositories copied from another repository, are 5.4 percent of column 1's inside repositories and 10.4 percent of the others, and dropping them leaves every significant estimate significant. Since

a fork can operate apart from its original and is not the same unit (Kalliamvakou et al., 2016), studies of GitHub drop forks from their samples (Abou El-Komboz and Goldbeck, 2025; Medappa et al., 2025).

On the repositories that the second labeler also labels, the column-1 estimate is close under both sets of labels, and the column-4 estimate is imprecise under both (Appendix C.3). The comparison of the two sets of labels is read as agreement between labelers that shows how the estimate moves with the labeler, not as accuracy.

Table 5: The inside difference in column 4's form under other definitions of inside

	Repositories	Firms	Inside repositories	New participants	Contributions
Panel A. The share of label weight in the firm's patented main groups					
Any of the label weight (> 0)	96,854	626	83,001	32.0^{***}	67.2^{***}
				(10.2)	(15.0)
At least half ($\geq 1/2$)	96,854	626	82,046	29.5^{***}	45.5^{***}
				(8.1)	(12.9)
At least two thirds ($\geq 2/3$)	96,854	626	72,041	30.9^{***}	55.5^{***}
				(6.8)	(15.7)
All of the label weight ($= 1$)	96,854	626	70,489	16.4^{**}	19.9^{**}
				(6.7)	(8.4)
Panel B. The level of the CPC, at half the label weight					
Subclass	96,854	626	92,365	43.3	53.9
				(25.0)	(29.5)
Main group	96,854	626	82,046	29.5^{***}	45.5^{***}
				(8.1)	(12.9)
Subgroup	96,854	626	73,184	27.8^{***}	21.6^{*}
				(7.0)	(11.0)

Notes: Each row repeats column 4 of Table 4, with firm-by-year effects, main-group-by-year effects, language effects and the founding controls, and changes only how inside is drawn. Each estimate is $100(e^{\beta} - 1)$, with its standard error clustered by firm in parentheses. The repository and firm counts, the same in every row, are those of each row's estimate for new participants. The row at half the label weight in Panel A and the main-group row of Panel B are column 4 of Table 4, and the subgroup row is its column 5. The inside repositories are counted among the 102,987 reference repositories. Panel B keeps half the label weight and compares labels and patents at another level of the CPC. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method for the five count outcomes of the row's estimate.

5 Conclusion

The paper makes three contributions, a method, data and one result. The method places open-source code in the patent classification, repository by repository, with an LLM's labels in CPC main groups (section 3.2). The labels were checked against two references built independently of them. They share at least one main group with those of the European Patent Office's text categorizer for 76.4%

of the repositories both classify, against 38.5% when the label sets are dealt out at random among repositories of the same CPC section. Run on patents with another LLM, the same method gives labels that share a main group with the examiners' classification for 89.1% of them, against 20.8% when the model's label sets are dealt out at random among patents of the same CPC section. The method allows a firm's code and its patents to be set side by side in one classification.

The data link Compustat firms to their patents and to their GitHub accounts, each link with its evidence and the year in which the firm owns the account (section 3.1). The Global Corporate Patent Dataset, which never entered the crosswalk, names the same firm for 92.8% of the 2015–2022 patents both link to a Compustat firm, and the commit-email data of Spinellis et al. (2020) name the same firm for 93.9% of the accounts both hold. The data also show that US patent documents cite the OSS-active firms' projects more often than comparable projects of other listed firms, and that citations are rare in both groups (Appendix D.1).

Conditional on firm-by-year and main-group-by-year effects and the founding controls, inside repositories have 29.5 percent more new participants than other repositories (95 percent interval 14.6 to 46.4) and 45.5 percent more contributions from new participants (95 percent interval 22.3 to 73.0), alongside 37.6 percent more contributions from everyone. For new participants with no earlier activity at the firm on GitHub, the inside difference is 27.6 percent. Without the five largest firms, the estimates for new participants and contributions are 28.1 and 40.1 percent. With inside read from the first label alone, or from any label with one row per label, and with main-group-by-year and label-count effects, they are 22.0 and 26.6 percent for new participants and 49.1 and 45.2 percent for contributions. Each of these differences has an adjusted p-value below one percent. The association is partly across firms (section 4.2). A repository's overlap with its firm's own patents differs from the enforcement and the change in patent scope that Lin and Rai (2024) discuss, so this result and theirs stand side by side.

To my knowledge, this paper is the first to compare participation across a firm's open-source projects by whether each lies in the firm's own patented technologies, and much remains to be done with that comparison. Why firms publish code where they patent, and what lies behind the difference between inside and other repositories, are questions for further work. A match of actors to employers would show who employs the people who come, and repository text dated to each repository's first activity would allow labels from that date. The crosswalk now extends to grants of 2011–2014 (section 3.4), so the comparison can reach earlier years. The same classification can take in the firms' products, and a model of the firm's and the contributors' choices can build on these facts. Because the labels place a firm's open-source code and its patents in one classification, the data also allow a comparison of the two portfolios within a firm. A distance between them can be read against a reference built from the firm's own projects and patents, and that comparison is the subject of separate work.

Use of AI tools

I used LLMs for three data-construction tasks and report the checks or bounds for each. To find firms' GitHub accounts, I used LLMs from several providers, run through command-line agents that search the web. I accepted every account the search reported as a verified official page. A recall check bounds the share of firms whose accounts the search missed (Appendix A.4). In the patent crosswalk, LLMs from several providers, run the same way, assisted the identification of the firm for 6.91% of all firm–patent links (Gong, 2026). I required evidence for the exact identifiers in each link (Appendix A.4). I used GLM-5.3-Flash to label the open-source projects. In Appendix B.1, I check the labels against the EPO's text categorizer and, by running DeepSeek-V4-Flash on patents, against the examiners' labels. The AI assistants Claude Code and Codex also helped with coding, editing and drafting. I reviewed all text and results and take responsibility for them.

References

Abebe, G., Caria, A. S., Fafchamps, M., Falco, P., Franklin, S., and Quinn, S. (2021). Anonymity or distance? Job search and labour market exclusion in a growing African city. *Review of Economic Studies*, 88(3):1279–1310.

Abou El-Komboz, L. and Goldbeck, M. (2025). Career concerns as a public good: The role of signaling for open source software development. *Labour Economics*, 97:102800.

Abramitzky, R., Boustan, L., Eriksson, K., Feigenbaum, J., and Pérez, S. (2021). Automated linking of historical data. *Journal of Economic Literature*, 59(3):865–918.

Albert, C. and Caggese, A. (2021). Cyclical fluctuations, financial shocks, and the entry of fast-growing entrepreneurial startups. *Review of Financial Studies*, 34(5):2508–2548.

Alexy, O., George, G., and Salter, A. J. (2013). Cui bono? The selective revealing of knowledge and its implications for innovative activity. *Academy of Management Review*, 38(2):270–291.

Allison, P. D. and Waterman, R. P. (2002). Fixed-effects negative binomial regression models. *Sociological Methodology*, 32(1):247–265.

An, M., Kim, S., Sung, S., and Yoo, D. (2026). A context-aware framework for strategic patent intelligence: Keyword expansion and non-patent literature classification. *Online Information Review*, 50(5):962–980.

Anderson, M. L. (2008). Multiple inference and gender differences in the effects of early intervention: A reevaluation of the Abecedarian, Perry Preschool, and Early Training Projects. *Journal of the American Statistical Association*, 103(484):1481–1495.

Angelopoulos, A. N., Bates, S., Fannjiang, C., Jordan, M. I., and Zrnic, T. (2023). Prediction-powered inference. *Science*, 382(6671):669–674.

Argente, D., Baslandze, S., Hanley, D., and Moreira, S. (2025). Patents to products: Product innovation and firm dynamics. Working Paper 34592, National Bureau of Economic Research.

Arora, A., Belenzon, S., and Sheer, L. (2021). Matching patents to Compustat firms, 1980–2015: Dynamic reassignment, name changes, and ownership structures. *Research Policy*, 50(5):104217.

Arts, S., Cassiman, B., and Gomez, J. C. (2018). Text matching to measure patent similarity. *Strategic Management Journal*, 39(1):62–84.

Athey, S. and Ellison, G. (2014). Dynamics of open source movements. *Journal of Economics & Management Strategy*, 23(2):294–316.

Autor, D., Chin, C., Salomons, A., and Seegmiller, B. (2024). New frontiers: The origins and content of new work, 1940–2018. *Quarterly Journal of Economics*, 139(3):1399–1465.

Bailey, M. J., Cole, C., Henderson, M., and Massey, C. (2020). How well do automated linking methods perform? Lessons from US historical data. *Journal of Economic Literature*, 58(4):997–1044.

Balla, S., Zacchiroli, S., Degueule, T., Falleri, J.-R., and Robbes, R. (2026). DRAGON: Robust classification for very large collections of software repositories. arXiv:2602.09071, version 1.

Bandiera, O., Best, M. C., Khan, A. Q., and Prat, A. (2021). The allocation of authority in organizations: A field experiment with bureaucrats. *Quarterly Journal of Economics*, 136(4):2195–2242.

Bar-Gill, O. and Parchomovsky, G. (2003). The value of giving away secrets. *Virginia Law Review*, 89(8):1857–1895.

Barkemeyer, D., Bena, J., and Li, X. (2026). Technology life cycles: Investment, growth, and firm value. Working paper 6350538, SSRN.

Baron, J. and Pohlmann, T. (2018). Mapping standards to patents using declarations of standard-essential patents. *Journal of Economics & Management Strategy*, 27(3):504–534.

Batistich, M. K. and Bond, T. N. (2023). Stalled racial progress and Japanese trade in the 1970s and 1980s. *Review of Economic Studies*, 90(6):2792–2821.

Battaglia, L., Christensen, T., Hansen, S., and Sacher, S. (2025). Inference for regression with variables generated by AI or machine learning. Technical report, arXiv:2402.15585.

Belenzon, S. and Schankerman, M. (2015). Motivation and sorting of human capital in open innovation. *Strategic Management Journal*, 36(6):795–820.

Bena, J., Ferreira, M. A., Matos, P., and Pires, P. (2017). Are foreign investors locusts? The long-term effects of foreign institutional ownership. *Journal of Financial Economics*, 126(1):122–146.

Benatia, D., Bellégo, C., and Pape, L.-D. (2025). Dealing with logs and zeros in regression models. arXiv:2203.11820v3, September 2025.

Bloom, N., Schankerman, M., and Van Reenen, J. (2013). Identifying technology spillovers and product market rivalry. *Econometrica*, 81(4):1347–1393.

Borges, H. and Valente, M. T. (2018). What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146:112–129.

Brea, E. (2024). The yin yang of AI: Exploring how commercial and non-commercial orientations shape machine learning innovation. *Research Policy*, 53(6):105008.

Cao, Y., Mkrtchyan, A., Tucker, J. W., and Wan, C. (2026). Corporate disclosure of innovation on GitHub. SSRN Working Paper 5932514, Donald G. Costello College of Business at George Mason University. First draft June 2025; this draft March 2026.

Carlson, N. A. and Burbano, V. (2026). The use of LLMs to annotate data in management research: Foundational guidelines and warnings. *Strategic Management Journal*, 47(3):699–725.

Casalnuovo, C., Vasilescu, B., Devanbu, P., and Filkov, V. (2015). Developer onboarding in GitHub: the role of prior social links and language experience. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2015)*, pages 817–828. ACM.

Chen, J. and Roth, J. (2024). Logs with zeros? some problems and solutions. *Quarterly Journal of Economics*, 139(2):891–936.

Chen, W., Jin, F., and Xue, L. (2022). Flourish or perish? The impact of technological acquisitions on contributions to open-source software. *Information Systems Research*, 33(3):867–886.

Coelho, J., Valente, M. T., Silva, L. L., and Shihab, E. (2018). Identifying unmaintained projects in GitHub. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–10. ACM.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

Cohen, W. M., Nelson, R. R., and Walsh, J. P. (2000). Protecting their intellectual assets: Appropriability conditions and why U.S. manufacturing firms patent (or not). Working Paper 7552, National Bureau of Economic Research.

Cohn, J. B., Liu, Z., and Wardlaw, M. I. (2022). Count (and count-like) data in finance. *Journal of Financial Economics*, 146(2):529–551.

Conti, A., Gupta, V., Guzman, J., and Roche, M. P. (2023). Incentivizing innovation in open source: Evidence from the GitHub sponsors program. NBER Working Paper 31668, National Bureau of Economic Research.

Conti, A., Peukert, C., and Roche, M. P. (2025). Beefing IT up for your investor? Engagement with open source communities, innovation, and startup funding: Evidence from GitHub. *Organization Science*, 36(4):1551–1573.

Correia, S., Guimarães, P., and Zylkin, T. (2020). Fast Poisson estimation with high-dimensional fixed effects. *Stata Journal*, 20(1):95–115.

Crowston, K. and Howison, J. (2005). The social structure of free and open source software development. *First Monday*, 10(2).

Cunningham, C. and Kapacinskaite, A. (2026). Keeping invention confidential. *Management Science*, 72(2):1658–1679.

Dabbish, L., Stuart, C., Tsay, J., and Herbsleb, J. (2012). Social coding in GitHub: Transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work (CSCW)*, pages 1277–1286.

de Chaisemartin, C. and D'Haultfœuille, X. (2020). Two-way fixed effects estimators with heterogeneous treatment effects. *American Economic Review*, 110(9):2964–2996.

Dell, M. (2025). Deep learning for economists. *Journal of Economic Literature*, 63(1):5–58.

Dell, M. and Rambachan, A. (2026). The measurement revolution? credible measurement and inference in the age of AI. arXiv:2608.23524; NBER Working Paper 35744.

Deryugina, T. and Molitor, D. (2020). Does when you die depend on where you live? Evidence from Hurricane Katrina. *American Economic Review*, 110(11):3602–3633.

Duan, J. and Pelger, M. (2026). Inference with AI-generated covariates. Working Paper 35481, National Bureau of Economic Research.

Economides, N. and Katsamakas, E. (2006). Two-sided competition of proprietary vs. open source technology platforms and the implications for the software industry. *Management Science*, 52(7):1057–1071.

Egami, N., Hinck, M., Stewart, B., and Wei, H. (2023). Using imperfect surrogates for downstream inference: Design-based supervised learning for social science applications of large language models. In *Advances in Neural Information Processing Systems 36*, pages 68589–68601.

Emer, L., Lippi, M., Mina, A., and Vandin, A. (2026). Large language models, encoder architectures and hybrid approaches for patent classification. arXiv:2601.23200, version 2.

Emery, L. P., Lim, C., and Ye, S. (2025). The private value of open-source innovation. SSRN Working Paper 5054880, revised November 2025.

Fosfuri, A., Giarratana, M. S., and Luzzi, A. (2008). The penguin has entered the building: The commercialization of open source software products. *Organization Science*, 19(2):292–305.

Freyaldenhoven, S., Hansen, C., and Shapiro, J. M. (2019). Pre-event trends in the panel event-study design. *American Economic Review*, 109(9):3307–3338.

Galasso, A. and Schankerman, M. (2015). Patents and cumulative innovation: Causal evidence from the courts. *Quarterly Journal of Economics*, 130(1):317–369.

Gambardella, A. and Hall, B. H. (2006). Proprietary versus public domain licensing of software and research products. *Research Policy*, 35(6):875–892.

Ganglmair, B. and Kann, A. (2025a). Looking for innovation beyond the patent system: Evidence from research disclosures. Slides presented at the DRUID 2025 Conference, June 27, 2025.

Ganglmair, B. and Kann, A. (2025b). Looking for innovation beyond the patent system: Evidence from research disclosures. Discussion Paper 656/2025, CRC TR 224, University of Bonn and University of Mannheim.

Gentzkow, M., Kelly, B., and Taddy, M. (2019). Text as data. *Journal of Economic Literature*, 57(3):535–574.

Gilardi, F., Alizadeh, M., and Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Goldsmith-Pinkham, P., Hull, P., and Kolesár, M. (2024). Contamination bias in linear regressions. *American Economic Review*, 114(12):4015–4051.

Gong, X. (2026). Firms, patents and open source code: Two crosswalks. Working paper.

Gortmaker, J. (2025). Open source software policy in industry equilibrium. Working paper, January 19, 2025; jeffgortmaker.com.

Habibi, M. (2025). Open sourcing GPTs: Economics of open sourcing advanced AI models. arXiv:2501.11581.

Haese, J. and Peukert, C. (2026). Open at the core: Moving from proprietary technology to building a product on open source software. *Management Science*, 72(5):4173–4199.

Hansen, S., Lambert, P. J., Bloom, N., Davis, S. J., Sadun, R., and Taska, B. (2023). Remote work across jobs, companies, and space. NBER Working Paper 31007, National Bureau of Economic Research. Revised January 2026.

Harhoff, D., Henkel, J., and von Hippel, E. (2003). Profiting from voluntary information spillovers: How users benefit by freely revealing their innovations. *Research Policy*, 32(10):1753–1769.

Hausman, J., Hall, B. H., and Griliches, Z. (1984). Econometric models for count data with an application to the patents-R&D relationship. *Econometrica*, 52(4):909–938.

Henkel, J. (2006). Selective revealing in open innovation processes: The case of embedded Linux. *Research Policy*, 35(7):953–969.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Horstmann, I., MacDonald, G. M., and Slivinski, A. (1985). Patents as information transfer mechanisms: To patent or (maybe) not to patent. *Journal of Political Economy*, 93(5):837–858.

Jaffe, A. B., Trajtenberg, M., and Henderson, R. (1993). Geographic localization of knowledge spillovers as evidenced by patent citations. *Quarterly Journal of Economics*, 108(3):577–598.

Kalliamvakou, E., Damian, D., Blincoe, K., Singer, L., and German, D. M. (2015). Open source-style collaborative development practices in commercial projects using GitHub. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 574–585.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.

Kelly, B., Papanikolaou, D., Seru, A., and Taddy, M. (2021). Measuring technological innovation over the long run. *American Economic Review: Insights*, 3(3):303–320.

Kerr, W. R. (2010). Breakthrough inventions and migrating clusters of innovation. *Journal of Urban Economics*, 67(1):46–60.

Kogan, L., Papanikolaou, D., Seru, A., and Stoffman, N. (2017). Technological innovation, resource allocation, and growth. *Quarterly Journal of Economics*, 132(2):665–712.

Lakhani, K. R. and von Hippel, E. (2003). How open source software works: "free" user-to-user assistance. *Research Policy*, 32(6):923–943.

Lee, J.-S. and Hsiang, J. (2020). Patent classification by fine-tuning BERT language model. *World Patent Information*, 61:101965.

Lerner, J. and Tirole, J. (2002). Some simple economics of open source. *Journal of Industrial Economics*, 50(2):197–234.

Lerner, J. and Tirole, J. (2005). The economics of technology sharing: Open source and beyond. *Journal of Economic Perspectives*, 19(2):99–120.

Lin, Y.-K. and Rai, A. (2024). The scope of software patent protection in the digital age: Evidence from Alice. *Information Systems Research*, 35(2):657–672.

Ludwig, J., Mullainathan, S., and Rambachan, A. (2026). Large language models: An applied econometric framework. *Annual Review of Economics*, 18:283–316.

Marx, M. and Fuegi, A. (2020). Reliance on science: Worldwide front-page patent citations to scientific articles. *Strategic Management Journal*, 41(9):1572–1594.

Medappa, P. K., Srivastava, S. C., and Favaron, S. D. (2025). Write access provisioning and organizational ownership in open source software projects: Exploring the impact on project novelty and survival. *Research Policy*, 54(8):105284.

Mummolo, J. and Peterson, E. (2018). Improving the interpretation of fixed effects regression results. *Political Science Research and Methods*, 6(4):829–835.

Nagaraj, A. and Piezunka, H. (2024). The divergent effect of competition on platforms: Deterring recruits, motivating converts. *Strategy Science*, 9(3):277–296.

Nagle, F. (2018). Learning by contributing: Gaining competitive advantage through contribution to crowdsourced public goods. *Organization Science*, 29(4):569–587.

Nanba, H. and Takezawa, T. (2009). Classification of research papers into a patent classification system using two translation models. In Kan, M.-Y. and Teufel, S., editors, *Proceedings of the 2009 Workshop on Text and Citation Analysis for Scholarly Digital Libraries*, pages 27–35, Suntec City, Singapore. Association for Computational Linguistics.

Parker, G. and Van Alstyne, M. (2018). Innovation, openness, and platform control. *Management Science*, 64(7):3015–3032.

Petralia, S. (2025). Open source software as digital platforms to innovate. *Journal of Economic Behavior and Organization*, 237:107109.

Petralia, S. and Boschma, R. (2026). Actionable disclosure in patents: How open code reduces spatial barriers to knowledge diffusion. Papers in Evolutionary Economic Geography 26.11, Utrecht University. SSRN Working Paper 7090442, version of 9 July 2026.

Pinto, G., Steinmacher, I., Dias, L. F., and Gerosa, M. (2018). On the challenges of open-sourcing proprietary software projects. *Empirical Software Engineering*, 23(6):3221–3247.

Qiao, M. and Huang, K.-W. (2025). Correcting measurement error in regression models with variables constructed from aggregated output of data mining models. *MIS Quarterly*, 49(1):29–60.

Raymond, E. S. (1998). The cathedral and the bazaar. *First Monday*, 3(3).

Raymond, E. S. (2001). *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. O'Reilly & Associates, Sebastopol, CA, revised edition. The essays are also at http://www.catb.org/~esr/writings/cathedral-bazaar/.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912.

Santos Silva, J. M. C. and Tenreyro, S. (2006). The log of gravity. *Review of Economics and Statistics*, 88(4):641–658.

Spinellis, D., Kotti, Z., Kravvaritis, K., Theodorou, G., and Louridas, P. (2020). A dataset of enterprise-driven open source software. In *Proceedings of the 17th International Conference on Mining Software Repositories*, pages 533–537.

Steinmacher, I., Conte, T. U., Gerosa, M. A., and Redmiles, D. F. (2015). Social barriers faced by newcomers placing their first contribution in open source software projects. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work and Social Computing (CSCW '15)*, pages 1379–1392.

Teece, D. J. (1986). Profiting from technological innovation: Implications for integration, collaboration, licensing and public policy. *Research Policy*, 15(6):285–305.

Thompson, P. and Fox-Kean, M. (2005). Patent citations and the geography of knowledge spillovers: A reassessment. *American Economic Review*, 95(1):450–460.

Valiev, M., Vasilescu, B., and Herbsleb, J. (2018). Ecosystem-level determinants of sustained activity in open-source projects: A case study of the PyPI ecosystem. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)*, pages 644–655.

von Hippel, E. (2005). *Democratizing Innovation*. The MIT Press, Cambridge, MA.

von Hippel, E. and von Krogh, G. (2003). Open source software and the "private-collective" innovation model: Issues for Organization Science. *Organization Science*, 14(2):209–223.

von Krogh, G., Spaeth, S., and Lakhani, K. R. (2003). Community, joining, and specialization in open source software innovation: A case study. *Research Policy*, 32(7):1217–1241.

Wang, S., McCormick, T. H., and Leek, J. T. (2020). Methods for correcting inference based on outcomes predicted by machine learning. *Proceedings of the National Academy of Sciences*, 117(48):30266–30275.

Webb, M. (2020). The impact of artificial intelligence on the labor market. Working paper, Stanford University. January 2020 version; SSRN Working Paper 3482150.

Wen, W., Ceccagnoli, M., and Forman, C. (2016). Opening up intellectual property strategy: Implications for open source software entry by start-up firms. *Management Science*, 62(9):2668–2691.

Wen, W., Forman, C., and Graham, S. J. H. (2013). The impact of intellectual property rights enforcement on open source software project success. *Information Systems Research*, 24(4):1131–1146.

Won, P. Y. (2026). Commoditize your complement: Strategic open-source releases and the cannibalization of rival firms. Conference draft, May 2026. https://www.paris-december.eu/sites/default/files//papers/2026/7456_sutari4697_2026_complete.pdf.

Wooldridge, J. M. (1999). Distribution-free estimation of some nonlinear panel data models. *Journal of Econometrics*, 90(1):77–97.

Wooldridge, J. M. (2023). Simple approaches to nonlinear difference-in-differences with panel data. *Econometrics Journal*, 26(3):C31–C66.

Wright, N. L., Nagle, F., and Greenstein, S. (2023). Open source software and global entrepreneurship. *Research Policy*, 52(9):104846.

Xu, K. and Quispe, A. (2026). Industry classification of GitHub repositories using the North American Industry Classification System (NAICS). arXiv:2607.06505.

Yamashita, K., Kamei, Y., McIntosh, S., Hassan, A. E., and Ubayashi, N. (2016). Magnet or sticky? measuring project characteristics from the perspective of developer attraction and retention. *Journal of Information Processing*, 24(2):339–348.

Zhang, Y., Zhou, M., Mockus, A., and Jin, Z. (2021). Companies' participation in OSS development—an empirical study of OpenStack. *IEEE Transactions on Software Engineering*, 47(10):2242–2259.

Zhou, M. and Mockus, A. (2015). Who will stay in the FLOSS community? Modeling participant's initial behavior. *IEEE Transactions on Software Engineering*, 41(1):82–99.

Zhou, S., Vasilescu, B., and Kästner, C. (2020). How has forking changed in the last 20 years? A study of hard forks on GitHub. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE 2020)*, pages 445–456. ACM.

Ziems, C., Held, W., Shaikh, O., Chen, J., Zhang, Z., and Yang, D. (2024). Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A The participation data in detail

- This appendix follows the data of section 4 from the event records to each estimation sample.
  - Appendix C.3 gives two further estimates, and the other parts describe the preparation.

A.1 Data sources and dates

- Table 6 lists the sources and their dates.
- The GitHub events are held only for the organizations the crosswalk links to firms, so activity on personal accounts and other organizations goes unobserved.
- The account link has two parts, the original links and the added organizations, each with its own event extract.
  - The account search ran in April and May 2026 on LLMs from several providers, through command-line agents, and a second search in July 2026 added 54 rows.
  - A first extraction selects 4,483 of the accepted accounts of Appendix A.4, 4,120 organizations and 363 users' accounts. Events are selected by organization, so the users' accounts carry none, and 3,955 organizations have events in 2015–2024.
  - The other 974 accepted accounts enter no extraction. Of these, 153 were no longer on GitHub in July 2026, 587 owned no public repository and 234 owned only forks.
  - A second extraction covers 2,555 of the 2,586 organizations behind repositories the account link added later through the second route of Appendix A.4, called the added organizations. The others are already in the first extraction.
  - Together the two extracts hold 99.7 percent of the OSS-active firms' events in 2016–2024.
  - The original links exclude forks, while the added organizations' repositories include them.
- Each repository's owner in each year comes from dated ownership, which follows repositories through acquisitions and the ends of firms' records.

Table 6: The sources of the participation data

Source	What is used	Years covered	Date
GitHub events	GH Archive in Google BigQuery: one row per organization, repository, event type, calendar quarter and actor, with the count of events, and no timestamps or contents	2015–2024	Corrected October 2026; the added organizations extracted October 2026
The January correction	The archive's yearly files for 2018–2020 file early January of the next year as the first quarter. A correction map moves, duplicates or restores those rows, and events dated before the repository's creation quarter are dropped	2018–2020 files	October 2026
Bot screen	Actors flagged as bots by name or by behavior. Actors the screen does not cover are kept	—	August 2026
Firm-to-repository links	The account crosswalk, with each repository's owner in each year from the dated ownership rule	—	October 2026
Creation dates and the archived flag	A snapshot of repository metadata and a listing of the added organizations	—	July and October 2026
Patents	Utility patents granted in 2015–2024, linked to firms by the patent crosswalk, with PatentsView's current CPC, inventive codes only, each main group sharing the patent's weight equally	2015–2024 grants	CPC processed August 2026
Repository labels	At most three CPC main groups from GLM-5.3-Flash, read from the README, description and topics, each with equal weight	—	Texts fetched August and October 2026

Notes: A dash marks a source with no year span of its own. The bot screen, the links and the snapshot are applied to the events of 2015–2024, and the labels are read once from texts fetched in 2026. The bot screen covers 6,354,670 actors and flags 12,153 of them. The paper's labels cover 59,789 of the population's repositories, and runs of the same prompts on the added organizations cover 43,198.

A.2 Coverage of the GitHub data

- The GitHub data observe public activity on the linked organizations only.
  - Private repositories lie outside the data, as they do for Cao et al. (2026).
  - Code hosted on other platforms lies outside the data (Lin and Rai, 2024).
  - The data also omit open-source code that employees release under their own accounts.
- The account crosswalk misses some organizations that linked firms run on GitHub.
  - One example is Google's People + AI Research group. Code on such organizations counts as code outside the linked firms.
  - Among 100 patent references drawn at random from those matching no linked project (Appendix D.4), one names such an organization.
  - A name search bounds the accounts the crosswalk misses (Appendix A.4).

A.3 Sample construction

- Table 7 follows the data from the event records to each estimation sample.
  - The firms are the 713 OSS-active firms of section 3.4.
  - A repository's firm is the OSS-active firm that owns it in its first-activity year. I call this the ownership rule.
  - The 111,009 repositories first active in 2015–2022 form the *population*.
- Panel A follows the firms from the crosswalk to the OSS-active firms.
  - Its first four rows apply the condition of section 3.4 (a five-year window with five patents and five labeled repositories) to the crosswalk's current links.
  - Under that condition, 756 firms pass.
  - Dated ownership moves repositories with acquisitions and with ownership records that end, so the OSS-active firms differ from these firms at the edges.
  - Of the OSS-active firms, 691 are among the firms that pass. Of the other 22 OSS-active firms, 5 pass every step but the window, and 17 lie outside the linked firms, 16 of them acquirers and the other a firm whose ownership record ends.
  - Of the firms that pass, 65 are outside the OSS-active firms.
- In Panel C, 173,027 of the 368,855 repositories with a held event under a linked organization have no ownership record in any year, because the crosswalk does not link them.
  - Of these, 41,806 are forks.
- Among the linked repositories, the *ownership rule*, which assigns each repository to the firm whose organizations hold it in its first-activity year (section 3.4), drops 54,604.
  - In their first-activity year, 49,215 belong to one of 1,901 linked firms outside the OSS-active firms.
  - Another 5,389 have ownership records but none in that year.
  - Of the 54,604, 8,061 have an OSS-active firm as owner in a later year, and they stay out.
- After the ownership rule, the cohort window drops 30,215 repositories and the label step drops 8,022.
- The 8,022 population repositories without a label split by their labeling record.
  - The labeler returned none for 2,336.
  - Another 3,914 were never sent, since the labeling runs take only repositories with README text.

Table 7: From the event records to the estimation samples

Step	Kept	Dropped	Reason
Panel A. Firms			
Firms the crosswalk links to a repository	2,755		
with a utility patent granted in 2015–2024	1,462	1,293	no patent on the patent crosswalk
and a labeled repository	1,424	38	no repository with a main-group label
and a five-year window with both	756	668	no window ending in 2019–2024 with five patents and five labeled repositories
OSS-active firms, on dated ownership	713		691 among the firms above. The other 22 through dated ownership
Panel B. Event rows			
Rows read, all linked organizations, 2015–2024	54,728,343		
without an identifier		3	actor, repository or organization missing
dated before the repository's creation quarter		1,876	the archive's misfiled January
by bots		1,579,538	actor flagged by the screen
Panel C. Repositories (firms in brackets)			
With a held event under a linked organization	368,855		
with no ownership record in any year		173,027	not linked by the crosswalk
owned in the first-activity year by a firm outside the OSS-active firms		49,215	the ownership rule
with ownership records, none in the first-activity year		5,389	the ownership rule
Owned by an OSS-active firm in the first-activity year	141,224		the owner of the first-activity year is kept for every later quarter
First active in 2015–2022	111,009 [695]	30,215	first active in 2023–2024, with fewer than eight quarters of follow-up
With main-group labels	102,987	8,022	no label
With a single firm	102,987 [693]	0	two firms in the first-activity year
of which inside	82,046		
Panel D. Estimation samples			
Columns 1 and 2, linear-probability rows	102,980 [686]	7	firms with one repository
Columns 1 and 2, Poisson, new participants	102,827 [658]	153	firms with no new participant on any repository
of which forks	6,572		all but 2 among the repositories of the added organizations
Column 4, a ranked first main group	100,111 [692]	2,876	no usable ranked first main group
Column 4, linear-probability rows	98,795 [667]	1,316	cells with one repository
Column 4, Poisson, new participants	96,854 [626]	1,941	cells with no new participant
of which forks	6,204		all but 1 among the repositories of the added organizations

Notes: Panel A counts firms. The counts in Panels C and D are repositories, with firms in brackets where they are recorded. In their first-activity year, 13 repositories have two OSS-active firms as owners. The cohorts of 2015 to 2022 hold 14,242, 9,821, 11,201, 11,959, 13,960, 17,001, 16,654 and 16,171 repositories. Of the 2,876 repositories without a ranked first main group, 1,155 have a name that collides with another's, 607 have an ambiguous first choice, and 1,114 have no matching ranked answer. In the Poisson rows, the other outcomes keep slightly different counts, since each drops the firms or cells whose outcome is zero throughout.

  - Calls failed for 17, 20 have a label that the final weights file omits, and 1,735 have no record among the held labeling runs.

- Column 3 of Table 4 starts from the repositories of every firm the crosswalks link to a repository.
  - That population holds 153,610 repositories, and the comparison holds 107,018 of them, of 1,110 firms.
  - Of the repositories outside the comparison, 32,451 have no grant of their firm in the five years up to their first activity, so inside is not defined for them (section 3.4).
- Within 24 months, 9,826,995 actors are new to a population repository, and 1,402,368 of them are new participants.

A.4 Assigning repositories to firms

The crosswalks.

- Neither Compustat nor the patent records list a firm's GitHub accounts, so I build the link from firms to their open-source code.
  - Spinellis et al. (2020) use committers' email domains to find company-developed GitHub projects, but they record no technology classes and no patents.
  - A search looks for the GitHub accounts of 75,293 Compustat records with LLMs from several providers, run through command-line agents that search the web.
  - The search model is told to return only official GitHub root pages that it has verified, preferring the firm's own web pages as evidence.
  - Every page the model labels as found and verified is accepted without a further check, provided its address names a GitHub account.
  - This gives 5,457 accounts for 5,403 of these Compustat records.
  - An account can be an organization or a user's, and only organizations' events are extracted (Appendix A.1).
  - A second route adds organizations whose public profile declares a firm's web or email domain, or that match a firm's subsidiaries by name or domain, after an LLM judges from the profile whether the organization is the firm's own (Gong, 2026).
  - It adds 78,762 repositories to the 130,590 linked through the accepted accounts.
- Against the data of Spinellis et al. (2020), the account link names the dataset's firm for 247 of the 263 accounts both hold (93.9%).
  - Fifteen of the disagreements were read by hand.

  - Of these, 10 differ in the level of the firm, mostly through acquisitions, 3 favor the link and 2 are unclear.
  - This measures precision on the overlap with that dataset only.
  - As an independent check of agreement, not of accuracy, a second search, run with another LLM, repeated the search for a random sample of the records drawn separately within groups of records (strata).
  - Bailey et al. (2020) check a linkage of historical records the same way, against a second linkage of a random sample that is built without sight of the first.
  - Over the 182 records the second search finished, it agrees with the original search on 94% of records, weighted by the strata's sizes as in Marx and Fuegi (2020).
  - On the records the original search accepted, it accepts the same account for 59.8% (95 percent interval 49.6% to 69.2%), and it accepts none for 32 of the 92.
  - Most records have no accepted account in the original search, so the agreement over all records is higher than over the accepted ones.

- A second crosswalk, built separately, links Compustat records to PatentsView assignee identifiers.
  - The release used here, restricted to utility grants of 2015–2024, holds 1,948,899 patents in 2,072,295 firm–patent links, held by 13,651 firms at the top of their ownership trees.
  - Over the grant years it covers, an earlier release links 22,312 Compustat records to their patents.
  - I accept a link only with evidence for the exact firm and assignee identifiers, never on a match of names alone, and each link records the kinds of evidence behind it.
  - Links whose strongest evidence is a published crosswalk make up 78.7% of the firm–patent links.
  - LLMs from several providers, run through command-line agents, assisted the identification of the firm for 6.91% of the firm–patent links.
  - In the earlier release, 443,295 of the 738,519 pairs counted as LLM-assisted come from a cleaning of assignee names that involves no LLM.

- I validate the patent crosswalk against two published crosswalks as baselines, on the years each shares with it.
  - It covers 99.7% of the 2015–2021 patents that DISCERN (Arora et al., 2021) links to firms headquartered in the United States, and 96.3% of the 2015–2024 patents that the extended data of Kogan, Papanikolaou, Seru and Stoffman (2017, KPSS) link to CRSP firms.

  - Both sources are among the crosswalk's inputs, so agreement with them is partly built in.
  - Among links that rely on no published crosswalk, KPSS names the same firm for 92.6% of the 180,463 2015–2024 patents both link, and DISCERN for 2.5% of the 10,258 2015–2021 patents both link.
  - The low figure for DISCERN describes a selected remainder, not the crosswalk at large.
  - The crosswalk takes published links first, DISCERN's among them, so these are the patents for which it took no published link.
  - Over all 592,427 patents that both link, DISCERN names the same firm for 96.6%.
  - The patents linked without a published crosswalk rest on other evidence, mostly evidence for the assignee's own firm, whereas DISCERN assigns each patent to its ultimate owner in the grant year.
  - KPSS, which names the assignee's own firm, agrees on most patents linked without a published crosswalk.
  - A disagreement with DISCERN can reflect this difference in convention or an error in either link.

- A third crosswalk serves as a held-out check.
  - The Global Corporate Patent Dataset (GCPD) of Bena et al. (2017) and Barkemeyer et al. (2026) never entered the crosswalk, so agreement with it is not built in.
  - Where both link a 2015–2022 patent to a Compustat firm, GCPD names the same firm for 92.8% of the patents.
  - Abramitzky et al. (2021) and Bailey et al. (2020) likewise check automated links of historical records by their agreement on the records that two separately built linkages both link.

- Neither DISCERN nor KPSS links firms to open-source code, so a name search can bound the accounts the GitHub crosswalk misses.
  - From patenting firms with no accepted account, I draw a stratified random sample and search GitHub's public index for each firm's name.
  - An account with a matching name exists for 39.4% of them (stratified 95 percent interval 34.3% to 44.6%).
  - The share counts as non-matches the firms whose search returned only logins that do not match the name.
  - If the name search finds every account the crosswalk missed, that share bounds the misses from above, since the matching accounts are unchecked for activity or for a profile that names the firm.

  - Marx and Fuegi (2020) likewise estimate the matches a linkage misses from a random sample searched by another route.
  - Appendix A.2 gives the organizations the account crosswalk misses, and Gong (2026) documents both crosswalks in full.

- The counts change meaning along the chain from Compustat records to the repositories studied, since the search counts Compustat records and the later steps count firms at the top of the ownership tree.
  - A firm's subsidiaries hold records of their own, and the accepted accounts resolve to 3,878 firms at the top of their ownership trees.
  - These accounts and the added organizations link 209,352 repositories to 2,755 firms.
  - Of these firms, 1,462 hold a patent granted in 2015–2024, 1,424 also a labeled repository, and 713 are the OSS-active firms (section 3.4).
  - Events are extracted for whole organizations, so they also reach repositories the crosswalk does not link.
  - Of the 368,855 repositories with a held event that is not a bot's, 173,027 have no ownership record, and 141,224 belong to an OSS-active firm in their first-activity year (Appendix Table 7).
  - The patent crosswalk counts firms on its own side, 13,651 with a linked patent, most of which have no linked account.

The repositories.

- A repository is a GitHub repository with at least one held event under a firm-linked organization.
  - Events are attributed to the organization the event names, so a repository that moved between organizations carries each organization's events.

- Its start, q_0, is the first calendar quarter with a held event of any type by an actor who is not a bot, after events dated before creation are dropped.
  - The cohort y_0 is the year of q_0.
  - The start is the first held activity under a firm-linked organization, which may come after the repository's creation.
  - Of the population, 96,131 repositories were created in the four quarters up to q_0, among the 107,031 that hold a creation date.
  - The other 3,978 hold none, so their events cannot be checked against creation.

- The firm is the OSS-active firm that owns the repository in its first-activity year under dated ownership, and it stays the firm through any later acquisition.
  - The firm's organizations are all the held organizations observed on any repository assigned to that firm, in any year of 2015–2024.
- The build keeps forks, archived repositories, empty repositories and mirrors, with no screen for any of them.
  - A repository enters only with a held event that is not a bot's, and enters the comparison only with a label.
- Forks remain among the added organizations' repositories, since forks were removed from the original links only.
  - In column 1's Poisson rows for new participants, 6,572 of the 102,827 repositories are forks, 6.4 percent of the rows, 5.4 percent of those inside and 10.4 percent of those outside.
  - All but 2 belong to the added organizations, and column 4's rows hold 6,204.
  - Appendix C.3 drops them.
- The archived flag is read from 2026 records and enters no sample rule.
  - It marks 33,346 of column 1's rows, 34.9 percent of those inside and 22.6 percent of those outside.
  - It describes 2026 rather than the outcome window.
- Mirrors stay in the samples uncounted, since the held fields do not mark them.
  - A mirror's pushes by an account the bot screen flags drop out as bot events.

A.5 Identifying new participants

- The held event types fall into the three families of section 3.3 (Table 8).
  - A contribution event records an attempt or a discussion, such as a pull request or a comment, so the record leaves open whether a pull request was merged.
- The held record carries no timestamps within a quarter and no event contents.
  - An issue opened and an issue closed are both issue events, a pull request opened, closed or merged is a pull-request event, and the commits inside a push are not counted.
- On the 102,987 repositories of the main estimates (section 3.4), the records hold 114,936,122 events of every type by 5,994,624 distinct actors (Table 8).

– Of these, 62,496,484 are contribution-family events, by 2,002,193 distinct people, and 69,937 of the repositories have at least one participant.

Table 8: The event types of the GitHub records, on the repositories of the main estimates

Family and event type	GH Archive name	Events	Distinct actors
Attention		24,053,907	5,094,117
Stars	`WatchEvent`	18,272,218	
Forks	`ForkEvent`	5,781,689	
Contribution		62,496,484	2,002,193
Issues opened, closed or reopened	`IssuesEvent`	7,420,964	
Issue comments	`IssueCommentEvent`	20,359,715	
Pull requests opened, closed or merged	`PullRequestEvent`	12,978,524	
Pull-request reviews	`PullRequestReviewEvent`	10,603,067	
Pull-request review comments	`PullRequestReviewCommentEvent`	11,005,148	
Commit comments	`CommitCommentEvent`	129,066	
Write and administration		28,385,731	134,182
Pushes	`PushEvent`	18,625,845	
Branch or tag created	`CreateEvent`	4,986,852	
Branch or tag deleted	`DeleteEvent`	3,677,901	
Releases	`ReleaseEvent`	447,760	
Collaborators added	`MemberEvent`	362,558	
Repository made public	`PublicEvent`	38,230	
Wiki edits	`GollumEvent`	246,585	
All event types		114,936,122	5,994,624

Notes: Held events of 2015–2024 on the repositories of the main estimates (section 3.4), by actors the bot screen does not flag, with events dated before the repository's creation quarter dropped. The held record has one row per organization, repository, event type, calendar quarter and actor, with the number of events, and every event type is held in every year. Participation is a contribution-family event. Of the outcomes of section 3.3, the stars count watch events, and the one on whether new participants stay counts contribution- and write-family events. An actor active in several families is counted once in each, so the families' distinct actors sum to more than the total. A push can carry several commits, and the records do not count commits.

- A new participant is an actor who is not a bot and makes a first contribution-family event on the repository in quarters $q_0 + 1$ to $q_0 + 8$.
  - The actor has no held event of any type on the repository in any quarter up to and including q_0.
  - Actors active in q_0 are the founding-quarter actors and never count as new.
  - Every population repository has eight quarters of follow-up, since the last held quarter is the fourth of 2024.
- A participant new to the firm is a new participant with no held event of any type on any repository of the firm's organizations before the first held event on this repository.

  - The group is defined by event histories, and no actor is matched to an employer.
  - New hires, staff of an unlinked subsidiary, and staff whose earlier work sat in personal accounts all count as having no earlier event, so the group may overstate the number of participants from outside the firm.
  - Events before 2015 are not held, so the look-back is shortest for the 2015 cohort.
  - An actor whose first events on the firm's organizations and on the repository fall in the same quarter counts as having no earlier event.

- A participant new to the firm who was active at another linked firm in the four quarters before arriving has at least one held event in those four quarters on an organization that is not the firm's.
  - The group requires activity at another linked firm.
  - "Another linked firm" covers the linked firms only, not all of GitHub.

- Bots are dropped from every count, including the first-activity quarter, and a bot the screen misses counts as a person.

- Actors carry no employer, so the design treats the firm's own staff in three ways.
  - The founding-quarter actors include whoever published the repository and never count.
  - Staff with earlier events on the firm's organizations are new participants but not participants new to the firm.
  - Staff without such events count among the participants new to the firm.

- New participants are split by their held history before arrival into three groups (Table 9).
  - With a history at the firm: at least one held event of any type on the firm's organizations before the quarter of the participant's first held event on the repository.
  - With a history only at another linked firm: no such event, and at least one on another linked firm's organizations.
  - With a history at no linked firm: no held event on any linked firm's organizations, the firm's included.
  - The last group still includes staff new to the linked organizations and people active only on personal accounts or unlinked organizations, so it covers the linked organizations rather than all of GitHub.
  - The full history runs from 2015 to that quarter. A variant stops it at q_0, and another keeps an equal 16-quarter look-back on the cohorts of 2019–2022.
  - Across all new participants, 509,351 of the 853,648 distinct people have a history at no linked firm, on at least one repository.

Table 9: New participants' held histories before arrival

	With a history at the firm	Only at another linked firm	At no linked firm	All
Panel A. Share of new-participant pairs (percent)				
Inside repositories	39.5	21.0	39.5	
Other repositories	27.1	34.4	38.5	
Panel B. Distinct people				
Inside repositories	242,089	202,861	436,314	751,045
Other repositories	34,657	63,248	76,482	160,219
All	262,671	247,166	509,351	853,648

Notes: New participants in the reference repositories, by their held events before their first event on the repository. A pair is one person on one repository. Panel A's rows may not sum to one hundred because of rounding. In Panel B, a person new on several repositories can fall in different groups on each, so the groups' counts sum to more than the total. Each group is named by where its people were active before arriving. Participants new to the firm are the last two groups together, and every one of them active at another linked firm in the four quarters before arriving falls in the middle group.

- New participants who arrive in the repository's first year are also split by what they do in the four quarters after arriving, into four groups (Table 10).
  - Two questions place each person, both on held contribution-family events: any on another repository of the firm's organizations, and any on another linked firm's organizations.
  - Go on to neither: no later contribution on the firm's other repositories or at another linked firm in the four follow-up quarters.
  - Go on to another linked firm only: a later contribution at another linked firm and none on the firm's other repositories.
  - Go on to the firm's other repositories only: a later contribution on the firm's other repositories and none at another linked firm.
  - Go on to both: later contributions on the firm's other repositories and at another linked firm.
  - Two unions combine the groups.
    - Go on to none of the firm's other repositories: the groups that go on to neither, or to another linked firm only.
    - Go on to no other linked firm: the groups that go on to neither, or to the firm's other repositories only.
  - The arrival quarter itself is left out of the follow-up, since the order of events within a quarter is not held. Counting it moves 11.27 percent of pairs to another group.

  - The follow-up ends by $q_0 + 8$, so every person has the same four quarters.
  - Pushes are write-family events, so they place no one.

- The groups are defined by observed events, so they describe who arrives. No actor is matched to an employer, so the people who go on to the firm's other repositories only are defined by their later contributions there and are not known to be staff.

Table 10: New participants by what they do in the four quarters after arriving

	Go on to				Go on to	
	neither	another linked firm only	the firm's other repositories only	both	none of the firm's other repositories	no other linked firm
Panel A. Share of new-participant pairs (percent)						
Inside repositories	52.12	13.87	23.65	10.36	65.99	75.77
Other repositories	51.68	21.58	16.91	9.82	73.26	68.59
Panel B. Distinct people						
Inside repositories	293,804	67,926	78,750	32,156	349,432	356,805
Other repositories	54,945	21,163	11,093	6,623	75,146	64,532
All	344,895	82,490	86,857	36,189	409,293	412,877

Notes: New participants who arrive in quarters $q_0 + 1$ to $q_0 + 4$, followed for the four quarters after their arrival quarter: 707,439 pairs of 482,024 distinct people, of whom 25,251 are new on both an inside and an outside repository. A person new on several repositories can fall in different groups on each. "Another linked firm" covers the linked firms' organizations only.

A.6 The distribution of the outcomes and controls

- Table 11 gives the full distribution of each outcome and founding control of the regression dataset, for all, inside and other repositories. Table 3 gives their means and standard deviations.

Table 11: The distribution of the outcomes and controls in the regression dataset

	N	Mean	SD	Median	p75	p90	Max	% above 0	Total
Panel A. All repositories									
All new participants	102,987	13.45	147.94	1	5	18	23,766	53.5	1,385,308
new to the firm, no earlier activity there	102,987	8.39	114.35	0	2	10	18,864	41.1	863,849
of those, active at another linked firm in the prior year	102,987	2.36	30.4	0	0	3	6,102	23.9	242,681
Contributions by new participants	102,987	110.24	1098.11	1	17	108	101,937	53.5	11,353,727
Stars by new users	102,987	68.76	716.36	2	13	65	76,518	66.0	7,081,885
Maintained in the second year (0/1)	102,987	0.422	0.494	0	1	1	1	42.2	43,509
Long-term contribution, per arrival (0/1)	651,778	0.115	0.319	0	0	1	1	11.5	75,119
Attention events in q_0	102,987	25.19	286.02	1	4	15	29,013	57.1	2,594,130
Contribution events in q_0	102,987	19.41	126.63	0	5	30	8,584	39.9	1,998,856
Write events in q_0	102,987	18.98	75.61	5	14	40	10,492	93.5	1,954,323
Founding-quarter actors	102,987	26.34	275.55	3	6	17	27,094	100.0	2,712,361
Text length (characters)	102,987	2951.38	2218.62	2,491	5,498	6,062	6,800	97.4	—
Panel B. Inside repositories									
All new participants	82,046	14.36	160.59	1	5	19	23,766	55.2	1,178,206
new to the firm, no earlier activity there	82,046	8.69	122.96	0	2	10	18,864	42.2	712,872
of those, active at another linked firm in the prior year	82,046	2.24	30.44	0	0	3	6,102	23.9	183,873
Contributions by new participants	82,046	119.82	1187.03	1	19	118	101,937	55.2	9,830,359
Stars by new users	82,046	71.95	760.6	2	14	70	76,518	68.7	5,903,317
Maintained in the second year (0/1)	82,046	0.43	0.495	0	1	1	1	43.0	35,264
Long-term contribution, per arrival (0/1)	546,091	0.116	0.32	0	0	1	1	11.6	63,448
Attention events in q_0	82,046	26.6	304.82	1	4	16	29,013	58.9	2,182,152
Contribution events in q_0	82,046	20.68	135.17	0	5	32	8,584	40.7	1,696,994
Write events in q_0	82,046	19.27	80.37	5	14	40	10,492	93.9	1,581,233
Founding-quarter actors	82,046	27.75	293.85	3	6	18	27,094	100.0	2,276,736
Text length (characters)	82,046	3029.2	2219.18	2,619	5,707	6,067	6,800	97.5	—
Panel C. Other repositories									
All new participants	20,941	9.89	81.13	0	4	13	5,602	46.9	207,102
new to the firm, no earlier activity there	20,941	7.21	71.16	0	2	8	4,933	36.4	150,977
of those, active at another linked firm in the prior year	20,941	2.81	30.25	0	0	3	2,472	23.9	58,808
Contributions by new participants	20,941	72.75	638.81	0	10	70	44,098	46.9	1,523,368
Stars by new users	20,941	56.28	506.95	1	7	42	39,656	55.3	1,178,568
Maintained in the second year (0/1)	20,941	0.394	0.489	0	1	1	1	39.4	8,245
Long-term contribution, per arrival (0/1)	105,687	0.11	0.313	0	0	1	1	11.0	11,671
Attention events in q_0	20,941	19.67	195.56	0	2	11	9,220	49.9	411,978
Contribution events in q_0	20,941	14.41	85.12	0	4	22	3,974	36.7	301,862
Write events in q_0	20,941	17.82	52.95	5	14	38	2,334	91.7	373,090
Founding-quarter actors	20,941	20.8	187.29	2	5	13	8,650	100.0	435,625
Text length (characters)	20,941	2646.49	2,190	2,039	4,703	6,043	6,576	96.9	—

Notes: The rows are the reference set's repositories (Table 2), except the long-term-contribution row, which has one row per new participant who arrives within 12 months on a repository first active in 2015–2021. The outcomes count the 24 months after the first-activity quarter q_0, and Table 1 defines the groups of new participants. Stars count watch events by actors whose first held event on the repository falls in the window, not every star. Maintained means a push or a release in quarters $q_0 + 5$ to $q_0 + 8$. The founding controls of section 4.1 enter the regressions as the logarithm of one plus each. The text is the repository's as it stood in 2026, after every outcome window, while the other controls are measured at or before q_0. Text length is the README's characters up to 6,000 plus the description's up to 400 from each held record of it, so a description can be counted twice, and the largest value is 6,800 characters.

A.7 Measuring inside

- A repository's labels are at most three CPC main groups from the labeler, each with weight $1/k$ for k labels.
- The firm's patented main groups are those where it holds positive patent weight in grant years $y_0 - 4$ to y_0, the first-activity year included.
  - Grants are held from 2015, so the window $y_0 - 4$ to y_0 is truncated for the earliest repositories, whose patented main groups rest on fewer grant years.
- With s the sum of the repository's label weights over the firm's patented main groups, the repository is inside if $s \geq 1/2$.
  - That is, at least half its label weight lies where the firm patents.
  - With equal weights, a repository with three labels is inside when two of them fall in those main groups, and one with two labels when one does.
- Other cuts of s give these counts of inside repositories among the 102,987.
  - Any weight in those main groups gives 83,001.
  - At least two thirds gives 72,041.
  - All of it gives 70,489.
  - The CPC subclass in place of the main group, at one half, gives 92,365.

What the labels show.

- A labeled open-source project spans fewer main groups than a patent.
  - Among the 2015–2024 patents and labeled projects of the 713 OSS-active firms, a patent carries 3.01 distinct main groups on average, and a labeled open-source project carries 1.78.
- Firms' open-source code lies mainly in software engineering and computing, while their patents span computing, communications and the physical sciences (Table 12).
  - CPC section G (physics, including computing) holds 88.7% of the firms' open-source code and 39.38% of their patents.
  - Section H (electricity, including communications) holds 39.68% of the patents, and sections A to F, the physical sciences, hold 20.95%.
  - Four main groups hold 60.4% of the open-source code, and software engineering (G06F 8/00) alone holds 27.73%.

Table 12: The distribution of open-source code and patents across patent classes

Technology		Share of open-source code	Share of patents
Panel A. Main groups			
G06F 8/00	Software engineering	27.73%	0.75%
G06F 9/00	Architectures and processors	16.31%	2.5%
G06F 16/00	Databases and retrieval	9.13%	3.74%
G06F 11/00	Error detection and correction	7.23%	1.76%
Top four main groups		60.4%	
Panel B. Sections			
G	Physics, including computing	88.7%	39.38%
H	Electricity, including communications	10.67%	39.68%
A–F	Physical sciences and mechanics	0.63%	20.95%

Notes: The technologies are main groups and sections of the Cooperative Patent Classification (CPC). The table pools the five-year windows of the 713 OSS-active firms. Each column, open-source code or patents, pools the windows with at least five projects or five patents. The windows overlap, so a project or patent active in several of a firm's windows counts in each. Each open-source project is weighted by its events, unlike the inside rule of section 4.1, which gives each repository's labels equal weight. Each patent's weight is split equally across its main groups. Each column gives the technology's share of that column's pooled weight.

A.8 Limitations

- *The label text was captured in 2026.* Of the 102,987 repositories with labels and a single firm, 99,519 have a dated text, and none was captured before the end of its outcome window.
  - The capture follows first activity by 3.87 to 11.81 years, with a median of 7.31.
  - Text written during or after the window can shape the label, and no dated history of the text is held.
- *The labels are a model's.* The outcome-blind review of Appendix B.1 covers only repositories near the threshold, and its reviewers were LLM agents.
  - The other labelings of Appendix C.3 cover a little over half of the labeled repositories at most, all of them in the original links.
- *The history starts in 2015.* The look-back is shortest for the earliest cohorts, and first observed participation is not necessarily first-ever participation.
- *Only linked organizations are held.* Activity on personal accounts and other organizations goes unobserved, so "no earlier event" describes the linked firms only.
- *The clock is quarterly, and events carry no contents.* The order of events within a quarter is unknown, and so is whether a pull request was merged.
- *The archive misfiles January.* The corrected files and the creation check remove what can be dated.

  - For the first quarters of 2018, 2019 and 2020, a second version of the shares leaves out the participants who arrive in those quarters, and the gap between the two versions bounds the remainder.
  - Those quarters hold the start of 9,755 of the population's repositories.

- *The firm's staff are not identified,* and the group of participants new to the firm may overstate the number of participants from outside the firm.

- *The bot screen is a screen.* A bot it misses counts as a person.

- *The owner is fixed at the first-activity year.* A repository whose owner in that year is not an OSS-active firm is outside the frame, whatever owns it later.

- *Forks are dropped from the original links only, and archived repositories and mirrors stay in.* The check of Appendix C.3 drops the added organizations' forks as well.

- *The patent window is truncated for the earliest repositories and includes grants of the first-activity year.*

- *Column 4 needs a ranked first main group.* The repositories without one are left out, never imputed.

- *Samples differ by outcome,* since each Poisson estimate drops the groups whose count is zero throughout.

- *Creation dates come from 2026 records,* and 3,978 population repositories have none.

A.9 A case study of one repository

- The repository facebookresearch/fastMRI passes every step and shows the overlap rule with one label outside the firm's patented main groups.

- *Link and owner.* It sits under the facebookresearch organization, and in every year from 2018 to 2024 the ownership rule assigns it to Meta Platforms, one of the OSS-active firms.

- *Events.* The record holds 2,278 rows with 3,810 events from the fourth quarter of 2018 to the fourth quarter of 2024, all under one organization.
  - Flagged bots account for 35 rows, which are dropped, and every event falls at or after the repository's creation.

- *Start and cohort.* Its first-activity quarter is the fourth of 2018, and it was created on 26 October 2018.
  - It has 151 founding-quarter actors, none of whom can count as new.

- *Labels.* Its README was fetched on 13 August 2026, and the model returned three main groups, each with weight one third.
  - They are G06N 20/00 (machine learning), G06T 5/00 (image enhancement or restoration) and G16H 30/00 (handling of medical images).
- *Inside.* Meta holds patent weight in G06N 20/00 and G06T 5/00 in 2015–2018 and none in G16H 30/00 in any year, so s is two thirds and the repository is inside.
  - It stays inside at any weight and at two thirds, falls outside when all weight must lie in the firm's patented main groups, and is inside under the subclass rule.
- *New participants.* Of 551 actors new to the repository in 2019–2020, 43 make a contribution and are new participants.
  - Of these, 29 have no earlier event on Meta's organizations, and 4 of the 29 were active in the year before at another linked firm.
  - The 43 make 231 contribution events, and new actors give 423 stars.
- *Samples.* It is in the samples of columns 1, 2 and 4.

A.10 Separated observations

- Each Poisson estimate drops rows in two ways, groups with a single repository and groups whose count is zero throughout.
  - In columns 1 and 2, 7 rows belong to firms with a single repository, and 153 belong to firms whose new-participant count is zero on every repository.
  - In column 4, 1,316 rows sit in cells with a single repository, and 1,941 sit in cells whose count is zero throughout.
  - The estimator's later check for separation keeps every remaining row.

B The validation of the labels

- This appendix validates the repository labels of section 3.2 and reports how well they reproduce.
  - It then describes what the labeler reads, the methods that could label open-source code, a second check on patents, the coverage of the sample and the labels' limits, and one repository's labels.
  - It opens with the two label checks and what they mean for the inside indicator (section B.1).

B.1 Assessing the labels

- I check the labels against two references built independently of them (Table 13).
  - My labels share at least one main group with the EPO's text categorizer for 76.4% of the projects both classify, against 38.5% with my labels assigned at random within the reference's CPC section.
  - The second check tests the two-stage prompt on patents, whose classes the examiners' classifications give.
  - A different LLM, DeepSeek-V4-Flash, applies the same two stages to patent titles and abstracts. Its labels share a main group with the examiners' for 89.1% of them, against 20.8% at random.
- Each observed share is set against the share when the label sets are dealt out at random among items of the same CPC section, the classification's broadest level.
  - In the examiners' check, the share at random plays the role of the agreement expected by chance in Cohen (1960), drawn within strata of the classification as Jaffe et al. (1993) draw their control patents.
  - The section is the broadest stratum, so the gap between the observed share and the share at random can overstate the agreement beyond chance (Thompson and Fox-Kean, 2005).

Table 13: The two label checks

Reference	Items labeled	Items	Share with a common main group	
			Observed	At random
The EPO's text categorizer	Open-source projects, by GLM-5.3-Flash	721	76.4%	38.5%
			[73.2%, 79.4%]	[35.9%, 41.1%]
Patent examiners	Patents, by DeepSeek-V4-Flash	2,774	89.1%	20.8%
				[18.0%, 23.6%]

Notes: Each row compares labels from the two-stage prompt with a reference built independently of them. In the first row, the labels are this paper's labels for open-source projects, and the reference is the European Patent Office's text categorizer. In the second row, another LLM applies the same two stages to the titles and abstracts of patents, and the reference is the examiners' classifications. The observed share is the fraction of items whose labels share at least one main group with the reference. For the share at random, I assign the labels to the items at random within the reference's CPC section. Brackets give the 95 percent Wilson interval for the first row's observed share, and the 2.5th and 97.5th percentiles of the random draws for each share at random. No interval is held for the second row's observed share. Appendix B.2 reports the first check in full, and Appendix B.6 the second.

- These checks measure agreement with other classifications rather than accuracy, and further checks bear on the inside indicator.
  - The comparison of section 4 classifies each repository by its own labels, so a label error can move one repository across the inside threshold (section 4.1).
  - In an outcome-blind review of 305 labels on 120 repositories near that threshold, 4.9 percent of the labels were judged not to fit (95 percent interval 3.0 to 8.0 percent).
  - The reviewers were LLM agents rather than people.
  - Section 4.5 varies the cut on the share of label weight that defines inside, and the CPC level at which it is drawn.
  - A second run of the labeler's second stage changes the indicator for 3.9 percent of the repositories it covers, since label sets change between runs even at temperature zero (section B.3).
  - The second labeler's labels, from the second stage, change it for 13.2 percent, mostly from inside to outside.
  - On the rows the second labeler covers, the column-1 estimates for new participants and contributions are close under the two labelings, and three of the four are significant, while the paper's labels for new participants pass Holm's adjustment only at the ten-percent level (section 4.2).
  - On those rows, the within-technology comparison is weak under both labelings. Its column-4 estimates are smaller than on the full sample. For new participants, the

estimate under the paper's labels misses Holm's adjustment, and the estimate under the second labeler's passes it. For contributions, only the paper's labels pass.

  - Appendix C.3 reports this check and its limits.

B.2 Agreement with the EPO's text categorizer

- I measure agreement with references that are themselves automated.
  - The EPO's text categorizer serves as a check rather than as the labeler, because of its limits on input length and time (Table 14).
  - An audit compares the labels with the categorizer on 500 repositories.
  - Cohen's κ, agreement net of chance (Cohen, 1960), is 0.3444 (standard error 0.0248) at the main group on the READMEs the labeler reads, over the 432 items that both classify.
  - On those items, the two primary codes agree on 40.05%.
  - The categorizer reads patent prose, so an LLM also writes a patent-style abstract from each repository's full README. On that abstract-derived text, κ is 0.4521.
- In the literature, automated CPC classification has been benchmarked on patents, and my labels classify repositories at the main group.
  - The benchmarks run from a fine-tuned BERT model at the subclass (Lee and Hsiang, 2020) to encoders and open-weight LLMs whose evaluation stops at the subclass (Emer et al., 2026).
  - For repositories, the CPC has no human-coded gold standard, and agreement differs from accuracy.
  - Two automated systems can agree for shared reasons.
  - In other tasks, prompted LLMs label below the best fine-tuned classifiers while reaching fair agreement with human labels (Ziems et al., 2024).
  - As Dell and Rambachan (2026) argue, a measurement built with an LLM needs a validation sample drawn from the data it measures.
  - Such a sample serves to correct the downstream estimates as well as to report agreement (Duan and Pelger, 2026).
  - Validation data can remove the bias that measurement error brings into estimates built on predicted labels (Ludwig et al., 2026).
- As a pilot, I coded a probability sample by hand and scored it against the *production labels*, the labeler's labels that every estimate uses.
  - On 50 items, my coding shares at least one main group with the production labels in 34, carries 0.49 of their label weight, and equals the production set in 9 items.

  - This result is agreement with one expert's coding on one sample, short of an accuracy rate or a gold standard.
  - I coded the sample alone, so two-coder reliability is unavailable.

- For a wider comparison, I classified 800 repositories through the categorizer's web page.
  - The sample holds the pilot's items and new ones drawn by the pilot's rule.
  - A scoring rule declared before any score was read treats an item without a code from the categorizer as empty.
  - Such an item is counted and left out of the pairwise measures, never scored as disagreement.
  - The categorizer returned a code for 91% of the items.

- On the 721 items where both return codes, the production labels share a main group with the categorizer on 76.4% of items (95 percent Wilson interval 73.2% to 79.4%).
  - At the subclass, they share a code on 90.4%.
  - They agree on the exact set on 10.7%, with a mean Jaccard overlap of 0.369.
  - Of the production labels' weight, 56.4% falls inside the categorizer's set.
  - With the labels assigned at random within the categorizer's CPC section, the share with a common main group is 38.5%.
  - Most of the disagreement lies within a subclass.

- The model lists its main groups in order, and its first main group matches the categorizer's first code more often than chance would give.
  - Among the 716 items whose ordered answer matches the production labels, the model's first main group is the categorizer's first code on 43.9%.
  - With the production label sets dealt out at random among the items in the same CPC section as the categorizer's first code, the share is 14.5%.

- Against the same reference, the second labeler agrees less, and my own coding sits between the two.
  - The second labeler, an anonymous model served as `space-bunny-alpha`, is distinct from the labeler and from DeepSeek-V4-Flash of Appendix B.6.
  - Its labels share a main group with the categorizer on 43.3%, with a mean Jaccard overlap of 0.208.
  - On the pilot items, my coding shares a main group with the categorizer on 63.4%.
  - These figures measure agreement with an automated reference, short of an accuracy rate or a gold standard.

- The identity-leakage check asks whether the labeler places a repository in its owner's patent technologies for reasons unrelated to what the code does.
  - The check is an invariance test, which changes a label-irrelevant input such as a name and expects the prediction to stay the same (Ribeiro et al., 2020).
  - The *in-support share* is the share of a repository's label weight that lies in its owner's patented technologies.
  - The check uses a stratified random sample of 7,600 repositories, drawn with a fixed seed and fixed before the check, so that each repository's chance of selection is known. On this sample, masking the owner's name moves the in-support share by 0.0011 (95 percent interval −0.002 to 0.0042).
  - Before the check, a mean shift of at least 0.02 in the in-support share, with an interval that excludes zero and lies beyond run-to-run noise, was declared material. The shift lies below that threshold, and so does the upper bound of a pure re-run, 0.0041.
  - The check finds the labels clean of the owner's name at this resolution.
  - Masking owner names inside the repository text as well moves the share by 0.0037.
  - The interval of that shift alone excludes zero. The shift stays an order of magnitude below the threshold, and I read it as a small association that comes with the run-to-run change in the labels.
- The check leaves training exposure open.
  - Text posted after the model's training would address it (Gilardi et al., 2023), as would a split of agreement by whether a repository was created before or after the training cutoff. That split remains to be run.
  - The provider states no training cutoff, so any such split would rest on an assumed date.
  - The check and the reproducibility runs call the labeler under its second provider name, `z-ai/glm-5.3-flash` (Appendix B.7).
- A set of perturbations declared in advance tests how the labels depend on the prompt.
  - The *menu* is the list of CPC titles in the prompt. Five prompt and menu variants serve as invariance tests of the same kind (Ribeiro et al., 2020) and as the sensitivity analysis to prompt design that Carlson and Burbano (2026) propose.
  - Across the variants, the in-support share moves by at most 0.0071 on the labeler and 0.0076 on the second labeler, with every interval covering zero.
  - The in-support share thus holds under changes to the prompt's wording and the menu's order, while label sets change.
  - Dropping the menu's titles churns 63.5% of label sets.

  - On identical prompts, the second labeler sits 0.0577 below the production labels in in-support share and reproduces 16.4% of their label sets, against 60% for a second run of the labeler.
  - The battery measures how the labels depend on the formulation, and the references above bear on agreement rather than accuracy.

B.3 Reproducibility

- In the same run, re-running the identical prompts reproduces 60% of production label sets exactly.
  - About 40% of label sets change under every perturbation, including a pure re-run and a mask that removes no owner identity.
- Label sets churn, and a repository's in-support share holds.
  - Under every perturbation, comparing each repository's original and perturbed labels, the in-support share stays exactly the same for at least nine repositories in ten.
  - Owners' patent technologies are broad, and the churn stays largely inside them.
  - A re-labeling gives a new set of labels rather than a reproduction. The inside indicator of section 4 uses the same kind of share, measured against the firm's patented main groups of five grant years rather than the owner's patent technologies, and section B.1 reports how often a second run moves a repository across its threshold.
- The labels come from one retrospective classification pass.
  - The pass reads current repository text and applies it to the whole event history.
  - The activity, its events and each repository's entry carry dates.
  - What a repository's code is about carries the date of the pass alone, though code changes over a repository's life.

B.4 What the labeler reads

- In the first stage, I offer 206 CPC subclasses.
  - These are the subclasses that are most often a patent's primary subclass, plus 7 software subclasses this ranking would exclude.
  - The CPC's other subclasses are not offered, so no project receives them.
  - The model returns at most 3 subclasses, ranked by relevance.
  - In the second stage, I offer the main groups under the chosen subclasses, and the model returns at most 3, again ranked by relevance.
  - A returned code can name a subgroup within its main group.

  - The main estimates read every code at its main group, and one check reads the subgroups (section 4.1).
  - The prompt asks the model to classify the project by its function rather than its programming language, and either stage may return no applicable class, which leaves the project unlabeled.
  - Figure 2 draws the steps.
  - I run the model at temperature zero with low reasoning effort, using the CPC titles from August 2026.

- The pipeline caps a repository's cleaned README at 12,000 characters, and the model reads its first 6,000.
  - The model's cap truncates the README of 20.4% of labeled repositories.
  - The median truncated README keeps 60.9% of its text.
  - For 7.3% of labeled repositories, the README reaches the pipeline's cap.
  - The cap's effect on the labels remains open.
  - A classification of long READMEs in full, by word frequencies or embeddings, would show it.

B.5 Methods for labeling open-source code

- The method has antecedents in studies that place documents in patent classes and that build measures with LLMs.
  - Studies place other documents in patent classes, among them research papers (Nanba and Takezawa, 2009), non-patent literature (An et al., 2026) and standards (Baron and Pohlmann, 2018).
  - Closest to the method, Ganglmair and Kann (2025a) also compare a firm's research disclosures with its own patents by relevance.
  - Studies evaluate LLMs' ability to assign patents to CPC classes (Lee and Hsiang, 2020; Emer et al., 2026).
  - Hansen et al. (2023) classify job postings with an LLM tuned on human labels and checked against them.

- Researchers use text to relate patents to one another (Arts et al., 2018, Kelly et al., 2021), to occupations (Autor et al., 2024) and to job tasks (Webb, 2020).
  - Argente et al. (2025) match firms' patents by text to their own product introductions.
  - Studies classify open-source projects by industry (Xu and Quispe, 2026), topic (Emery et al., 2025, Balla et al., 2026), use case (Conti et al., 2025) or the libraries they use (Brea, 2024).

  - Economists build measures from text (Gentzkow et al., 2019), and deep learning turns text into structured data at scale (Dell, 2025).
  - Estimates built on such a measure are valid only when the measure is combined with a small validation sample (Ludwig et al., 2026).

- Table 14 compares the methods that could place an open-source project in the patent classification.
  - A prompted LLM reads any project's text at low cost, so it labels the projects.
  - The EPO's categorizer, built for patent prose, serves as the check.

Table 14: Methods for assigning patent classes to an open-source project

Method	Inputs and preparation	Coverage of varied projects	Cost at this scale
Keyword rules	A list of words for each of the 9,867 main groups	Misses projects that describe the same technology in other words	Experts writing and testing every list
Hand coding	Coders who know the patent classification	Reads any project	An expert reading each of the 209,352 projects
A classifier trained on hand labels	A set of projects already labeled in the patent classification, which would have to be built first	Covers only what its training set covers	The hand labels, before any training
Transfer from patents (Ganglmair and Kann, 2025b)	Patent text to learn from, and a patent-style abstract written for each text to be classified	Assigns subclasses, coarser than the main groups used here	Unstated in the source
Software categories (Brea, 2024; Conti et al., 2025)	A list of software categories	Coarse software categories, with no patent class, and Conti et al. (2025) leave about 18 percent of all repositories out of their categories, finding no coherent category for them	Low
The EPO's text categorizer	Nothing from the user. It learns from examiners' classifications	Built for patent prose, and reads at most 5,000 characters of a text	About 21 days for these data, behind captchas
A prompted LLM (this paper)	The titles of the patent classification, given in the prompt	Reads any project's name, description, topics and README	About an hour, at a low cost at list prices

Notes: Each row describes a method for assigning classes from the Cooperative Patent Classification to an open-source project. The EPO's time reflects throughput in my runs. The LLM's cost is an estimate at list prices for the corpus of September 2026, not a billed amount. This paper uses the classification at the level of the main group (section 3.2).

B.6 The second label check: DeepSeek-V4-Flash on patents

- On patents, another LLM, DeepSeek-V4-Flash, applies the same two stages to titles and abstracts.
  - The model receives 2,802 patents and labels 2,774 of them.
  - The examiners' classifications are the reference.

- Table 15 compares the two at the main group and at the subclass.
- Scoring a classifier against the examiners' codes follows Lee and Hsiang (2020).
- Each comparison sits beside the model's label sets permuted across the patents within the CPC section of the examiners' primary code.
- The permutation keeps both sets of codes and breaks their pairing, so the share at random plays the role of the agreement expected by chance in Cohen (1960), drawn within strata of the classification as Jaffe et al. (1993) draw control patents from the same class.
- The section is the broadest stratum, so the gap between the observed share and the share at random can overstate the agreement beyond chance (Thompson and Fox-Kean, 2005).
- Each patent is weighted by the inverse of its probability of entering the sample.
- The check measures agreement with the examiners on patents. It tests the two-stage prompt with another model, and leaves open how the labeler reads code.

Table 15: The second label check: DeepSeek-V4-Flash's labels for patents against the examiners'

	Main group		Subclass	
Agreement	Observed	At random [band]	Observed	At random [band]
A common group	89.1%	20.8% [18.0%, 23.6%]	95.4%	51.3% [48.7%, 54.0%]
The same first code	47.8%	5.5% [4.0%, 7.3%]	70.4%	29.1% [26.1%, 31.6%]
The same set	9.6%	0.5% [0.1%, 1.0%]	39.8%	10.9% [8.6%, 13.0%]
Mean Jaccard overlap	0.39	0.06 [0.05, 0.07]	0.64	0.27 [0.25, 0.29]
Cohen's κ, first codes	0.46	0.03 [0.01, 0.05]	0.63	0.15 [0.12, 0.18]

Notes: The labels are DeepSeek-V4-Flash's for 2,774 patents, and the reference is the examiners' classification of the same patents. At random, the model's label sets are permuted across the patents within the CPC section of the examiners' primary code. Each At-random column gives the draws' mean, with their 2.5th and 97.5th percentiles in brackets. Cohen's κ compares each patent's first code. The model lists at most three main groups, which bounds agreement on the same set.

B.7 The sample, the labeler and the labels' limits

- *The sample* is large firms, the OSS-active firms of section 3.4, described here on their firm-years that report in US dollars.
  - A firm is the company at the top of the ownership tree, to which its subsidiaries' patents and repositories are assigned.
  - In the same years, the median firm-year of the sample's firms is larger in total assets than 79.4% of the same year's Compustat North America firms.
  - Among the sample's firms' firm-years, 82.9% report research and development, against 25.9% of the universe.
  - Business services alone hold 44.9% of the sample's firms' firm-years that carry a sector code.

  - Of these firms, 76.1% are incorporated in the United States, and the rest are foreign firms carried in Compustat North America.

- The findings speak to large, listed firms and to the organizations that link to a firm.
  - Firms outside Compustat, code published by individuals, and code that a firm's own employees release under personal accounts lie outside the sample, much as anonymous research disclosures carry no firm (Ganglmair and Kann, 2025a).
  - Few listed patenting firms begin to publish code. Of the 4,361 patenting firms with no linked code in the first year observed, 338 (7.8%) begin to publish linked code in 2016–2024.
  - The sample's firms are the cathedrals, the firms in which both patents and code can be seen at scale, technology by technology.
  - The sample's firms hold 89.6% of all GitHub activity on the repositories the account crosswalk links to any firm.

- In the patent crosswalk, 1,948,899 granted patents of 2015–2024 are attributed to 13,651 firms.
  - Repositories are attributed to 2,755 firms.
  - Activity the link misses lies outside the measure.

- *The public code* comes from GitHub.
  - A harvest of 160,063 numeric GitHub repository identities under the accepted accounts resolves, at the 2024 firm, to 130,590 repositories. The second route of Appendix A.4 adds 78,762, for 209,352 attributed repositories in all.
  - These are the public repositories under the GitHub accounts that a fixed rule links to a firm, so the measure covers what the firm releases as a firm.
  - By GitHub's license field, read in spring 2026, 58.2% of the harvested repositories carry an open-source license, 27.1% carry no detected license, and 14.2% carry a proprietary or custom one.
  - The more active repositories carry an open-source license more often (70% in the most active third, 45.4% in the least).

- Four kinds of a firm's open-source work fall outside the measure.
  - The first is projects hosted by foundations.
  - The second is repositories transferred to a foundation within the window, which leave the firm's accounts with their whole history.
  - The third is code that comes with an acquisition, which shifts who contributes to the acquired projects (Chen et al., 2022).

  - The fourth is contributions to repositories the firm does not own.

- A repository's activity is its dated events in GH Archive, which the held files cover from 2015, and the events date its entry and its activity.

- Beyond these exclusions, repositories can move to another account or vanish from GitHub.

  - In a stratified sample of 130 labeled repositories, all 9 that moved to another account moved after their last recorded event under the firm.

  - Weighted to the sample's frame, the labeled repositories that vanished held 0.02% of the labeled repositories' events.

- *The labeler* is the model glm-5.3-flash.

  - The production labels carry two provider names of that one model family, `stealth/ox-alpha` and `z-ai/glm-5.3-flash`.

  - OpenRouter, a service that routes requests to model providers, serves the model at temperature 0, with at most 900 output tokens and low reasoning effort, on the CPC title list of August 2026.

  - The masking check and the reproducibility runs use the second name (Appendix B.3).

- *The labels' limits* start from the validation, which rests on automated references and a one-coder pilot.

  - The EPO's categorizer is compared with the labels on 500 repositories and on the sample of 800, where the production labels share a main group with it on 76.4% of the items both labeled.

  - The 50-item pilot that I coded blind gives one coder's agreement.

  - Masking checks whether the owner's name pulls the labels toward the owner's patented technologies, and training exposure stays open (Appendix B.2).

  - Label sets change between runs, while a repository's in-support share stays stable (Appendix B.3).

- The statements on where a project lies rest on one assumption, that label errors are unrelated to the channel, code or patent, and to the firm's patent technologies.

  - Should it fail, a human-coded probability sample of the labels would let the downstream statistics be corrected as well as scored (Wang et al., 2020; Egami et al., 2023; Angelopoulos et al., 2023).

  - Treating machine-generated variables as data biases estimates and invalidates inference (Battaglia et al., 2025).

  - Corrections exist for designs, like this one, that aggregate predicted labels into group-level variables (Qiao and Huang, 2025).
  - Each of these corrections needs a gold sample. The pilot has one coder and an automated reference, so it is not one, and the paper's estimates carry no correction for label error.

B.8 One repository's labels

- The model reads a repository's name, description, topics, programming language and README, with long READMEs cut (Appendix B.4).
  - It answers in two steps, first up to three CPC subclasses, then the main groups inside them.
  - It judges the repository by what it does rather than what it is written in.
- For Microsoft's code editor, `microsoft/vscode`, the model returns two main groups, `G06F 8/00` (arrangements for software engineering) and `G06F 9/00` (arrangements for program control).
  - Given the same repository, the EPO's text categorizer returns `G06F 8/00`.
  - Each returned main group carries an equal share of the repository's label weight, here a half each.
- Appendix A.9 follows one repository, facebookresearch/fastMRI, from its link through its samples and labels to the inside rule.

C Participation in full

- This appendix gives the checks behind section 4: the construction of inside, the estimates without forks or with other labels, the profile by arrival quarter, who arrives, the other outcomes, and the perspectives' mapped contrasts.
  - Every estimate is $100(e^{\beta} - 1)$, the percent difference in the expected count (section 4.1), in the form of one of the columns of Table 4, with standard errors clustered by firm and Holm's adjustment within each declared family. An estimate that passes Holm's adjustment, with an adjusted p-value below five percent, is significant in the sense of section 4.2, and the tables mark the adjusted p-values with stars.
  - Each reading states its limits beside the estimate, and every reading stays within association.

C.1 Estimation details

- Section 4.1 estimates β by Poisson pseudo-maximum likelihood. The estimator rests on two results, from Santos Silva and Tenreyro (2006) and Wooldridge (1999).
  - *A moment condition.* Write equation (1) of section 4.1 as $E[y_r \mid w_r] = \exp(w_r' b)$, where w_r collects the inside indicator, the effects and the controls, and b their coefficients. The estimator solves

$$\sum_r \left[y_r - \exp(w_r' \hat{b}) \right] w_r = 0, \tag{2}$$

    the first-order condition of the Poisson likelihood (Santos Silva and Tenreyro, 2006). It is the sample counterpart of $E\{[y_r - \exp(w_r' b)]\, w_r\} = 0$, which holds whenever the conditional mean is right. So the counts need not be Poisson, or even integers, for the estimator to be consistent (Santos Silva and Tenreyro, 2006), and a zero count enters equation (2) like any other (Santos Silva and Tenreyro, 2006).
  - *The firm effect.* Let $n_p = \sum_{r \in p} y_r$ be the firm's total over its repositories, and $\mu_r = \exp(\beta \,\mathrm{Inside}(r) + \gamma_{y_0} + X_r' \delta)$, in the form of columns 1 to 3. In column 4's form, equation (1), the same steps hold with the firm-by-year effect α_{py_0} in place of α_p, the total taken over the firm's repositories of one year, and $\lambda_{g(r)y_0}$ in place of γ_{y_0} in μ_r. If the counts were Poisson with a firm effect and independent given it, the counts given that total would be multinomial, with probabilities

$$\pi_r = \frac{\mu_r}{\sum_{s \in p} \mu_s}, \tag{3}$$

    from which α_p cancels (Hausman et al., 1984; Wooldridge, 1999). The fixed-effects Poisson estimator maximizes $\sum_p \sum_{r \in p} y_r \log \pi_r$ (Wooldridge, 1999).

- Its score, $\sum_{r \in p}(\nabla \pi_r / \pi_r)(y_r - \pi_r n_p)$, has mean zero when only the conditional mean holds in the form $E[y_r \mid \alpha_p, \text{the regressors of all the firm's repositories}] = e^{\alpha_p}\mu_r$, since then $E[y_r \mid \cdot] = \pi_r E[n_p \mid \cdot]$ (Wooldridge, 1999). As the number of firms grows, with firms independent of each other (Wooldridge, 1999), the estimator is therefore consistent without the Poisson distribution and without any assumption on how the counts of one firm's repositories depend on each other (Wooldridge, 1999). That dependence within a firm is what the clustering by firm allows for. A firm with $n_p = 0$ adds nothing to the likelihood and drops out.
- In columns 1–3, where the firm is the one set of many-level effects beside a few cohort effects, estimating the firm effects as parameters, as Correia et al. (2020) do, brings no incidental-parameter problem (Wooldridge, 2023). Columns 4 and 5 also estimate the main-group-by-cohort effects as parameters.
- *Column 4.* Conditioning on each firm-and-cohort total, in place of n_p in equation (3), removes the firm-by-cohort effect, and the main-group-by-cohort effects enter μ_r and are estimated beside it.
- *Column 6.* The same conditioning removes the firm-by-cohort effect, and no main-group-by-cohort effects enter μ_r.
- The method sources describe this estimator as a natural way to deal with zeros (Santos Silva and Tenreyro, 2006; Correia et al., 2020). When the effect changes whether the outcome is zero, the effect on $\log(1 + y)$ can be made any size by rescaling the units of y (Chen and Roth, 2024), and its estimate can have the wrong sign in expectation (Cohn et al., 2022). The conditional fixed-effects negative binomial does not control for all stable covariates (Allison and Waterman, 2002), and with group dummies it faces an incidental-parameter problem (Cohn et al., 2022).

- Applied studies of open-source and GitHub activity with count outcomes mostly estimate OLS on $\log(1 + y)$ or on its inverse hyperbolic sine (Wright et al., 2023; Lin and Rai, 2024; Cao et al., 2026), and some of them report Poisson estimates as a check (Cao et al., 2026).
  - Conti et al. (2023) use Poisson regression as their main estimator.
  - None of these studies reports an adjustment for multiple outcomes.
  - Reporting $100(e^{\beta} - 1)$ follows Santos Silva and Tenreyro (2006) and Chen and Roth (2024), and the difference is relative to the expected count of the other repositories (Benatia et al., 2025).

C.2 Construction of inside

- Inside can be drawn with a lower or higher cut on the share of label weight in the firm's patented main groups, and at a coarser or finer level of the CPC.

  - A lower cut, or a coarser level, can only add repositories, so each definition's inside repositories contain those of the next stricter one.
  - At the subgroup, no repository moves from outside to inside, and 8,862 move from inside to outside, 8.6 percent of the reference set.
  - With the share itself in place of the indicator, the coefficient for new participants in column 4 is positive, with an interval of 21.0 to 70.1 percent for a move from no weight in the firm's patented main groups to all of it.
  - With subgroup-by-cohort effects in place of the main-group effects, on repositories whose primary main group carries a single subgroup cell, the inside difference for new participants is 34.1 percent, which passes Holm's adjustment.
  - Comparing within subclass and cohort, while keeping inside at the main group, gives 34.8 percent more new participants, which also passes it.
  - The levels stand side by side, without a declared test of their difference.

- Table 17 varies the cut on the share s of label weight in the firm's patented main groups and the CPC level at which technologies are cut.
  - Appendix A.7 defines s and each cut.
  - Moving the cut follows Kerr (2010) and Bandiera et al. (2021).
  - Reporting the share itself beside the indicator also follows Bandiera et al. (2021).
  - Redrawing inside at other levels of the classification follows Thompson and Fox-Kean (2005) and Bloom et al. (2013).
  - Every construction gives a positive inside difference in every column.
  - At $s \geq 1/2$ and $s \geq 2/3$, and for the share itself, new participants and contributions pass Holm's adjustment in every column, and at $s > 0$ they pass it in every column except contributions in column 2, which pass it only at the ten-percent level.
  - At $s = 1$, the column-4 inside difference on new participants is 16.4 percent and passes Holm's adjustment narrowly, as section 4.5 qualifies.
  - Raising the cut from two thirds to all of the weight moves 1,552 reference repositories from inside to the other repositories (Panel A).
  - Their raw mean of new participants is 19.67, against 14.94 for the repositories with all of their label weight in the firm's patented main groups, and no other range of the share has a higher raw mean.

- At the CPC subclass, 92,365 repositories are inside, so inside says little more than that the firm patents somewhere in the repository's broad area.
  - The subclass estimate misses Holm's adjustment in columns 1, 2 and 4.

  - For new participants and contributions, its point estimates are larger than the main group's in column 4 and smaller in columns 1 and 2.

- At the CPC subgroup, one level below the main group, a repository's labels and the firm's patents are cut one level finer.
  - Repositories move only from inside to outside, 8,862 of the reference set.
  - Under the primary rule, a label coded only to its main group counts as outside the firm's patented main groups.
  - An alternative rule counts such a label as in the firm's patented main groups when the firm holds patents coded only to the same main group, or to a code of that main group that the subgroup taxonomy does not map. Under it, 7,449 repositories move.
  - In column 4's form, contributions are 21.6 percent higher under the primary rule and 38.2 percent under the alternative rule, against 45.5 percent in Table 4. The rule for these labels thus moves that estimate.
  - Among the subgroup constructions, stars pass Holm's adjustment in one, with subgroup-by-cohort effects.

Reading column 4.

- Inside varies within a main group and year for two reasons.
  - One firm patents the main group and another does not, so column 4's comparison is partly across firms.
  - A repository's other labels can also make it inside or outside within the same firm, primary main group and year, since inside counts all its labels and the main-group effect only the top-ranked one.
  - Inside status and whether the firm patented the primary main group disagree for 5,237 of the 100,111 repositories with a primary main group (5.23%), the 4,969 inside repositories whose primary main group the firm has not patented and the 268 outside repositories whose primary main group it has.

- Column 4 summarizes the inside difference with one coefficient. The negative-weight results below concern its linear analogue and do not show how the Poisson coefficient weights differences across firms and main groups.
  - In a linear regression with two sets of effects, the coefficient is a weighted sum of the underlying differences, with weights that can be negative, so it can be negative when the difference is positive in every firm and main group (de Chaisemartin and D'Haultfœuille, 2020; Goldsmith-Pinkham et al., 2024).

  - A weight turns negative where the fitted value of inside, from a regression on the two sets of effects and the controls, exceeds one.
  - The sources prove this for linear regression, so its use for the Poisson estimator of the main estimate rests on analogy.
  - This qualification was written after the results were seen.

- If a repository's new participants are the users who leave contribution-family work on it, the inside difference combines two ratios that the data cannot separate.
  - The expected count of an inside repository, relative to that of another, is the ratio of their users times the ratio of the shares of those users who leave work on them.
  - Inside code may have more users.
  - A larger share of an inside repository's users may leave work on it.
  - Firm-by-year and main-group-by-year effects remove only the parts of either ratio that are a level for each firm and year or for each main group and year.
  - The expected count identifies only the product of the two ratios, so the data cannot separate them, even in sign.
  - Use is assumed here, as the setting in which the counts are read, and the estimates count the people who leave work on the repositories and their contributions.

- The raw ratio of the means in Table 3 is smaller than the difference with firm and year effects alone.
  - The raw means pool firms, and firms differ in how many of their repositories lie inside and in how many new participants their repositories draw.
  - The firm effects remove those differences, so the comparison within the firm can be larger or smaller than the raw one.

- The founding controls can absorb part of the inside difference itself.
  - The first-quarter counts are measured in q_0, before any outcome is counted, but inside status may itself raise first-quarter activity, so controls measured at the first quarter can absorb part of the difference.
  - The length of the text comes from the text the labeler read, as it stood in 2026, after every outcome window.
  - Where either moves with inside status, the controls absorb part of the inside difference, so the estimates with them are not a purer version of those without, and the fall in the estimate when they are added describes how much of the difference goes with these controls.

  - In the two-ratio reading above, the estimates without the founding controls, columns 1 and 3, carry both ratios together, the whole association between inside status and the counts that the firm and year effects leave.
  - The first-quarter counts (section 3.4), stars and forks among them, stand in part for a repository's early use, as Medappa et al. (2025) control for a project's stars to account for its popularity.
  - The founding controls capture part of a repository's early activity, and the data cannot separate the number of users from the share of them who leave work on the repository.

What the effects leave of inside.

- The variation in inside that each set of effects leaves, in a linear regression of inside on those effects, shows what the comparison rests on, as Mummolo and Peterson (2018) recommend reporting.
  - On column 4's estimation sample, inside has a standard deviation of 0.395.
  - Firm-by-year effects leave 32.3% of its variance, main-group-by-year effects 88.1%, both together with the programming-language effects and the controls 28.9%, and firm-by-year with subclass-by-cohort effects 30.8%.
  - The main-group effects therefore absorb little of inside, and the firm-by-year effects do most of the absorbing.
  - Inside is constant within the firm-by-year cells of 24.0% of the repositories and within the main-group-by-year cells of 1.4%.
- The weights of de Chaisemartin and D'Haultfœuille (2020), computed on the linear analogue of the estimate for new participants, show how much of that regression rests on negative weights.
  - Negative weights fall on 34.7% of the inside repositories in column 4's form and 18.4% in column 6's, and they sum to -0.206 and -0.025.
  - The ratio of the coefficient's size to the weights' standard deviation is 0.757 in column 4's form and 1.083 in column 6's.
  - de Chaisemartin and D'Haultfœuille (2020) recommend another estimator when many weights are negative and that ratio is not large.
  - Column 4's linear analogue carries more negative weight than column 6's, and the paper reports the two side by side.
  - These figures and the variance shares above are diagnostics of the linear analogue, not a decomposition of the Poisson coefficient.

Inside without the share.

- Two designs draw inside without a cut on the share, on the repositories of column 4's sample.
  - The first reads inside from the repository's first label alone, with one row per repository.
  - The second has one row per repository and label, each weighted by one over the number of labels, and marks every row of a repository with any label in the firm's patented main groups.
  - Without main-group or label-count effects, the first gives 26.7 percent more new participants and 26.9 percent more contributions, and the second 35.3 and 53.8 percent.
  - With both sets of effects, the first gives 22.0 and 49.1 percent, and the second 26.6 and 45.2 percent.
  - Every one of these estimates has an adjusted p-value below one percent, so neither design is taken to zero.
  - The estimates are similar with and without main-group effects, except the first label on contributions, which goes from 26.9 and 30.3 percent without them to 47.3 and 49.1 percent with them, each pair without and with label-count effects.
  - The intervals overlap, and no test compares the forms.

A check with repository size.

- As a check, I add the logarithm of one plus GitHub's reported size of the repository.
  - The sizes come from fetches of July and November 2026, after every outcome window, so the data cannot tell whether participation grew a project or the project was large from the start.
  - Nothing here is an effect of size, and a fall does not show that size explains inside.
  - Of the 100,111 repositories of column 4's sample before the estimator's drops, 240 have no size.
  - The estimator's drops then leave 96,612 repositories of 626 firms for both sets of estimates, and 172,862 rows of 653 firms for any label.
  - Holm's adjustment here runs over this check's own families (Table 16).
  - Table 16 gives column 4, the first label, any label and the count of labels inside, each without and with the size term.
  - Column 4's estimates stay positive and similar, and so do those at two thirds of the weight, where new participants go from 30.5 to 27.5 percent and contributions from 55.2 to 48.2 percent.
  - At all of the weight they go from 16.1 to 16.6 percent, with an adjusted p-value of 0.0705, which is not significant, and from 19.6 to 20.5 percent, with 0.0495.

– Any weight, the first label and any label fall toward zero without reaching it, any weight from 30.9 to 23.0 percent on new participants and from 66.3 to 54.9 percent on contributions, and the first label from 21.3 to 11.3 percent on new participants.

– On new participants, the first label and any label are no longer separated from zero after Holm's adjustment, and on contributions they stay separated.

– For those three, the fall is compatible with inside standing in part for size, which this check cannot show, and why they differ from column 4 is not tested.

Table 16: Alternative measures of inside, without and with log size (percent)

			New participants		Contributions	
	Repositories	Firms	Without size	With size	Without size	With size
Panel A. The inside difference						
Column 4 of Table 4	96,612	626	28.4***	26.3***	44.8***	48.5***
			(7.9)	(7.5)	(12.8)	(13.4)
Alternative measures of inside						
First label only	96,612	626	21.3***	11.3*	48.7***	32.0**
			(7.3)	(6.3)	(12.2)	(11.7)
Any label inside (rows)	172,862	653	25.0**	17.7*	44.0***	35.9**
			(10.7)	(9.0)	(13.2)	(13.1)
Labels inside, against none						
One	96,612	626	27.5***	22.3**	53.5***	45.6***
			(10.2)	(9.2)	(13.2)	(14.5)
Two	96,612	626	34.3**	28.2**	88.9***	82.2***
			(14.3)	(14.2)	(22.9)	(22.8)
Three	96,612	626	42.9**	43.4**	74.5**	83.9***
			(21.4)	(22.3)	(35.1)	(34.7)
Panel B. New participants' share of contributions, on repositories with a contribution						
			Inside difference (percentage points)			
Column 4's form	57,181	570	1.02 (0.78)			

Notes: Panel A's entries are $100(e^{\beta} - 1)$, with standard errors clustered by firm, on the same scale, in parentheses, for new participants and for their contributions. The first-label and any-label rows are separate regressions with main-group-by-year and label-count effects, and the rows for one, two and three labels inside come from one model in column 4's form with label-count effects, against repositories with none of their labels inside. Samples differ, so each row gives its repositories and firms, and the any-label row counts one row per repository and label. Each row's estimates without and with size use the same repositories, those with a size, so the estimates without size differ slightly from column 4 of Table 4. Size is the logarithm of one plus GitHub's reported size, measured in July and November 2026, after the outcome window. The data cannot tell whether participation grew a project or the project was large from the start, so the columns with size are a check and no entry is an effect of size. Panel B is a linear estimate in column 4's form, with no size term, on the repositories with a contribution, and inside repositories are 2.53 points more likely to have one. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method within its family. In Panel A, the first three rows and the cuts of inside in section 4.5 form one family without size and one with size, and the rows of labels inside form two more. Panel B's estimate has a family of its own.

C.3 Robustness checks

- Two checks re-estimate columns 1 and 4 of Table 4, and Table 18 gives both.
  - The first drops forks, and the second recomputes the inside indicator from other labels of the same repositories.

Table 17: The inside difference in participation under each construction of inside (percent)

		Column 1		Column 2		Column 4	
Construction	Inside	New part.	Contrib.	New part.	Contrib.	New part.	Contrib.
Panel A. Main groups, by the cut on the share s of label weight in the firm's patented main groups							
Any weight, $s > 0$	83,001	95.2*** (32.0)	110.7*** (39.0)	28.2** (10.9)	30.7* (15.1)	32.0*** (10.2)	67.2*** (15.0)
At least one half, $s \geq 1/2$, as in Table 4	82,046	77.3*** (25.7)	84.5*** (27.6)	34.3*** (11.1)	28.1** (12.8)	29.5*** (8.1)	45.5*** (12.9)
At least two thirds, $s \geq 2/3$	72,041	77.1*** (18.7)	102.4*** (29.0)	37.8*** (8.3)	44.9*** (12.8)	30.9*** (6.8)	55.5*** (15.7)
All of it, $s = 1$	70,489	53.3*** (16.2)	47.6*** (15.2)	29.4*** (7.8)	24.1** (9.5)	16.4** (6.7)	19.9** (8.4)
The share s, per move from 0 to 1		107.4*** (33.7)	118.2*** (38.2)	46.9*** (13.5)	45.3*** (17.6)	43.5*** (12.5)	75.4*** (19.3)
Panel B. Other CPC levels							
CPC subclass, $s \geq 1/2$	92,365	60.3 (33.9)	57.5 (39.9)	21.5 (14.1)	7.5 (15.3)	43.3 (25.0)	53.9 (29.5)
CPC subgroup, $s \geq 1/2$	73,184	67.1*** (19.3)	69.5*** (19.7)			27.8*** (7.0)	21.6* (11.0)
CPC subgroup, alternative rule for main-group labels	74,597	88.8*** (25.2)	93.8*** (25.4)			34.9*** (8.8)	38.2*** (10.1)
Panel C. Column 4's form with a finer or coarser technology cell							
Subclass-by-cohort effects, inside at the main group	82,046					34.8*** (9.5)	49.6*** (14.3)
Subgroup-by-cohort effects, inside at the subgroup	73,184					34.1*** (9.8)	28.2** (14.5)

Notes: Entries are $100(e^{\beta} - 1)$ for the inside indicator in the forms of columns 1, 2 and 4 of Table 4, with standard errors clustered by firm in parentheses. Holm's adjustment runs over the five outcomes of each column. "Inside" counts the inside repositories among the 102,987 of the reference set. The share's row is the coefficient on s itself and is read for its sign only. At the subgroup, repositories move only from inside to outside, and the column-2 form was not run. The alternative rule counts a label coded only to its main group as in the firm's patented main groups when the firm holds patents coded only to that main group, or to a code the subgroup taxonomy does not map. The subgroup-by-cohort row uses the repositories whose primary main group carries one subgroup cell. The rows stand side by side, without a declared test of their difference. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

  - Without forks, the estimates of Table 4 keep their size and still pass Holm's adjustment. On the second labeler's rows, the column-4 comparison is weaker under both labelings.

Without forks.

- Of the labeled repositories with a single firm, 6,600 are forks that GitHub flags. All but 2 of them are in the added organizations (Appendix A.1). By the inside indicator, 4,411 of them are inside and 2,189 outside.
  - The original links already exclude forks, so dropping them applies the same exclusion to the added organizations.
  - Since a fork can operate apart from its original and is not the same unit (Kalliamvakou et al., 2016), studies of GitHub exclude forks (Medappa et al., 2025) or drop them as a check (Abou El-Komboz and Goldbeck, 2025).
- With the forks dropped, every estimate that passes Holm's adjustment in columns 1 and 4 still passes it, and stars pass it in neither column, as before.
  - Across the five outcomes, three of them outside the table, dropping forks moves column 1's estimate most for stars by new users, from 44.0 to 40.9 percent.
  - In column 4 it moves most for participants new to the firm active at another linked firm before arriving, from 21.9 to 21.4 percent, and by as much in the other direction for contributions, from 45.5 to 46.0 percent.

Inside from other labels.

- Four other labelings of the same repositories let me recompute s and the inside indicator by the rule above.
- Two labelings come from the paper's labeling model.
  - A second run of its second stage changes the indicator for 3.9 percent of the repositories it covers, about evenly in each direction.
  - A fresh run of both stages on text fetched in October 2026 changes it for 3.8 percent, 2.3 toward outside and 1.5 toward inside.
- Two labelings come from other LLMs, and these models place fewer labels in the firm's patented main groups, so their changes run mostly toward outside.
  - The second labeler's labels, from the second stage, change the indicator for 13.2 percent of its repositories, 11.8 percent toward outside and 1.4 percent toward inside.
  - DeepSeek-V4-Flash, run on both stages, changes it for 10.8 percent, 7.0 percent toward outside and 3.9 toward inside.

- Only the second labeler covers enough repositories to estimate with any precision, 57,004 in column 1 and 52,892 in column 4, all of them in the original links.
  - On those rows, each column is estimated twice, once with the paper's labels defining inside and once with the second labeler's.
- In the form of column 1, the estimates under the two labelings are close, and three of the four pass Holm's adjustment, while the paper's labels for new participants pass it only at the ten-percent level.
  - They are 49.8 and 51.9 percent more new participants, and 57.4 and 50.4 percent more contributions.
- In the form of column 4 on the second labeler's rows, every estimate lies below the full sample's 29.5 and 45.5 percent.
  - For new participants, the estimate is 16.3 percent under the paper's labels and 16.9 under the second labeler's. The first misses Holm's adjustment, and the second passes it.
  - For contributions, only the paper's labels pass, at 32.0 percent, against 18.4 under the second labeler's.
  - The within-technology comparison is weak on these rows under both labelings.
- On the three smaller labelings, with 24 to 286 firms as clusters, 3 of the 24 estimates for new participants and contributions pass Holm's adjustment, all three in column 4.
  - On the second run's rows, contributions pass under both labelings.
  - On the fresh run's rows, contributions pass under the paper's labels with a negative estimate, −60.4 percent.
- The check has stated limits.
  - The two second-stage labelings reuse the paper's first-stage classes, so a first-stage error that they share with it stays hidden.
  - Every labeling reads text captured in 2026.
  - Only the paper's labels cover the 43,198 labeled single-firm repositories of the added organizations.
  - Column 4's technology cells keep the paper's first main group.
  - Each labeling's rows form its own sample, so the estimates describe those rows.
  - The paper's labels and each other labeling stand side by side, and no test compares them.
  - As in Ludwig et al. (2026), who re-run one regression under several models' outputs, the comparison measures agreement between labelings, not accuracy, since no validation sample says which is right.

Table 18: Columns 1 and 4 without forks, and with inside from the second labeler's labels (percent)

	Column 1		Column 4	
Sample and indicator	New participants	Contributions	New participants	Contributions
As in Table 4	77.3***	84.5***	29.5***	45.5***
	(25.7)	(27.6)	(8.1)	(12.9)
Without forks	75.5***	82.3***	29.6***	46.0***
	(25.9)	(27.4)	(8.3)	(13.0)
Second labeler's rows, the paper's labels	49.8*	57.4**	16.3	32.0**
	(24.4)	(24.8)	(8.1)	(13.4)
Second labeler's rows, its own labels	51.9***	50.4***	16.9**	18.4
	(17.5)	(18.3)	(7.2)	(12.3)

Notes: Entries are $100(e^{\beta} - 1)$ for the inside indicator, with standard errors clustered by firm in parentheses. Holm's adjustment runs over the five outcomes of each column. The forms of columns 1 and 4 are those of Table 4. Without forks, the Poisson rows for new participants hold 96,246 repositories of 654 firms in column 1 and 90,575 of 623 in column 4. The second labeler's rows hold 57,004 repositories of 622 firms in column 1 and 52,892 of 590 in column 4, and the two rows differ only in the labels that define inside. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

The arrival profile.

- Table 19 counts new participants by the quarter after q_0 in which they first contribute, and estimates the inside difference for each quarter.
  - In column 4, the inside difference is positive in every quarter, and its point estimates are larger in later quarters.
  - Its first quarter misses Holm's adjustment in column 4 and passes it in column 1.
  - The format follows Abebe et al. (2021), who estimate an effect for each period since a start date.
  - The quarters are read one by one, without a declared test across them, and pointwise intervals support only pointwise readings (Freyaldenhoven et al., 2019).
  - In the raw means, arrivals per repository fall from the first quarter to the eighth on both sides, and by more outside.

Table 19: New participants by arrival quarter: the inside difference (percent)

	Mean arrivals per repository		Inside difference		
Quarter after q_0	Inside	Outside	Column 1	Column 4	Column 4, interval
1	1.903	1.411	44.1** (24.4)	12.6 (8.8)	[−3.4, 31.3]
2	1.794	1.269	63.5*** (23.2)	22.4*** (7.1)	[9.3, 37.0]
3	1.782	1.275	74.4*** (25.5)	27.9*** (7.5)	[14.0, 43.5]
4	1.811	1.264	80.4*** (25.9)	35.8*** (8.3)	[20.4, 53.2]
5	1.771	1.229	83.6*** (26.2)	36.7*** (9.7)	[18.9, 57.1]
6	1.773	1.173	94.4*** (28.9)	38.5*** (10.8)	[18.8, 61.5]
7	1.761	1.147	95.6*** (29.8)	37.6*** (11.2)	[17.2, 61.5]
8	1.764	1.122	98.0*** (32.6)	41.1*** (11.4)	[20.5, 65.3]

Notes: New participants counted by the quarter of their first contribution-family event on the repository. The inside difference is $100(e^{\beta} - 1)$ in the forms of columns 1 and 4 of Table 4, with standard errors clustered by firm in parentheses. Holm's adjustment runs across the eight quarters. The quarters are read one by one, without a declared test across them. The clock starts at the first observed activity, not at an exposure date. Brackets give 95 percent intervals for column 4, not adjusted for multiple testing. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

C.4 Heterogeneity across new participants

- Table 20 estimates the inside difference separately for each group of held history before the participant's first event on the repository.
  - On the full history, measured to the first event or to q_0, every group's inside difference passes Holm's adjustment, and every contrast between groups misses it.
  - On the equal 16-quarter look-back, the inside difference for people with a history at no linked firm is 5.2 percent and misses Holm's adjustment, while the inside difference for people with a history at the firm is 46.2 percent.
  - On that look-back too, no group is distinguishable from another. The difference between people with a history at no linked firm and people with a history at the firm is −28.1 percent, which misses Holm's adjustment.
  - With a full history, the earliest cohorts' look-back is the shortest, so for them the group with a history at no linked firm also holds people whose earlier events fall before 2015 and are not held.
  - The equal look-back makes the chance of missing a person's earlier activity the same in every cohort, if the time since that activity is distributed alike across cohorts.
  - It does not remove the misclassification of those people.
  - The reading is fragile for that reason.
  - Splitting participants by their history follows Chen et al. (2022) and Haese and Peukert (2026), and the equal look-back follows Deryugina and Molitor (2020) and Cunningham

and Kapacinskaite (2026).

- Table 21 estimates the inside difference separately for each group defined by what people do in the four quarters after arriving.
  - In column 4, the inside difference passes Holm's adjustment in every group.
  - In column 4, every pairwise contrast misses Holm's adjustment, and the wide intervals leave open whether the groups differ.
  - The difference between people who go on to both the firm's other repositories and another linked firm, and people who go on to another linked firm only, passes in column 2 alone.
  - The group that goes on to neither holds about half the new-participant pairs on each side, 52.12 percent inside and 51.68 percent outside (Appendix Table 10).
  - Following all arrivals to the end of the window gives the same reading.
  - Classing newcomers by what they do later follows Zhou and Mockus (2015) and Chen et al. (2022), and, taken together, the groups describe who arrives and what the arrivals then do, not what any group prefers.

Table 20: The inside difference by new participants' held history (percent)

	Full history, to first event	Full history, to q_0	16 quarters, to first event
With a history at the firm	32.8*** (6.7)	33.6*** (6.4)	46.2*** (12.2)
With a history only at another linked firm	23.7*** (8.1)	18.3*** (7.5)	27.4 (18.8)
With a history at no linked firm	31.7*** (12.0)	30.3*** (9.9)	5.2 (20.2)
At no linked firm minus at the firm	−0.9 (8.9)	−2.5 (7.5)	−28.1 (11.7)
At no linked firm minus only at another linked firm	6.5 (6.4)	10.1 (7.3)	−17.5 (7.5)
Only at another linked firm minus at the firm	−6.9 (7.3)	−11.5 (6.7)	−12.9 (11.5)

Notes: Column 4's form of Table 4, one model per group, with standard errors clustered by firm in parentheses. The groups are defined in Appendix A.5, by held events before the quarter of the participant's first held event on the repository, or before q_0. The 16-quarter look-back covers the cohorts of 2019–2022, so every participant has the same look-back. The contrasts are differences between groups' coefficients, expressed in percent. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

Table 21: The inside difference by what new participants do after arriving (percent)

	Column 1	Column 2	Column 4	Column 4, interval
Go on to neither	61.5*** (26.5)	25.8** (11.9)	23.4** (10.2)	[4.9, 45.1]
Go on to another linked firm only	57.9*** (26.3)	17.8* (10.9)	17.1** (7.8)	[2.7, 33.5]
Go on to the firm's other repositories only	83.9*** (18.2)	42.9*** (10.0)	30.8*** (7.1)	[17.7, 45.4]
Go on to both	83.9*** (28.3)	41.7*** (11.0)	22.3*** (6.3)	[10.5, 35.3]
Go on to none of the firm's other repositories	58.6*** (25.8)	22.1** (10.9)	21.5*** (9.2)	[4.8, 40.9]
Go on to no other linked firm	66.7*** (23.4)	28.9*** (10.1)	24.7*** (8.3)	[9.3, 42.2]
Go on to both minus go on to another linked firm only	16.5 (10.3)	20.3** (8.0)	4.4 (8.7)	
Go on to both minus go on to the firm's other repositories only			−6.5 (3.9)	
Go on to none of the firm's other repositories minus go on to at least one of them			−5.3 (8.4)	

Notes: New participants who arrive in the repository's first year, followed for four quarters after arrival, by their held contribution-family events in that follow-up. One model per group in the forms of Table 4, with standard errors clustered by firm in parentheses. The contrasts are differences between groups' coefficients. In the last contrast, "at least one of them" covers the people who go on to the firm's other repositories only and those who go on to both. The table shows two of the six pairwise contrasts among the four groups in column 4. The other four, in percent with unadjusted 95 percent intervals, are another linked firm only minus neither −5.1 [−15.7, 6.8], the firm's other repositories only minus neither 6.1 [−12.0, 27.8], the firm's other repositories only minus another linked firm only 11.7 [−6.6, 33.7], and both minus neither −0.9 [−18.2, 20.1]. None of the six passes Holm's adjustment at the ten-percent level. Groups are defined in Appendix A.5. Brackets give 95 percent intervals for column 4, not adjusted for multiple testing. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

C.5 Other outcomes

In every column, the inside difference in the share of new participants who contribute over the long term is not significant, and the inside difference in stars by new users passes Holm's adjustment only at the ten-percent level in columns 1, 2, 3 and 5 and is not significant in column 4 (Table 4, Panel C). Within main group and year, stars are 7.1 percent higher, and the long-term-contribution share is 0.2 percentage points higher. Stars are significant only under one of the ways of drawing inside at the subgroup (Appendix C.2). Inside repositories are more often maintained in their second year, by 2.4 percentage points within main group and year, and the difference is significant in columns 1 to 4 (Table 4, Panel C).

Mapped as in section 4.3, the perspective of Lerner and Tirole (2002) predicts a larger inside difference on issues and pull requests than on stars, since an issue or pull request carries work an audience can see and judge, and a star carries none (Kalliamvakou et al., 2015; Pinto et al., 2018; Borges and Valente, 2018). The test splits the new participants' contributions into their issues and their pull requests. With inside as in Table 4, the inside difference on issues and on pull requests exceeds that on stars by 31.6 and 36.3 percent. The pointwise interval for issues excludes zero and the one for pull requests includes it, and each misses Holm's adjustment over issues, pull requests

and commit comments, with adjusted p-values of 0.1152 and 0.1193, so they give no statistically clear support for the mapped prediction (section 4.3).

C.6 The perspectives' mapped contrasts

Table 22: The perspectives, their mappings to repositories and the contrasts that read them

Perspective	Observed proxy	Mapping assumption	Contrast and predicted sign	A reading without clear support leaves open
Less participation in code tied to patents (Lin and Rai, 2024, p. 658; Wen et al., 2013, p. 1131)	Inside	Code in the firm's patented main groups is the code tied to its patents	β, negative	A deterring part offset by other parts of the difference
Appropriability (Teece, 1986, p. 288)	Inside, and the firm's patent weight in the repository's top-ranked main group	The firm's patents in a main group mark where it holds the capabilities its code there is used with, so that code is better supported, and more so where the weight is greater	β, positive; inside × patent weight, positive	The perspective, the mapping, or the precision of a contrast that rests on few outside repositories with any weight
Code tied to products (Gambardella and Hall, 2006, pp. 882–883; Athey and Ellison, 2014, p. 312)	Two classes of a rule-based sorting of repositories by their text	Inside code is used with the firm's products, which the two classes hold	Inside × the two classes, positive	The perspective, the mapping, or the sorting
Visible contributions (Lerner and Tirole, 2002, pp. 214, 216)	Issues and pull requests, against stars	Inside code reaches an audience more interested in its technology, and an issue or pull request carries work that audience can see and judge, while a star carries none	The inside difference on issues and on pull requests minus that on stars, positive	The perspective, the mapping, or the precision of two contrasts that pass no declared test
Related experience (Athey and Ellison, 2014, pp. 297–298)	Earlier activity in the same or a related main group	Inside code is more useful to people who work in related technologies	The inside difference with related experience minus that without, positive	The perspective, the mapping, or histories that cover only the firms in the data

Notes: β is the coefficient on inside. Each contrast is estimated with firm-by-year and main-group-by-year effects and the controls, as in column 4 of Table 4 (section 4.1). The patent weight is the logarithm of one plus the firm's patent weight in the repository's top-ranked main group over the five grant years. The two classes are software development kits, interfaces and libraries, and drivers, integrations and plugins. What an issue or a pull request carries follows Kalliamvakou et al. (2015, p. 577), Dabbish et al. (2012, pp. 1279, 1281–1282) and Pinto et al. (2018, p. 3232), and what a star carries follows Pinto et al. (2018, p. 3223) and Borges and Valente (2018, pp. 7–8). Sections 3 and 4.1 define the other terms.

- Each test interacts inside with a repository or participant characteristic that a perspective's mapping to repositories (Table 22) ties to a larger inside difference, in the form of column 4, and Table 23 gives the results.
  - As in Galasso and Schankerman (2015), each prediction is read from one interaction or

contrast, and as in Anderson (2008), the predictions and the outcomes adjusted together by Holm's method were specified before the results.

  - The patent weight is zero for 19,982 of the 20,250 outside repositories covered, so the slope among outside repositories rests on the 268 with positive patent weight, each of which has three labels with only the primary one patented, and the interaction compares the inside slope with the slope among those few.

- The visible-contribution contrasts pass no declared test (Appendix C.5).

Table 23: The perspectives' mapped contrasts, read against the inside difference in participation (percent)

Perspective, as mapped in Table 22	Contrast	New participants	Contributions	Reading
Appropriability (Teece, 1986)	Inside × the firm's patent weight in the primary main group	6.5 (7.6)	14.3 (14.3)	No clear support
Code tied to products (Athey and Ellison, 2014; Gambardella and Hall, 2006)	Inside × the two classes of code tied to products	−10.1 (10.3)	−8.4 (9.8)	No clear support, point estimate opposite
Related experience (Athey and Ellison, 2014)	Inside difference with related experience minus without	−7.5 (5.1)		No clear support, point estimate opposite
Visible contributions (Lerner and Tirole, 2002)	Issues minus stars, inside as in Table 4		31.6 (17.5)	No clear support
	Pull requests minus stars, inside as in Table 4		36.3 (22.4)	

Notes: Entries are $100(e^{\beta}-1)$ for the interaction or contrast named, in the form of column 4 of Table 4 unless stated, with standard errors clustered by firm in parentheses. Holm's adjustment runs within each declared family. The patent-weight and product interactions use column 4's estimation sample for new participants, 96,854 repositories of 626 firms. The related-experience contrast stacks the counts of new participants with and without related experience, 189,138 repository-outcome rows from the same 626 firms, with every effect and control interacted with the stacked count (Albert and Caggese, 2021; Batistich and Bond, 2023). The visible-contribution contrasts use a stacked panel of 362,822 repository-outcome rows from 648 firms. The patent weight is defined as in Table 22. A contrast "minus stars" compares the inside difference on that event count with the inside difference on stars. For the visible-contribution contrasts, the declared Holm family holds three contrasts (issues, pull requests and commit comments, each minus stars). The commit-comment contrast, not shown, is 19.4 percent, with an adjusted p-value of 0.4244. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$, with each p-value adjusted by Holm's method.

D Projects cited in patent documents, in full

- This appendix gives in full the comparison of projects cited in patent documents, and the citation match behind it, from the extract to the limits.
  - It opens with the comparison, then covers the extract, the rule that reads a GitHub address, and the match to the firms' projects.
  - It then gives the known misses and false matches, a few examples and the limits.

D.1 The comparison

- US patent documents cite some of the firms' projects, some of the citing documents belong to the project's own firm, and inside repositories are cited more often than the others.
  - A project counts as cited when its GitHub address appears among a US patent document's non-patent references.
  - US patent documents cite 0.65% of the projects of the OSS-active firms, against 0.179% of other listed firms' projects with the same mix of creation year, stars and language.
  - Citations are rare in both groups, and the rest of this subsection gives the comparison in full.
  - Of the 1,748 patent documents that cite the OSS-active firms' projects, 294 belong to the firm whose project they cite.
  - The records thus tie some of a firm's code to its own patents through citations, as the labels tie it through the classification.
  - The citation match can name 78,067 of the 82,046 inside repositories of the reference set and 19,935 of the 20,941 others.
  - Among these, US patent documents cite 565 inside repositories (0.72%) and 67 others (0.34%).
  - The difference is 0.388 percentage points (95 percent interval 0.12 to 0.655), from a linear probability model with standard errors clustered by firm and no firm level held fixed.
  - Neither citation comparison enters the estimates of section 4, and the one with other firms' projects is across firms.
- The crosswalks also follow the firms' projects into US patent documents, which can list an open-source project among their references. This subsection reports how often the OSS-active firms' projects appear there, against comparable projects of other listed firms, and it uses the crosswalks and the projects rather than the labels.
  - Patent applicants and examiners list references that bear on an invention, and some of these references are GitHub projects.

  - A patent document that lists a project among its references *cites* the project.
  - Petralia (2025) finds that organizations contribute more to the projects a patent mentions in the month before its filing than in the month before its grant, and Petralia and Boschma (2026) compare each patent that names a project with the same assignee's contributions to it. Here a project counts as cited when a US patent document of any owner, the project's own firm included, lists it among its references, for the OSS-active firms' projects and the comparable projects of other listed firms alike.

- *Finding a citation.* The references come from the publication records of US patent documents in Google's public patent data. Subsections D.2 and D.3 give the extract and the rule in full.
  - I keep every document of any kind published from 2008 and every non-patent reference whose text contains a GitHub address.
  - This gives 25,386 references in 12,682 documents.
  - I read the first GitHub address in each reference and keep it when it names a repository, directly or through the repository's GitHub Pages site.
  - I match the repository's name, ignoring case, to the projects of the firms that the crosswalk links.
  - Of the references, 2,699 match a linked firm's project. Most of the rest name repositories of owners that the crosswalk links to no Compustat firm.

- *The comparison.* The *comparison projects* are the projects of Compustat firms other than the OSS-active firms, so the comparison is with other listed firms' code rather than with all of GitHub.
  - A project belongs to the firm that owns its account in the project's creation year.
  - This sample and owner rule differ from the participation analysis of section 4, which takes repositories first active in 2015–2022 and assigns each to its firm in its first-activity year.
  - With forks and projects created before 2008 set aside, the comparison holds 85,084 projects of the OSS-active firms and 36,208 comparison projects.
  - Both groups keep projects that are not forks, were created in 2008–2024 and record stars and a programming language.
  - I compare the two groups within strata of creation year, stars and programming language. The strata pool 2008–2012 and then take each year, put stars in five bands from zero to a thousand or more, and take the twelve most common languages among the OSS-active firms' projects plus a group for the rest.
  - For the comparison projects, I weight each stratum's share by the OSS-active firms' share of projects in that stratum.

  - Within strata, I also estimate a linear probability model of being cited on an indicator for the OSS-active firms' projects, with stratum effects and standard errors clustered by the firm that owns the project.

- *The two shares.* Among the compared projects, 0.65% of the OSS-active firms' projects are cited by at least one patent document, against 0.179% of the comparison projects with the OSS-active firms' mix of strata (Table 24).

  - Within strata, the difference is 0.21 percentage points (95 percent interval 0.13 to 0.3).
  - The regression weights each stratum by how much the indicator for the OSS-active firms varies within it, so its difference need not equal the gap between the two shares.
  - With citing documents of the project's own firm left out, the difference within strata is 0.151 percentage points. Documents whose owner the crosswalk does not identify, most of them grants of 2025 and 2026, count here as another owner's.
  - Counting only the references that the examiner's search added, the difference is 0.026 points, and the comparison projects' share is zero.

- *The hand check.* I read 100 matched and 100 unmatched references, drawn at random by a rule fixed before any was read.

  - Of the drawn matched references, 99 are still matches under the current links, and of these 98 name the project they are matched to (Wilson 95 percent interval 94.5 to 99.8 percent), one is unclear and none is wrong.
  - Each of the 100 unmatched references names code outside the linked projects or only an owner, a site or a GitHub page.
  - A reading of every GitHub address in all 22,687 unmatched references finds five that name an OSS-active firm's project the rule misses. One of their three projects is among the compared projects and not already counted, and with it the OSS-active firms' share stays 0.65%.

- *Limits.* Citations are rare in both groups, and the comparison rests on few.

  - The OSS-active firms' share rests on 556 cited projects of 85,084, and the comparison's on 27 of 36,208 before the strata reweight it.
  - A citation records that one document lists the project, a link between a patent document and a project and not a legal finding. It leaves open whether an examiner relied on the project, whether it limited a claim, and why it was listed.
  - The strata hold creation year, stars and language fixed, and the OSS-active firms also differ from other listed firms in size, in how much they patent, and in how they name and host code.

– The match reads first addresses only and matches names on their current form. Code in organizations that the account crosswalk leaves unlinked to their firm counts as code outside the linked projects (Appendix D.6).

Table 24: Projects cited by US patent documents

	OSS-active firms' projects	Comparison projects	Difference within strata (percentage points)
Projects compared	85,084	36,208	
Cited at least once	556	27	
Share cited (percent)			
By any document	0.653	0.179	0.213
			[0.130, 0.296]
By a document of another owner	0.561	0.166	0.151
			[0.087, 0.215]
Through the examiner's search	0.063	0.000	0.026
			[0.015, 0.037]

Notes: The projects compared are those that are not forks and were created in 2008–2024, with stars and a programming language. The comparison projects belong to Compustat firms other than the OSS-active firms. The comparison share weights each stratum's share by the OSS-active firms' share of projects in that stratum, over strata of creation year, stars and language. The difference within strata is from a linear probability model with stratum effects, and brackets give 95 percent intervals with standard errors clustered by the owning firm. The regression weights the strata differently from the comparison share, so the difference need not equal the gap between the two shares. A document of another owner is one that the crosswalk does not assign to the project's own firm. The examiner's search counts only the references the examiner added.

D.2 The extract

- One query of Google's public patent publications keeps US documents of every kind published from 1 January 2008, one row for each non-patent reference whose text contains a GitHub address.
  - It returns 25,386 references from 12,682 documents.
  - By kind, 21,060 references come from utility grants of the B2 kind, 3,284 from grants of the B1 kind, 628 from published applications, 406 from design patents and 8 from reissues.
  - The query names no project or firm, and the match to firms uses only the returned references.
  - Its 15,210 references from grants of 2015–2024 equal the count of an independent extraction of the same table.

D.3 Reading an address and matching a project

- Each reference's text is parsed for its first GitHub address, and the address gives a repository's name.
  - An address of the form github.com/owner/repository names a repository, and so does a GitHub Pages address, owner.github.io/repository.
  - Names are set in lower case, and a trailing `.git` and trailing punctuation are removed.
  - Gists, addresses that name only an owner, and GitHub's own site pages form classes of their own, outside the projects.
  - Table 25 gives the classes.
- A parsed name is matched, ignoring case, to the full name of a repository among the projects of the Compustat firms that the crosswalk links.
  - That universe holds 186,103 projects with metadata, 139,145 of the OSS-active firms and 46,958 of other firms.
  - A project belongs to the firm that owns its account in the project's creation year under dated ownership.

Table 25: References by what their GitHub address names, and by match

Panel A. What the address names			*Panel B. The match*		
	Count	Percent		Count	Percent
A repository	18,936	74.6	A project of a linked firm	2,699	10.6
A GitHub Pages site	4,816	19.0	of an OSS-active firm	2,615	10.3
An owner only	697	2.7	of another firm	84	0.3
No usable address	615	2.4	A repository of an unlinked owner	21,053	82.9
A gist	279	1.1	An owner only	697	2.7
A GitHub site page	43	0.2	No project	937	3.7
All references	25,386		All references	25,386	

Notes: References are rows of the extract, one per non-patent reference that contains a GitHub address. Percent is each row's share of all references. Of the references that name only an owner, 37 name an owner whose projects are in the universe, and none is attributed to a project. "No project" covers gists, site pages and references with no usable address.

D.4 Known misses and false matches

- The rules imply misses and false matches, and the two checks below count some of them.
  - A reference that names two projects counts once, since only its first address is read.
  - A project renamed or moved to another owner after it was cited is matched on its current name, so a citation of the old name misses.

  - References to code hosted outside GitHub, documents from other offices, documents published before 2008 and references whose text is missing from the public table lie outside the extract.
  - On the other side, a GitHub Pages path that is not a repository's name can match a repository of the same name.
  - A name reused after a deletion or rename can point to a different project, and where two repositories share a name the match keeps one.

- For 0.73% of the citations of the OSS-active firms' projects, the citing document was filed in a year before the project's creation year.
  - A renamed repository, a filing date inherited from an earlier application, or a wrong match would each give that order, and the data do not separate them.

- *The hand check.* A rule fixed before any reference was read draws 100 matched and then 100 unmatched references from the extract in its stored order, with a fixed seed.
  - Every drawn reference is read, with none replaced.
  - Of the 100 matched references, 99 name the project they are matched to (Wilson 95 percent interval 94.6 to 99.8 percent), and 99 of the matched projects belong to OSS-active firms.
  - Under the current links, 99 of the 100 drawn matched references are still matches, 98 of them name the project they are matched to (Wilson 95 percent interval 94.5 to 99.8 percent), and the bound on false matches below is computed on them.
  - The one unclear reference, in US 10,783,017 B2, gives the address of an archived project site while its title names a different work.
  - Of the 100 unmatched references, 94 name code outside the linked projects and 6 name only an owner, a site or a GitHub page.
  - In 3 of the 100 the rule misreads the old Pages form, owner.github.com/repository, and in 2 it reads a documentation page of GitHub as a repository's name.
  - Each of those names lies outside the universe.

- *Every unmatched reference.* The second check reads every GitHub address in each of the 22,687 unmatched references, every occurrence and not only the first.
  - It reads six forms. They are `github.com/owner/repo`, the two GitHub Pages forms, the old Pages form, the raw-file form `raw.githubusercontent.com/owner/repo` and the interface form `api.github.com/repos/owner/repo`.
  - Each name is cleaned as the rule cleans it and tested against every project name in the universe, and each hit is read by hand.

- Table 26 lists the 5 references that name an OSS-active firm's project the rule misses.
- Each of the 5 lists that project, and they are 0.19% of the 2,615 matched references to the OSS-active firms' projects.
- Each unmatched reference that names a project of the universe names an OSS-active firm's project, so this check finds no missed citation of a comparison project.

Table 26: References that name an OSS-active firm's project the rule misses

Grant	Project (firm)	Why the rule misses it	Already counted as cited
US 11,527,056 B2	NVlabs/FUNIT (NVIDIA)	A later github.com address, after another project's	No
US 12,056,743 B2	NVlabs/FUNIT (NVIDIA)	The same reference text as US 11,527,056 B2	No
US 12,619,654 B2 (two references)	google-deepmind/dm-haiku (Alphabet)	A later github.com address, after one with the project's archived old name	Yes
US 11,494,348 B2	MicrosoftDocs/winrt-api (Microsoft)	An api.github.com address, a form the rule leaves unread	Yes, outside the compared projects

Notes: The five references come from four grants and name three projects. Only NVlabs/FUNIT is among the 85,084 compared projects of the OSS-active firms and not already counted as cited. Counting it gives 557 cited projects, and the share stays 0.65%. The shares of Appendix D.1 count the projects as the rule matches them.

- *What the checks leave uncounted.* Misses through renamed projects stay uncounted, since no history of repository names is held.
  - The check of every unmatched reference finds google-deepmind/dm-haiku only because the same reference also gives the project's new name.
  - Across all 2,699 matched references, the hand check bounds the share of false matches at 5.5 percent, the upper end of its interval.
- *Organizations the crosswalk misses.* The account crosswalk leaves some GitHub organizations that linked firms run unlinked to their firm, such as pair-code, Google's People + AI Research organization.
  - A citation of code in such an organization counts as a citation of code outside the universe.
  - The gap lies in the crosswalk's coverage rather than in the match.
  - Only the hand check's sample counts them, and among its 100 unmatched references, 1 names such an organization.

D.5 Examples

- Table 27 lists five citations drawn at random from the 2,615 references matched to OSS-active firms' projects, by a rule fixed before the draw, as Marx and Fuegi (2020) draw the references

they list.

  - Each matched reference had the same chance of being drawn, and each row gives the address as the reference prints it.

Table 27: Examples of projects cited by patent documents

Project (firm)	Citing document	Address as the reference prints it	Title
azure/storage (Microsoft)	US 12,730,778 B2	https://azure.github.io/Storage/docs/storage-partners/partners/MultiProtocol/Caringo-Azure.pdf	Not held
azure/autorest (Microsoft)	US 10,824,489 B2	https://web.archive.org/web/20170604204029/https://github.com/Azure/autorest	Dynamic node rebalancing between container platforms
microsoft/vsextensibility (Microsoft)	US 12,353,394 B2	https://github.com/microsoft/VSExtensibility/	Using a complex value-bearing language expression to generate metadata associated with an artifact
netflix/hystrix (Netflix)	US 11,640,350 B2	https://github.com/Netflix/Hystrix/,2013	Automated test input generation for integration testing of microservice-based web applications
google/snappy (Alphabet)	US 12,699,877 B2	https://github.com/google/snappy	Not held

Notes: The rows are drawn at random, in the order drawn, from the references that the rule matches to a project of an OSS-active firm. The firm is the project's firm, and the address is the token of the reference that holds the GitHub address. Titles come from PatentsView's grant records. Two grants are later than PatentsView's release, so their titles are not held.

D.6 Limits

- *Citations are rare, and the comparison rests on few.* The comparison share rests on 27 cited projects of 36,208, and on 32 in the whole universe.
  - Through the examiner's search, the comparison projects' share is zero.
- *A citation is a listing.*
  - A citation records that the applicant or the examiner listed the project.
  - It leaves open whether the examiner relied on it, whether it limited a claim, and why it was listed.
- *Own citations.* The project's own firm holds 294 of the citing documents, and the share cited by a document of another owner in Table 24 leaves them out.

- *Unidentified citers.* The crosswalk identifies no owner for many citing grants, most of them of 2025 and 2026, and the share cited by a document of another owner in Table 24 counts them as another owner's.
- *Comparability.* The OSS-active firms differ from other listed firms beyond creation year, stars and language, and the strata hold only those three fixed.
  - The differences include size, how much the firms patent, and how they name and host code.
- *Matching.* Only first addresses are read, names are matched on their current form, and addresses that name only an owner are left unattributed.
  - The hand check finds 98 of the 99 drawn matches that remain under the current links real.
  - The check of every unmatched reference finds 5 misses that name 3 projects (section D.4).
  - A citation whose only address gives a renamed project's old name is missed and stays uncounted.
- *The crosswalk's coverage of firms' accounts.* Some organizations that OSS-active firms run are left unlinked to their firm, and code in them counts as code outside the universe.
- *Coverage.* The data cover US documents published from 2008 and code on GitHub, and the last year is partial.

E Lin and Rai (2024) beside the participation result

- Lin and Rai (2024) study how firms' open-source activity changed after the *Alice* decision narrowed the scope of software patents.
  - This appendix sets their study beside the participation result of section 4.
  - The two studies differ in their exposure, their outcome and their comparison, so neither speaks directly to the other.

E.1 The two studies side by side

Table 28: Lin and Rai (2024) and the participation result

	Lin and Rai (2024)	This paper
Data and period	Compustat firms linked to GitHub through the firm's website and to USPTO patents, in a panel of 2010–2018	GH Archive events of 2015–2024 on the organizations of linked firms, with grants linked by the patent crosswalk and repositories labeled in CPC main groups
Unit	The firm-quarter: 75,885 observations of 3,062 public firms	The repository, over the 24 months after its first-activity quarter: 102,980 repositories of 686 firms in columns 1 and 2
Outcome	The firm's commits on all its public repositories, by anyone, with no split between the firm's staff and others	People new to the repository and their contributions. The founding-quarter actors never count, and pushes, which carry commits, are not counted
Exposure	The narrowing of software-patent scope by *Alice* in June 2014, assigned by industry, with prepackaged software (SIC 7372) most exposed	A repository's overlap with its own firm's grants in the five years up to its start, with no change in law and no enforcement
Design	A difference-in-differences across firms and quarters, with firm and quarter effects and standard errors clustered by firm, with checks of pre-trends, an alternative treatment group and a matrix-completion estimator	A comparison conditional on firm-by-year and main-group-by-year effects and the founding controls, by Poisson pseudo-maximum likelihood with standard errors clustered by firm and Holm's adjustment across outcomes
Comparison group	Firms outside software, though some of them may also rely heavily on software	The firm's other repositories, conditional on firm and year effects in column 1, and on firm-by-year and main-group-by-year effects and the founding controls in column 4
Main finding	Open-source activity of software firms rose by 39.2 percent after *Alice*, mainly among firms with a large stock of software patents beforehand, and their software patenting fell	Inside repositories have 29.5 percent more new participants and 45.5 percent more contributions than other repositories, conditional on firm-by-year and main-group-by-year effects and the founding controls

Notes: Page numbers refer to Lin and Rai (2024). Their 39.2 percent is a coefficient on the log of one plus commits, which their text reads as a percent. This paper's entries are $100(e^{\beta} - 1)$ from Table 4.

E.2 What Lin and Rai say about participation

- Lin and Rai make three statements about participation, each of which motivates a hypothesis or summarizes earlier work rather than reporting their own estimate.
  - They suggest that patenting software could deter open communities from contributing, citing Lerner and Tirole (2005) among others.
  - They summarize earlier work as finding that projects tied to patented inventions attract less interest from users, citing Bar-Gill and Parchomovsky (2003) and Wen et al. (2013).
  - Their next sentence turns to a firm that leans less on software patents, rather than to a project's overlap with its firm's patents.
  - In the argument for their moderator, they expect less participation at firms that put more weight on patenting.
- In their discussion, Lin and Rai set their firm-level finding beside Wen et al. (2013), who report lower user interest and developer activity under the enforcement of intellectual property rights.
  - Of the earlier work behind that summary, the study they set beside their own concerns enforcement, while Lin and Rai's own exposure is the narrowing of scope.

E.3 Where the two differ

- *The variation.* Lin and Rai compare firms before and after a legal change, while this paper compares repositories of one firm started in the same year.
  - Here, the firm effects absorb every difference between firms.
- *The exposure.* Their exposure is a narrowing of what software patents can claim and enforce, assigned to firms by industry.
  - Here, the exposure is where a firm's own repository sits relative to its own recent grants.
- *The outcome.* Their outcome counts every commit on a firm's repositories, the firm's staff included.
  - This paper counts people new to a repository and their issues, pull requests, reviews and comments.
  - It leaves out the founding-quarter actors.
- *The window.* Their panel ends in 2018, and the events here start in 2015, after *Alice*, so the two windows overlap only in 2015–2018.
- *The scale.* Their 39.2 percent is a coefficient on the log of one plus commits.

  - A linear regression of the log of one plus the count on the inside indicator, with the effects of column 1, gives coefficients of 0.241 for new participants and 0.353 for contributions.
  - The two sets of coefficients measure different exposures and outcomes, so their sizes are not comparable.

E.4 The checks of section 4 beside Lin and Rai

- *The level of technology.* Lin and Rai give each firm one dose, set by its industry, and a robustness check widens the treated group to software-intensive firms.
  - Here, inside is assigned repository by repository from the firm's own grants at every CPC level (Appendix C.2).
  - A coarser level changes how many repositories count as inside, and at the subclass the comparison rests on the few left outside. At the subgroup, one level finer, the inside difference in participation stays close to the main group's, and the inside difference for contributions is smaller.
  - At every level, the firm effects absorb the differences between firms that Lin and Rai's design uses, and no CPC level corresponds to their industry dose or their moderator.
- *Who is counted.* Lin and Rai count every commit on a firm's repositories, while their explanation of why firms open code rests on contributions from outside the firm.
  - Here, new participants can be split by their history and by what they do next (Appendix C.4).
  - The groups nearest their outside contributors are the people with a history at no linked firm and the people who go on to none of the firm's other repositories. For the latter, the column-4 inside difference is 21.5 percent, which is significant. The reading for people with a history at no linked firm is fragile, and no group's inside difference is distinguishable from another's.
  - These groups are built from event histories, and no actor is matched to an employer, so they leave employment unobserved and leave open who made Lin and Rai's commits.
  - The outcomes differ in kind. The counts here leave out pushes, the events that carry commits, so Lin and Rai's outcome lies outside them.
- *Time.* Lin and Rai's panel is quarterly, and the article reports its dynamics as a check of pre-trends, with the estimates in an online appendix.
  - The arrival profile here runs by the repository's age since its first observed activity, and every repository starts after *Alice*, so the two time axes measure different things and support no dynamic comparison.

- *The moderator.* Lin and Rai's moderator is a firm's stock of software patents before *Alice*, and the rise in open-source activity concentrates among firms with a large stock.
  - Here, the firm effects absorb any stock held at the level of the firm.
  - The nearest analogue within the firm is its patent weight in the repository's primary main group, from its grants of the five years up to the repository's first activity.
  - Its interaction with inside misses Holm's adjustment for every outcome (Appendix C.6). That patent weight is a within-firm quantity, and their stock is compared across firms.
  - The firms here are firms linked to code rather than an industry, so their software-firm group has no counterpart.
- *Their statement on projects tied to patented inventions.* It is the statement nearest the unit here, a project.
  - At the CPC subgroup, inside is the narrowest overlap held, and the inside difference stays positive and passes Holm's adjustment. That overlap is still a field shared with the firm's grants, not a project built on a patented invention.
  - The nearest observable to the user interest of the earlier work is stars. Stars are 44.0 percent higher inside in column 1 and 7.1 percent in column 4. The first passes Holm's adjustment only at the ten-percent level, the second is not significant, and stars are not equated with that interest.
  - Their other two statements, on deterrence and on their moderator, concern how much a firm patents, across firms, and none of the checks here measures that.
- The checks describe the inside difference here more fully. The two studies concern different exposures, and the checks claim neither agreement nor conflict with Lin and Rai.

E.5 What each can say about the other

- Lin and Rai's estimate is a firm-level change in total commits after a legal change, and their statements on participation cite earlier work.
  - Their paper measures activity at the level of the firm, and it reports no measure of how participation is spread across one firm's repositories by technology.
- In the other direction, this paper compares repositories within a firm, on events that begin in 2015, after *Alice*.
  - Its firm effects absorb the difference between firms that rely more and less on patents, which is the comparison of Lin and Rai's statement on their moderator.
  - Its outcome counts new arrivals, while their commit count includes the firm's own activity.
- The two studies share their source, firms' public repositories on GitHub.